Modul 7: CRUD dan Upload File pada Laravel

7.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep CRUD (Create, Read, Update, Delete), resource controller, dan resource route pada Laravel.
2. Memahami konsep unggah berkas (file upload) pada Laravel, meliputi form multipart, validasi file, disk public, dan symbolic link storage.
3. Mampu membangun fitur CRUD yang dilengkapi upload file menggunakan Laravel Breeze, sehingga data hanya dapat dikelola oleh pengguna yang sudah login.
4. Mampu membangun fitur CRUD yang dilengkapi upload foto secara manual tanpa Breeze menggunakan layout Blade dan Bootstrap.
5. Mampu menampilkan, mengganti, dan menghapus file yang tersimpan di folder storage/app/public melalui antarmuka aplikasi.
6. Mampu menerapkan validasi input serta menampilkan pesan berhasil dan pesan galat yang jelas kepada pengguna.

7.2 Alat dan Bahan

Untuk menjalankan modul ini, diperlukan:

1. Perangkat Keras:
   - Komputer atau laptop dengan sistem operasi Windows/Linux/MacOS.
   - Minimal prosesor 1 GHz, RAM 4GB (disarankan 8GB), serta ruang penyimpanan kosong minimal 2GB.
2. Perangkat Lunak:
   - XAMPP atau Laragon sebagai server lokal (Apache & MySQL/MariaDB), termasuk phpMyAdmin untuk mengelola database.
   - Composer sebagai dependency manager PHP (sudah terinstal).
   - PHP versi 8.2 atau lebih tinggi (sudah termasuk dalam XAMPP versi terbaru). • Laravel Framework yang sudah terinstal melalui Composer.
   - Laravel Breeze beserta Node.js dan NPM untuk meng-compile asset tampilan Breeze (cara pemasangan dijelaskan pada subbab 7.4.4).
   - Teks Editor / IDE seperti Visual Studio Code atau PhpStorm.
   - Browser Web seperti Google Chrome atau Mozilla Firefox untuk menjalankan aplikasi.
   - Beberapa file contoh untuk diuji coba, misalnya gambar (.jpg/.png) dan dokumen (.pdf/.docx) berukuran di bawah 2 MB.

7.3 Tugas Pendahuluan

1. Apa yang dimaksud dengan CRUD? Jelaskan contoh penerapan setiap operasinya pada sebuah aplikasi web!
2. Apa yang dimaksud dengan unggah berkas (file upload), dan bagaimana Laravel

menanganinya dibandingkan dengan PHP murni?

3. Mengapa form untuk unggah berkas wajib menggunakan atribut enctype="multipart/form data"?
4. Apa perbedaan menyimpan berkas di server (storage/app/public) dengan menyimpan path berkas di database? Mengapa keduanya diperlukan?
5. Mengapa validasi jenis dan ukuran file penting dilakukan sebelum berkas disimpan di server?

7.4 Materi Praktikum

7.4.1 Pengertian CRUD

CRUD merupakan singkatan dari Create, Read, Update, dan Delete, yaitu empat operasi dasar untuk mengelola data di dalam database. Hampir semua aplikasi web memiliki fitur CRUD, misalnya aplikasi data mahasiswa, aplikasi laporan, atau aplikasi toko online. Agar mudah dibayangkan, CRUD ibarat buku catatan kontak di ponsel: kita dapat menambah kontak baru, melihat daftar kontak, mengubah nomor yang salah, dan menghapus kontak yang sudah tidak diperlukan. Keempat operasi tersebut adalah:

- **Create** → menambahkan data baru ke database, misalnya mengisi form tambah kontak lalu menekan tombol Simpan (perintah SQL INSERT).
- **Read** → menampilkan data dari database, baik dalam bentuk daftar (tabel) maupun detail satu data (perintah SQL SELECT).
- **Update** → mengubah data yang sudah ada, misalnya memperbaiki alamat atau mengganti foto (perintah SQL UPDATE).
- **Delete** → menghapus data dari database, misalnya menghapus kontak yang sudah tidak digunakan (perintah SQL DELETE).

7.4.2 Konsep Upload File pada Laravel

Unggah berkas (file upload) adalah proses mengirim file dari komputer pengguna ke server melalui form, misalnya mengunggah foto profil atau dokumen laporan. Laravel menyediakan fitur bawaan yang membuat proses ini jauh lebih ringkas dibandingkan PHP murni yang harus mengolah $_FILES dan move_uploaded_file() secara manual. Sebelum praktik, pahami terlebih dahulu beberapa istilah berikut:

- enctype="multipart/form-data" → atribut wajib pada tag <form> agar isi file ikut terkirim ke server. Tanpa atribut ini, yang terkirim hanya nama file sehingga $request->file() bernilai null. • $request->file('nama_input') dan $request->hasFile('nama_input') → mengambil file yang diunggah dan memeriksa apakah pengguna benar-benar memilih file.
- Validasi file → aturan seperti required, file, image, mimes:jpg,png,pdf, dan max:2048 memastikan hanya jenis file tertentu dengan ukuran maksimal 2 MB (2048 KB) yang diterima. Validasi ini mencegah pengguna mengunggah file berbahaya, misalnya file .php.
- store('folder', 'public') → menyimpan file ke folder storage/app/public/folder dengan nama acak yang unik, lalu mengembalikan path file (misalnya uploads/a1b2c3.jpg). Path inilah

yang disimpan di database, bukan isi file-nya.

- Disk public dan php artisan storage:link → folder storage/app/public tidak dapat diakses langsung dari browser. Perintah storage:link membuat jalan pintas (symbolic link) public/storage yang mengarah ke folder tersebut, sehingga file dapat ditampilkan menggunakan asset('storage/' . $path).
- Facade Storage → digunakan untuk mengelola file yang sudah tersimpan, misalnya Storage::disk('public')->delete($path) untuk menghapus file lama ketika data diubah atau dihapus.

Secara singkat, file disimpan di folder storage, sedangkan database hanya menyimpan path (alamat) file tersebut. Dengan cara ini, ukuran database tetap kecil dan file dapat dipanggil kembali kapan saja melalui path-nya.

7.4.3 Resource Controller dan Resource Route

Laravel menyediakan resource controller, yaitu controller yang secara otomatis berisi tujuh method standar untuk operasi CRUD. Resource controller dibuat dengan perintah php artisan make:controller NamaController --resource, sedangkan seluruh route-nya cukup didaftarkan dengan satu baris Route::resource(). Sebagai contoh, Route::resource('kontaks', KontakController::class) akan menghasilkan route berikut:

• GET /kontaks → index() → menampilkan daftar data (route kontaks.index). • GET /kontaks/create → create() → menampilkan form tambah data (route kontaks.create). • POST /kontaks → store() → menyimpan data baru ke database (route kontaks.store). • GET /kontaks/{kontak} → show() → menampilkan detail satu data (route kontaks.show). • GET /kontaks/{kontak}/edit → edit() → menampilkan form ubah data (route kontaks.edit). • PUT/PATCH /kontaks/{kontak} → update() → menyimpan perubahan data (route kontaks.update).

• DELETE /kontaks/{kontak} → destroy() → menghapus data (route kontaks.destroy).

Karena form HTML hanya mendukung method GET dan POST, operasi update dan delete dikirim menggunakan method POST yang ditambahkan directive @method('PUT') atau @method('DELETE'). Anda dapat melihat seluruh route yang terdaftar dengan perintah php artisan route:list.

7.4.4 CRUD dan Upload File Menggunakan Laravel Breeze

Pada subbab ini, kita akan membuat fitur pengelolaan berkas (CRUD Berkas) menggunakan Laravel Breeze, dimulai dari pembuatan project Laravel baru. Setiap pengguna yang sudah login dapat mengunggah berkas (gambar atau dokumen), melihat daftar berkas miliknya, mengubah judul atau mengganti file, serta menghapus berkas. Karena seluruh route diletakkan di dalam middleware auth, halaman ini tidak dapat dibuka sebelum login.

Tahap A: Menyiapkan Project Baru dan Memasang Laravel Breeze

1. Buka terminal (Command Prompt, PowerShell, atau terminal Visual Studio Code) di folder

tempat Anda menyimpan project, misalnya C:\xampp\htdocs atau folder www pada Laragon. Pastikan Apache dan MySQL pada XAMPP/Laragon sudah dijalankan (Start).

2. Buat project Laravel baru bernama crud-breeze dengan perintah berikut:

```
composer create-project laravel/laravel crud-breeze
```

Perintah ini mengunduh Laravel versi terbaru beserta seluruh dependency-nya ke dalam folder crud-breeze. Proses ini membutuhkan koneksi internet dan dapat memakan waktu beberapa menit. Jika sudah menginstal Laravel Installer, perintah laravel new crud-breeze juga dapat digunakan (pilih None ketika ditanya starter kit).

3. Masuk ke folder project lalu buka di Visual Studio Code:

```
cd crud-breeze
code .
```

Seluruh perintah pada langkah berikutnya dijalankan di dalam folder crud-breeze. Agar lebih mudah, gunakan terminal bawaan Visual Studio Code (menu Terminal → New Terminal).

4. Buka phpMyAdmin (http://localhost/phpmyadmin), klik New, lalu buat database baru bernama **db_crud_breeze**.

5. Buka file .env di folder project, lalu ubah konfigurasi database menjadi seperti berikut dan simpan (Ctrl + S):

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_crud_breeze
DB_USERNAME=root
DB_PASSWORD=
```

• DB_CONNECTION=mysql → secara bawaan Laravel versi terbaru menggunakan SQLite, sehingga perlu diganti menjadi mysql agar terhubung ke MySQL/MariaDB pada XAMPP. • Hapus tanda # di depan baris DB_HOST sampai DB_PASSWORD jika baris tersebut masih berupa komentar.

• DB_DATABASE → disesuaikan dengan nama database yang dibuat pada langkah sebelumnya. DB_PASSWORD dibiarkan kosong untuk XAMPP bawaan.

6. Instal package Laravel Breeze dengan perintah berikut:

```
composer require laravel/breeze --dev
```

Opsi --dev berarti Breeze hanya dibutuhkan selama pengembangan, karena Breeze cukup menyalin file authentication ke dalam project.

7. Pasang Breeze dengan tampilan Blade:

```
php artisan breeze:install blade
```

Perintah ini membuat route, controller, dan view untuk register, login, logout, lupa password, dan profil, serta file routes/auth.php. Jika muncul pertanyaan tambahan (dark mode atau framework testing), tekan Enter untuk memilih jawaban bawaan.

8. Unduh package front-end lalu jalankan Vite pada terminal pertama:

```
npm install
npm run dev
```

npm install cukup dijalankan satu kali, sedangkan npm run dev harus tetap berjalan selama pengembangan agar CSS Tailwind dan JavaScript Breeze dapat dimuat. Jangan tutup terminal ini. Jika npm tidak dikenali, lihat Modul 6 subbab 6.4.9.

9. Buka terminal kedua (split terminal), lalu jalankan migration untuk membuat tabel bawaan Laravel dan Breeze:

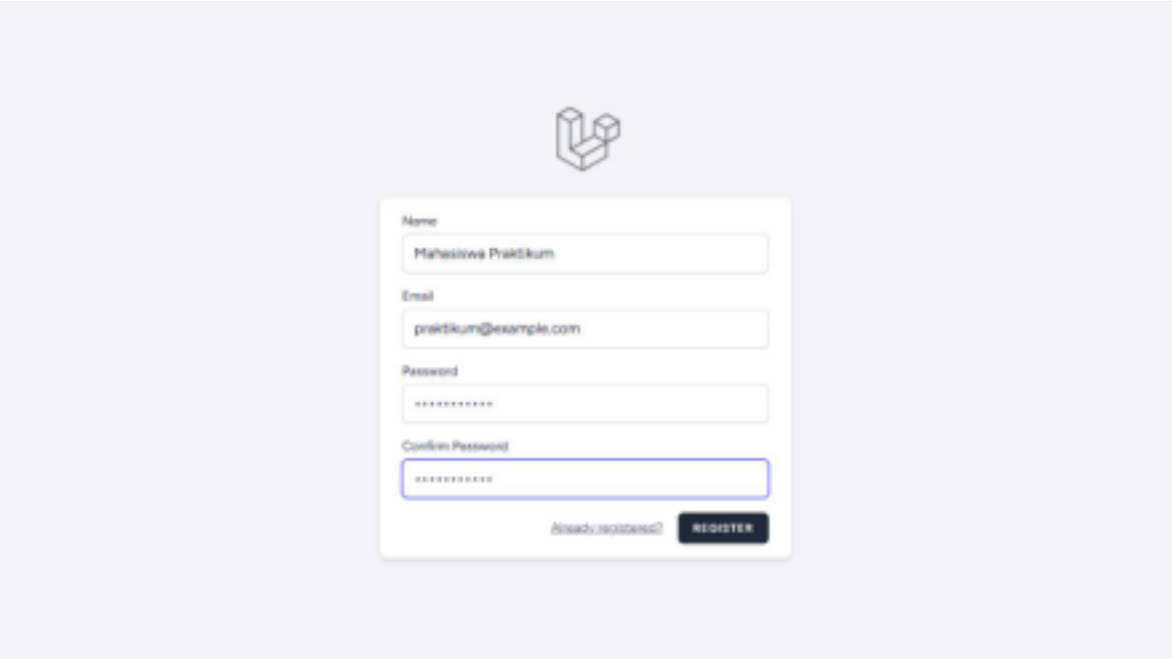
```
php artisan migrate
```

Perintah ini membuat tabel users, password_reset_tokens, sessions, cache, dan jobs di database db_crud_breeze. Periksa melalui phpMyAdmin bahwa tabel-tabel tersebut sudah terbentuk.

10. Jalankan server Laravel pada terminal kedua:

```
php artisan serve
```

Buka http://127.0.0.1:8000 di browser, klik Register, lalu daftarkan satu akun. Jika berhasil masuk ke halaman Dashboard, berarti project dan Breeze sudah siap digunakan. Akun ini akan dipakai untuk menguji CRUD Berkas pada tahap berikutnya.



Gambar 7.1 Form Register Laravel Breeze

Halaman register bawaan Breeze yang digunakan untuk mendaftarkan akun pengguna. **Tahap B: Membuat CRUD Berkas**

1. Buat model Upload beserta migration-nya dengan menjalankan perintah berikut di terminal:

```
php artisan make:model Upload -m
```

   Opsi -m membuat file migration secara bersamaan. Perintah ini menghasilkan file app/Models/Upload.php dan database/migrations/xxxx_xx_xx_xxxxxx_create_uploads_table.php. Nama model ditulis dalam bentuk tunggal dengan huruf awal kapital, sedangkan nama tabel otomatis menjadi bentuk jamak (uploads).

2. Buka file migration database/migrations/xxxx_xx_xx_xxxxxx_create_uploads_table.php, lalu ubah method up() menjadi seperti berikut:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
 public function up(): void
 {
 Schema::create('uploads', function (Blueprint $table) {
$table->id();
      $table->foreignId('user_id')->constrained()-
          >cascadeOnDelete(); // Pemilik berkas
 $table->string('judul'); // Judul berkas
$table->text('keterangan')->nullable(); // Keterangan  (boleh
kosong)
 $table->string('filename'); // Nama asli file
$table->string('filepath'); // Lokasi file di  storage
 $table->timestamps();
 });
 }

 public function down(): void
 {
 Schema::dropIfExists('uploads');
 }
};
```

- foreignId('user_id')->constrained() → membuat kolom user_id yang terhubung ke tabel users, sehingga setiap berkas memiliki pemilik.
- cascadeOnDelete() → jika akun pengguna dihapus, seluruh data berkas miliknya ikut terhapus.
- filename → menyimpan nama asli file yang diunggah (misalnya laporan.pdf) agar mudah dikenali pengguna.
- filepath → menyimpan path file hasil store(), misalnya uploads/Xy12AbCd.pdf. Isi file tetap disimpan di folder storage, bukan di database.
- nullable() → kolom keterangan boleh dikosongkan.

database/migrations/2026_10_25_174709_create_uploads_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('uploads', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->cascadeOnDelete(); // Pemilik berkas
            $table->string('judul');                    // Judul berkas
            $table->text('keterangan')->nullable();     // Keterangan (boleh kosong)
            $table->string('filename');                 // Nama asli file
            $table->string('filepath');                 // Lokasi file di storage
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('uploads');
    }
};
```

Gambar 7.2 Migration tabel uploads

3. Jalankan migration untuk membuat tabel uploads:

```
php artisan migrate
```

Jika berhasil, tabel uploads akan muncul di database dan dapat dilihat melalui phpMyAdmin. Pastikan konfigurasi database pada file .env sudah benar.

4. Buka file app/Models/Upload.php, lalu ganti isinya dengan kode berikut: `<?php`

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
```

```
class Upload extends Model
{
 protected $table = 'uploads'; // Nama tabel yang dihubungkan

 // Kolom yang boleh diisi secara massal (mass assignment)
protected $fillable = [
 'user_id',
 'judul',
 'keterangan',
 'filename',
 'filepath',
 ];

 // Relasi: setiap berkas dimiliki oleh satu user
 public function user()
 {
 return $this->belongsTo(User::class);
 }
}
```

• $fillable → daftar kolom yang boleh diisi menggunakan Upload::create() atau update(). Kolom id dan timestamps tidak perlu dicantumkan karena diisi otomatis. • belongsTo(User::class) → relasi yang menyatakan bahwa setiap berkas dimiliki oleh satu pengguna, sehingga data pemilik dapat dipanggil dengan $upload->user->name.

app/Models/Upload.php

```php
<?php

namespace App\Models;

use Database\Factories\UploadFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Upload extends Model
{
    /** @use HasFactory<UploadFactory> */
    use HasFactory;

    protected $table = 'uploads'; // Nama tabel yang dihubungkan

    /**
     * Kolom yang boleh diisi secara massal (mass assignment).
     *
     * @var list<string>
     */
    protected $fillable = [
        'user_id',
        'judul',
        'keterangan',
        'filename',
        'filepath',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'user_id' => 'integer',
        ];
    }

    /**
     * Relasi: setiap berkas dimiliki oleh satu user.
     *
     * @return BelongsTo<User, $this>
     */
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

Gambar 7.3 Model Upload

5. Buat symbolic link agar file di folder storage dapat diakses dari browser dengan perintah berikut:

```
php artisan storage:link
```

Perintah ini membuat folder pintasan public/storage yang mengarah ke storage/app/public. Jika muncul pesan The [public/storage] link already exists, berarti link sudah pernah dibuat sebelumnya. Perintah ini cukup dijalankan satu kali untuk setiap project.

6. Buat resource controller bernama UploadController dengan perintah berikut:

```
php artisan make:controller UploadController --resource
```

Opsi --resource membuat file app/Http/Controllers/UploadController.php yang sudah berisi tujuh method CRUD (index, create, store, show, edit, update, dan destroy) dalam keadaan kosong.

7. Buka file app/Http/Controllers/UploadController.php, lalu ganti seluruh isinya dengan kode berikut:

```
<?php

namespace App\Http\Controllers;

use App\Models\Upload;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth; // Untuk mengambil data user
         yang sedang login
  use Illuminate\Support\Facades\Storage; // Untuk mengelola
                 (menghapus) file di storage

class UploadController extends Controller
{
 // READ: menampilkan daftar berkas milik user yang sedang login
public function index()
 {
 $uploads = Upload::where('user_id', Auth::id())->latest()-
         >paginate(5);
 return view('uploads.index', compact('uploads'));  }

 // CREATE: menampilkan form unggah berkas
 public function create()
 {
 return view('uploads.create');
 }

 // CREATE: memvalidasi dan menyimpan berkas baru
 public function store(Request $request)
 {
```

```
 $request->validate([
 'judul' => 'required|string|max:255',
 'keterangan' => 'nullable|string',
 'file' =>
         'required|file|mimes:jpg,jpeg,png,pdf,doc,docx|max:2048', //
         maks. 2 MB
 ]);

 $file = $request->file('file'); // Ambil file dari  form
 $path = $file->store('uploads', 'public'); // Simpan ke
         storage/app/public/uploads

 Upload::create([
 'user_id' => Auth::id(),
 'judul' => $request->judul,
 'keterangan' => $request->keterangan,
 'filename' => $file->getClientOriginalName(), // Nama  asli file
 'filepath' => $path, // Path  file di storage
 ]);

 return redirect()->route('uploads.index')->with('success',
         'Berkas berhasil diunggah.');
 }

 // READ: menampilkan detail satu berkas
 public function show(Upload $upload)
 {
 abort_if($upload->user_id !== Auth::id(), 403); // Hanya
         pemilik yang boleh melihat
 return view('uploads.show', compact('upload'));  }

 // UPDATE: menampilkan form edit berkas
 public function edit(Upload $upload)
 {
   abort_if($upload->user_id   !==   Auth::id(),   403);
return view('uploads.edit', compact('upload'));  }

 // UPDATE: menyimpan perubahan (file baru bersifat opsional)
public function update(Request $request, Upload $upload)  {
 abort_if($upload->user_id !== Auth::id(), 403);

 $request->validate([
 'judul' => 'required|string|max:255',
 'keterangan' => 'nullable|string',
 'file' =>
         'nullable|file|mimes:jpg,jpeg,png,pdf,doc,docx|max:2048'
,  ]);
```

```
 $data = $request->only('judul', 'keterangan');

 if ($request->hasFile('file')) { // Jika  user memilih file baru
 Storage::disk('public')->delete($upload->filepath); //  Hapus
         file lama
 $file = $request->file('file');
 $data['filename'] = $file->getClientOriginalName();
$data['filepath'] = $file->store('uploads', 'public');  }

 $upload->update($data);

     return redirect()->route('uploads.index')->with('success',
                 'Berkas berhasil diperbarui.');
 }

 // DELETE: menghapus file di storage dan data di database
public function destroy(Upload $upload)
 {
 abort_if($upload->user_id !== Auth::id(), 403);

 Storage::disk('public')->delete($upload->filepath); // Hapus  file
         fisik
 $upload->delete(); // Hapus  data di tabel uploads

 return redirect()->route('uploads.index')->with('success',
          'Berkas berhasil dihapus.');
 }
}
```

- Upload::where('user_id', Auth::id())->latest()->paginate(5) → mengambil berkas milik pengguna yang sedang login saja, diurutkan dari yang terbaru, dan ditampilkan 5 data per halaman.
- mimes:jpg,jpeg,png,pdf,doc,docx|max:2048 → hanya menerima file gambar dan dokumen dengan ukuran maksimal 2 MB. Jika tidak sesuai, Laravel otomatis kembali ke form sambil membawa pesan galat.
- $file->store('uploads', 'public') → menyimpan file ke folder storage/app/public/uploads dengan nama acak yang unik, lalu mengembalikan path-nya.
- getClientOriginalName() → mengambil nama asli file dari komputer pengguna untuk ditampilkan di halaman daftar berkas.
- Upload $upload (route model binding) → Laravel otomatis mencari data berdasarkan id pada URL, misalnya /uploads/3, sehingga kita tidak perlu menulis Upload::find($id). • abort_if($upload->user_id !== Auth::id(), 403) → menolak akses (galat 403 Forbidden) jika pengguna mencoba membuka atau mengubah berkas milik orang lain.
- Storage::disk('public')->delete(...) → menghapus file lama dari storage ketika file diganti atau data dihapus, agar tidak ada file sampah yang menumpuk di server. •

redirect()->route(...)->with('success', ...) → kembali ke halaman daftar berkas sambil membawa pesan berhasil yang disimpan sementara di session.

app/Http/Controllers/UploadController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Upload;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Storage;
use Illuminate\View\View;

class UploadController extends Controller
{
    /**
     * READ: menampilkan daftar berkas milik user yang sedang login.
     */
    public function index(): View
    {
        $uploads = Upload::where('user_id', Auth::id())->latest()->paginate(5);

        return view('uploads.index', compact('uploads'));
    }

    /**
     * CREATE: menampilkan form unggah berkas.
     */
    public function create(): View
    {
        return view('uploads.create');
    }

    /**
     * CREATE: memvalidasi dan menyimpan berkas baru.
     */
    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'judul' => 'required|string|max:255',
            'keterangan' => 'nullable|string',
            'file' => 'required|file|mimes:jpg,jpeg,png,pdf,doc,docx|max:2048', // maks. 2 MB
        ]);

        $file = $request->file('file');
        $path = $file->store('uploads', 'public'); // Simpan ke storage/app/public/uploads

        Upload::create([
            'user_id' => Auth::id(),
            'judul' => $request->judul,
            'keterangan' => $request->keterangan,
            'filename' => $file->getClientOriginalName(),
            'filepath' => $path,
        ]);

        return redirect()->route('uploads.index')->with('success', 'Berkas berhasil diunggah.');
    }

```

Gambar 7.4 UploadController bagian 1 (index, create, store)



Gambar 7.5 UploadController bagian 2 (show, edit, update, destroy)

8. Buka file routes/web.php, lalu tambahkan route resource uploads di dalam grup middleware auth seperti berikut:

```
<?php

use App\Http\Controllers\ProfileController;
```

```
use App\Http\Controllers\UploadController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
 return view('welcome');
});

Route::get('/dashboard', function () {
 return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
 Route::get('/profile', [ProfileController::class, 'edit'])-
         >name('profile.edit');
 Route::patch('/profile', [ProfileController::class, 'update'])-
         >name('profile.update');
 Route::delete('/profile', [ProfileController::class, 'destroy'])-
         >name('profile.destroy');

 // CRUD Berkas: hanya dapat diakses setelah login
 Route::resource('uploads', UploadController::class);
});

require __DIR__.'/auth.php';
```

- use App\Http\Controllers\UploadController; → mengimpor controller agar dapat digunakan pada route.
- Route::resource('uploads', UploadController::class) → membuat tujuh route CRUD sekaligus (uploads.index, uploads.create, uploads.store, uploads.show, uploads.edit, uploads.update, dan uploads.destroy).
- Diletakkan di dalam Route::middleware('auth')->group(...) → pengguna yang belum login dan mencoba membuka /uploads akan otomatis diarahkan ke halaman login.

routes/web.php

```php
<?php

use App\Http\Controllers\ProfileController;
use App\Http\Controllers\UploadController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');

    // CRUD Berkas: hanya dapat diakses setelah login
    Route::resource('uploads', UploadController::class);
});

require __DIR__.'/auth.php';
```

Gambar 7.6 Route resource uploads di dalam middleware auth

Terminal

```
PS crud_breze> php artisan route:list --name=uploads

  GET|HEAD        uploads ..................... uploads.index › UploadController@index
  POST            uploads ..................... uploads.store › UploadController@store
  GET|HEAD        uploads/create ..... uploads.create › UploadController@create
  GET|HEAD        uploads/{upload} ..... uploads.show › UploadController@show
  PUT|PATCH       uploads/{upload} ..... uploads.update › UploadController@update
  DELETE          uploads/{upload} ..... uploads.destroy › UploadController@destroy
  GET|HEAD        uploads/{upload}/edit ..... uploads.edit › UploadController@edit

                                                          Showing [7] routes
```

Gambar 7.7 Hasil php artisan route:list --name=uploads

Tujuh route resource uploads (index, store, create, show, update, destroy, edit) sudah terdaftar.

9. Tambahkan menu Berkas pada navigasi Breeze. Buka file resources/views/layouts/navigation.blade.php, cari komponen <x-nav-link> untuk Dashboard, lalu tambahkan kode berikut tepat di bawahnya:

```
          <x-nav-link :href="route('uploads.index')"
         :active="request()- >routeIs('uploads.*')">
 {{ __('Berkas') }}
</x-nav-link>
```

Komponen x-nav-link merupakan komponen bawaan Breeze untuk membuat menu navigasi. Atribut :active membuat menu Berkas tampak aktif (bergaris bawah) ketika pengguna sedang membuka salah satu halaman uploads. Untuk tampilan mobile, tambahkan juga komponen <x-responsive-nav-link> dengan isi yang sama pada bagian Responsive Navigation Menu di file yang sama.

resources/views/layouts/navigation.blade.php

```
                <x-nav-link :href="route('dashboard')" :active="request()->routeIs('dashboard')">
                    {{ __('Dashboard') }}
                </x-nav-link>
                <x-nav-link :href="route('uploads.index')" :active="request()->routeIs('uploads.*')">
                    {{ __('Berkas') }}
                </x-nav-link>
            </div>
    ...
        <x-responsive-nav-link :href="route('dashboard')" :active="request()->routeIs('dashboard')">
            {{ __('Dashboard') }}
        </x-responsive-nav-link>
        <x-responsive-nav-link :href="route('uploads.index')" :active="request()->routeIs('uploads.*')">
            {{ __('Berkas') }}
        </x-responsive-nav-link>
```

Gambar 7.8 Menu Berkas pada navigasi Breeze

10. Buat folder baru resources/views/uploads, lalu buat file resources/views/uploads/index.blade.php untuk menampilkan daftar berkas (Read):

```
<x-app-layout>
 <x-slot name="header">
 <h2 class="font-semibold text-xl text-gray-800 leading
        tight">Daftar Berkas</h2>
 </x-slot>

 <div class="py-12">
 <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
 <div class="bg-white shadow-sm sm:rounded-lg p-6">  {{--
Pesan berhasil dari controller --}}
 @if (session('success'))
    <div class="mb-4 p-3 rounded bg-green-100 text green-700">{{
                    session('success') }}</div>
 @endif

 <a href="{{ route('uploads.create') }}"
 class="inline-block mb-4 px-4 py-2 bg-blue-600 text white rounded">+
        Unggah Berkas</a>

 <table class="w-full border text-left">
 <thead class="bg-gray-100">
```

```
 <tr>
 <th class="border p-2">No</th> <th class="border
                            p-2">Judul</th>
 <th class="border p-2">Nama File</th> <th class="border p-2">Tanggal
                            Unggah</th>
                            <th class="border p-2">Aksi</th>
 </tr>
 </thead>
                     <tbody>
 @forelse ($uploads as $upload)
 <tr>
  <td class="border p-2">{{ $uploads- >firstItem() + $loop->index
                              }}</td>
 <td class="border p-2">{{ $upload- >judul }}</td>
 <td class="border p-2">  {{-- Link untuk membuka file di tab  baru
--}}
       <a href="{{ asset('storage/' .  $upload->filepath) }}"
                           target="_blank"
    class="text-blue-600  underline">{{ $upload->filename
                            }}</a>
 </td>
                                 <td class="border p-2">{{ $upload-
         >created_at->format('d-m-Y H:i') }}</td>
  <td class="border p-2">  <a href="{{ route('uploads.show',  $upload)
                 }}" class="text-sky-600">Lihat</a> |
            <a href="{{ route('uploads.edit',  $upload) }}"
                  class="text-yellow-600">Edit</a> |
 <form
         action="{{ route('uploads.destroy', $upload) }}" method="POST"
         class="inline">
 @csrf
                                          @method('DELETE')
                                           <button type="submit"
         class="text-red-600"
 onclick="return  confirm('Yakin ingin menghapus berkas
ini?')">Hapus</button>  </form>
 </td>
 </tr>
 @empty
 <tr>
  <td colspan="5" class="border p-2 text center">Belum ada berkas yang
                            diunggah.</td>
 </tr>
 @endforelse
 </tbody>
 </table>
```

```
 <div class="mt-4">{{ $uploads->links() }}</div>  </div>
 </div>
 </div>
</x-app-layout>
```

• <x-app-layout> dan <x-slot name="header"> → menggunakan layout bawaan Breeze (resources/views/layouts/app.blade.php) sehingga halaman otomatis memiliki navigasi, menu profil, dan tombol Log Out.

• @forelse ... @empty ... @endforelse → menampilkan data berkas satu per satu, atau menampilkan pesan Belum ada berkas yang diunggah jika data masih kosong. • {{ $uploads->firstItem() + $loop->index }} → membuat nomor urut yang tetap berlanjut pada halaman pagination berikutnya.

- asset('storage/' . $upload->filepath) → membuat URL file, misalnya http://127.0.0.1:8000/storage/uploads/Xy12AbCd.pdf. URL ini hanya dapat dibuka jika storage:link sudah dijalankan.
- @method('DELETE') → mengubah form POST menjadi request DELETE agar diarahkan ke method destroy(), sedangkan onclick="return confirm(...)" menampilkan konfirmasi sebelum data dihapus.

• {{ $uploads->links() }} → menampilkan tombol navigasi halaman (pagination).



Gambar 7.9 View index.blade.php (daftar berkas)

11. Buat file resources/views/uploads/create.blade.php untuk form unggah berkas (Create):

```
<x-app-layout>
 <x-slot name="header">
 <h2 class="font-semibold text-xl text-gray-800 leading
         tight">Unggah Berkas</h2>
```

```
 </x-slot>

 <div class="py-12">
       <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">  <div
 class="bg-white shadow-sm sm:rounded-lg p-6">  {{-- enctype wajib
        ada agar file ikut terkirim --}}  <form action="{{
             route('uploads.store') }}"  method="POST"
                  enctype="multipart/form-data">
 @csrf

 <div class="mb-4">
     <x-input-label for="judul" value="Judul" />  <x-text-input
  id="judul" name="judul"  type="text" class="mt-1 block w-full"
 :value="old('judul')" required  autofocus />
     <x-input-error :messages="$errors- >get('judul')"
                      class="mt-2" />
 </div>

 <div class="mb-4">
   <x-input-label for="keterangan"  value="Keterangan
                     (opsional)" />
 <textarea id="keterangan" name="keterangan"  rows="3"
 class="mt-1 block w-full border-gray 300 rounded-md shadow-sm">{{
old('keterangan') }}</textarea>  <x-input-error :messages="$errors-
>get('keterangan')" class="mt-2" />
 </div>

 <div class="mb-4">
  <x-input-label for="file" value="Pilih Berkas  (jpg, jpeg, png, pdf,
                      doc, docx - maks. 2 MB)" />
   <input id="file" name="file" type="file"  class="mt-1 block
                        w-full" required>
 <x-input-error :messages="$errors->get('file')"  class="mt-2" />
 </div>

 <x-primary-button>Simpan</x-primary-button>  <a href="{{
route('uploads.index') }}" class="ms-3  text-gray-600">Batal</a>
 </form>
 </div>
 </div>
 </div>
</x-app-layout>
```

- enctype="multipart/form-data" → wajib ada pada form upload. Jika lupa ditambahkan, validasi file akan selalu gagal karena file tidak ikut terkirim.
- <input type="file" name="file"> → input untuk memilih file. Nama input (file) harus sama dengan nama yang digunakan pada $request->file('file') di controller.

- x-input-label, x-text-input, x-input-error, dan x-primary-button → komponen Blade bawaan Breeze untuk label, input, pesan galat validasi, dan tombol.
- :value="old('judul')" → mengisi kembali input dengan data sebelumnya ketika validasi gagal.

resources/views/uploads/create.blade.php

```
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">Unggah Berkas</h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white shadow-sm sm:rounded-lg p-6">
                {{-- enctype wajib ada agar file ikut terkirim --}}
                <form action="{{ route('uploads.store') }}" method="POST" enctype="multipart/form-data">
                    @csrf

                    <div class="mb-4">
                        <x-input-label for="judul" value="Judul" />
                        <x-text-input id="judul" name="judul" type="text" class="mt-1 block w-full"
                                      :value="old('judul')" required autofocus />
                        <x-input-error :messages="$errors->get('judul')" class="mt-2" />
                    </div>

                    <div class="mb-4">
                        <x-input-label for="keterangan" value="Keterangan (opsional)" />
                        <textarea id="keterangan" name="keterangan" rows="3"
                                  class="mt-1 block w-full border-gray-300 rounded-md shadow-sm">{{ old('keterangan') }}</textarea>
                        <x-input-error :messages="$errors->get('keterangan')" class="mt-2" />
                    </div>

                    <div class="mb-4">
                        <x-input-label for="file" value="Pilih Berkas (jpg, jpeg, png, pdf, doc, docx - maks. 2 MB)" />
                        <input id="file" name="file" type="file" class="mt-1 block w-full" required>
                        <x-input-error :messages="$errors->get('file')" class="mt-2" />
                    </div>

                    <x-primary-button>Simpan</x-primary-button>
                    <a href="{{ route('uploads.index') }}" class="ms-3 text-gray-600">Batal</a>
                </form>
            </div>
        </div>
    </div>
</x-app-layout>
```

Gambar 7.10 View create.blade.php (form unggah)

12. Buat file resources/views/uploads/edit.blade.php untuk form ubah berkas (Update):

```
<x-app-layout>
 <x-slot name="header">
 <h2 class="font-semibold text-xl text-gray-800 leading
        tight">Edit Berkas</h2>
 </x-slot>

 <div class="py-12">
 <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
  <div class="bg-white shadow-sm sm:rounded-lg p-6">  <form action="{{
        route('uploads.update', $upload) }}"  method="POST"
                  enctype="multipart/form-data">
 @csrf
```

```
                    @method('PUT') {{-- Mengubah request menjadi PUT
        agar menuju method update() --}}

 <div class="mb-4">
     <x-input-label for="judul" value="Judul" />  <x-text-input
   id="judul" name="judul"  type="text" class="mt-1 block w-full"
 :value="old('judul', $upload- >judul)" required />
     <x-input-error :messages="$errors- >get('judul')"
                     class="mt-2" />
 </div>

 <div class="mb-4">
    <x-input-label for="keterangan"  value="Keterangan
                     (opsional)" />
 <textarea id="keterangan" name="keterangan"  rows="3"
 class="mt-1 block w-full border-gray 300 rounded-md shadow-sm">{{
         old('keterangan', $upload- >keterangan) }}</textarea>
 </div>

 <div class="mb-4">
 <p class="text-sm text-gray-600">File saat ini:  <a href="{{
asset('storage/' . $upload- >filepath) }}" target="_blank"
 class="text-blue-600
         underline">{{ $upload->filename }}</a>
 </p>
                         <x-input-label for="file" value="Ganti Berkas
         (kosongkan jika tidak ingin mengganti)" class="mt-2" />
<input id="file" name="file" type="file"  class="mt-1 block
w-full">
 <x-input-error :messages="$errors->get('file')"  class="mt-2" />
 </div>

 <x-primary-button>Update</x-primary-button>  <a href="{{
route('uploads.index') }}" class="ms-3  text-gray-600">Batal</a>
 </form>
 </div>
 </div>
 </div>
</x-app-layout>
```

- @method('PUT') → form dikirim sebagai request PUT sehingga diproses oleh method update() pada controller.
- old('judul', $upload->judul) → menampilkan data lama dari database, atau data yang terakhir diketik jika validasi gagal.
- Input file tidak menggunakan atribut required → pengguna boleh hanya mengubah judul atau keterangan tanpa mengganti file. File lama hanya dihapus jika pengguna memilih file baru.



Gambar 7.11 View edit.blade.php (form ubah)

13. Buat file resources/views/uploads/show.blade.php untuk menampilkan detail berkas:

```
<x-app-layout>
 <x-slot name="header">
 <h2 class="font-semibold text-xl text-gray-800 leading
         tight">Detail Berkas</h2>
 </x-slot>

 <div class="py-12">
 <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
 <div class="bg-white shadow-sm sm:rounded-lg p-6 space-y 2">
        <p><strong>Judul:</strong> {{ $upload->judul }}</p>
  <p><strong>Keterangan:</strong> {{ $upload- >keterangan
                        ?? '-' }}</p>
 <p><strong>Nama File:</strong> {{ $upload- >filename
         }}</p>
 <p><strong>Diunggah oleh:</strong> {{ $upload->user- >name }}</p>
        <p><strong>Tanggal Unggah:</strong> {{ $upload-
            >created_at->format('d-m-Y H:i') }}</p>
```

```
@php
$ext = strtolower(pathinfo($upload->filepath,
        PATHINFO_EXTENSION));
@endphp

     {{-- Jika file berupa gambar, tampilkan pratinjau --}}  @if
        (in_array($ext, ['jpg', 'jpeg', 'png']))  <img src="{{
asset('storage/' . $upload- >filepath) }}" alt="{{ $upload->judul }}"
 class="max-w-sm rounded border">  @else
   <a href="{{ asset('storage/' . $upload- >filepath) }}"
                        target="_blank"
 class="inline-block px-4 py-2 bg-gray-800 text white rounded">Buka /
                            Unduh Berkas</a>
 @endif

 <div class="pt-4">
          <a href="{{ route('uploads.edit', $upload) }}"
                class="text-yellow-600">Edit</a> |
 <a href="{{ route('uploads.index') }}" class="text
        gray-600">Kembali</a>
 </div>
 </div>
 </div>
 </div>
</x-app-layout>
```

- $upload->user->name → memanggil nama pemilik berkas melalui relasi belongsTo pada model Upload.
- pathinfo(..., PATHINFO_EXTENSION) → mengambil ekstensi file untuk menentukan apakah file berupa gambar.
- <img src="{{ asset('storage/' . ...) }}"> → menampilkan pratinjau gambar. Untuk file PDF atau Word, ditampilkan tombol untuk membuka atau mengunduh berkas.

resources/views/uploads/show.blade.php

```blade
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">Detail Berkas</h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white shadow-sm sm:rounded-lg p-6 space-y-2">
                <p><strong>Judul:</strong> {{ $upload->judul }}</p>
                <p><strong>Keterangan:</strong> {{ $upload->keterangan ?? '-' }}</p>
                <p><strong>Nama File:</strong> {{ $upload->filename }}</p>
                <p><strong>Diunggah oleh:</strong> {{ $upload->user->name }}</p>
                <p><strong>Tanggal Unggah:</strong> {{ $upload->created_at->format('d-m-Y H:i') }}</p>

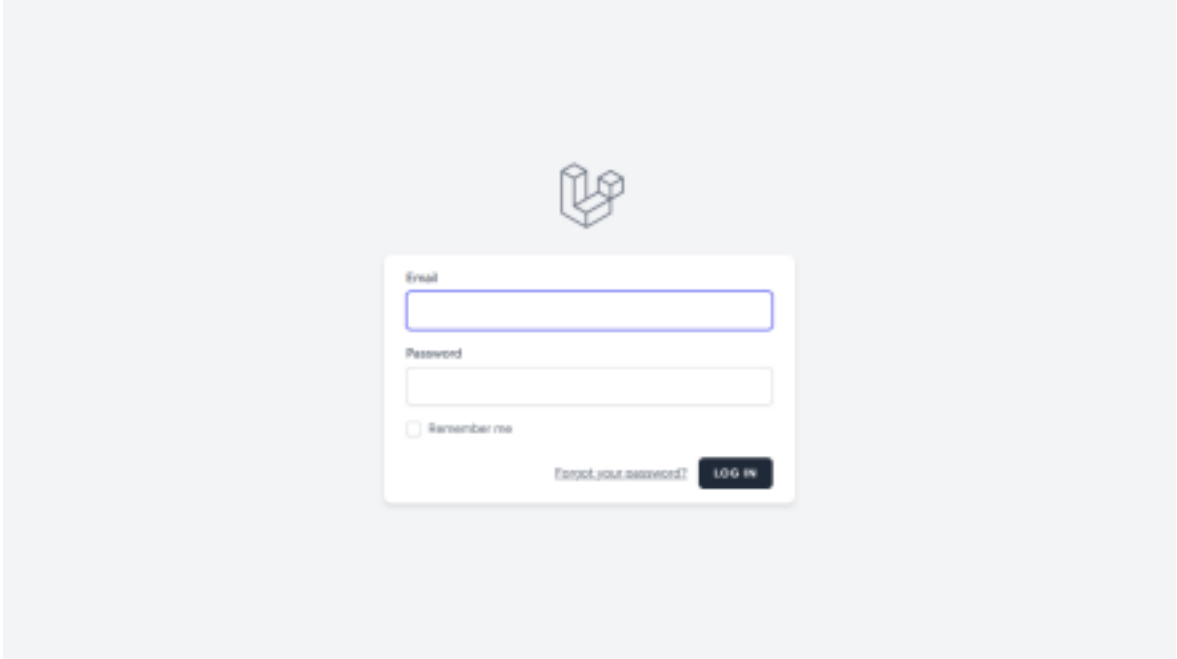
                @php
                    $ext = strtolower(pathinfo($upload->filepath, PATHINFO_EXTENSION));
                @endphp

                {{-- Jika file berupa gambar, tampilkan pratinjau --}}
                @if (in_array($ext, ['jpg', 'jpeg', 'png']))
                    <img src="{{ asset('storage/' . $upload->filepath) }}" alt="{{ $upload->judul }}"
                         class="max-w-sm rounded border">
                @else
                    <a href="{{ asset('storage/' . $upload->filepath) }}" target="_blank"
                       class="inline-block px-4 py-2 bg-gray-800 text-white rounded">Buka / Unduh Berkas</a>
                @endif

                <div class="pt-4">
                    <a href="{{ route('uploads.edit', $upload) }}" class="text-yellow-600">Edit</a> |
                    <a href="{{ route('uploads.index') }}" class="text-gray-600">Kembali</a>
                </div>
            </div>
        </div>
    </div>
</x-app-layout>
```

Gambar 7.12 View show.blade.php (detail berkas)

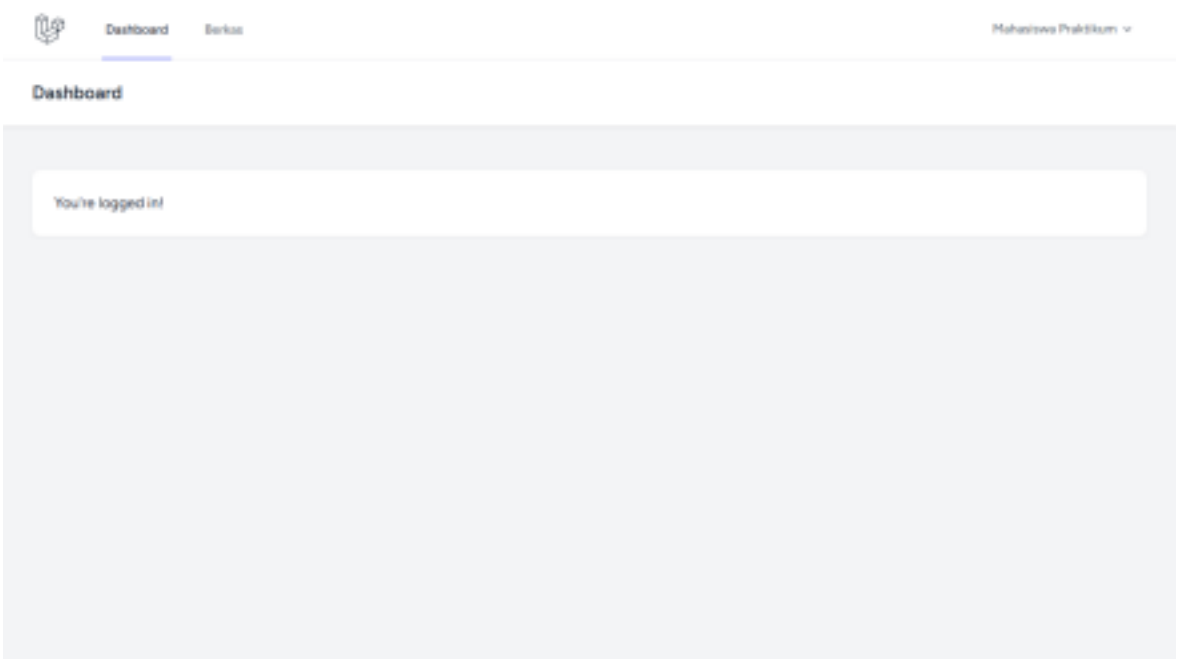
14. Uji fitur CRUD Berkas yang telah dibuat dengan langkah-langkah berikut: a. Pastikan npm run dev masih berjalan pada terminal pertama, lalu jalankan php artisan serve pada terminal kedua (atau cukup jalankan composer run dev).
    b. Buka http://127.0.0.1:8000/uploads sebelum login → Anda akan otomatis diarahkan ke halaman login oleh middleware auth.



Gambar 7.13 Halaman /uploads diakses sebelum login

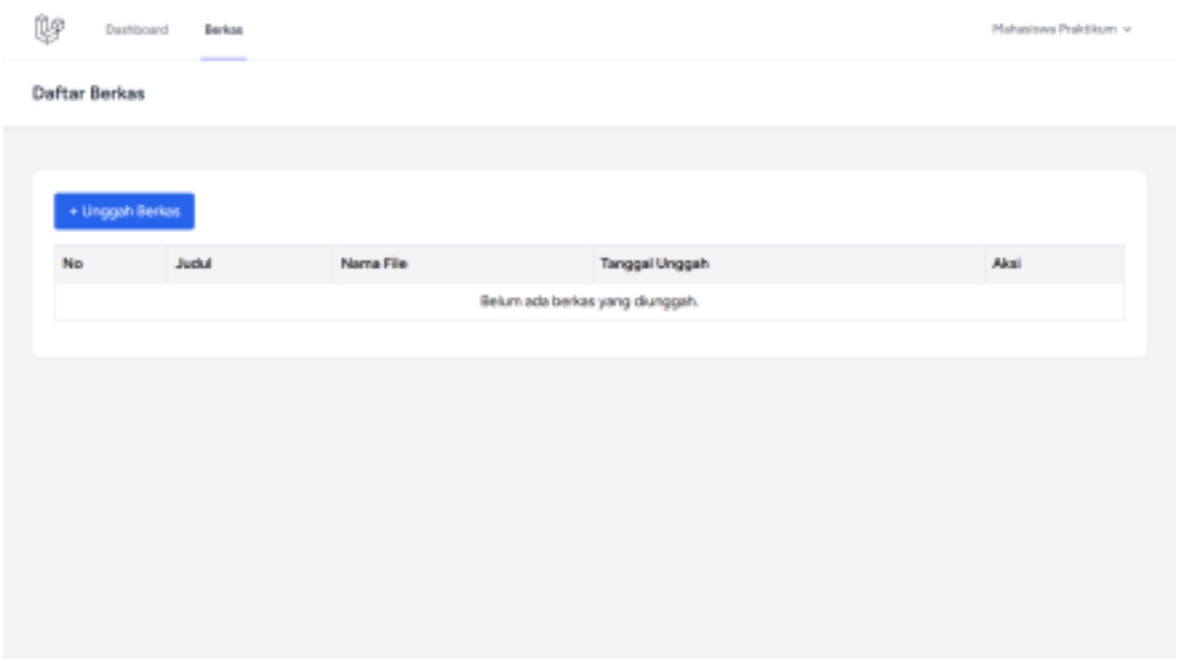
Saat membuka http://127.0.0.1:8123/uploads tanpa login, middleware auth mengarahkan pengguna ke halaman login (http://127.0.0.1:8123/login).

c. Login menggunakan akun yang didaftarkan pada Tahap A langkah 10, lalu klik menu Berkas pada navigasi.



Gambar 7.14 Dashboard setelah login dengan menu Berkas

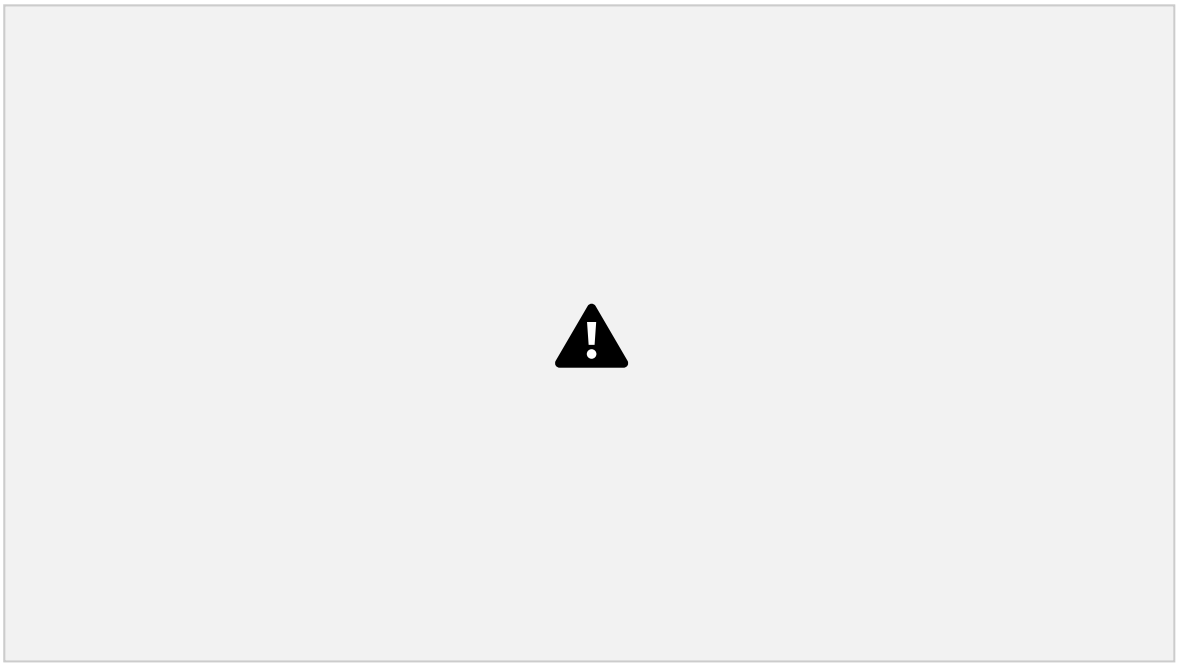
Setelah login, menu Berkas tampil pada navigasi Breeze di samping menu Dashboard.



Gambar 7.15 Daftar Berkas masih kosong

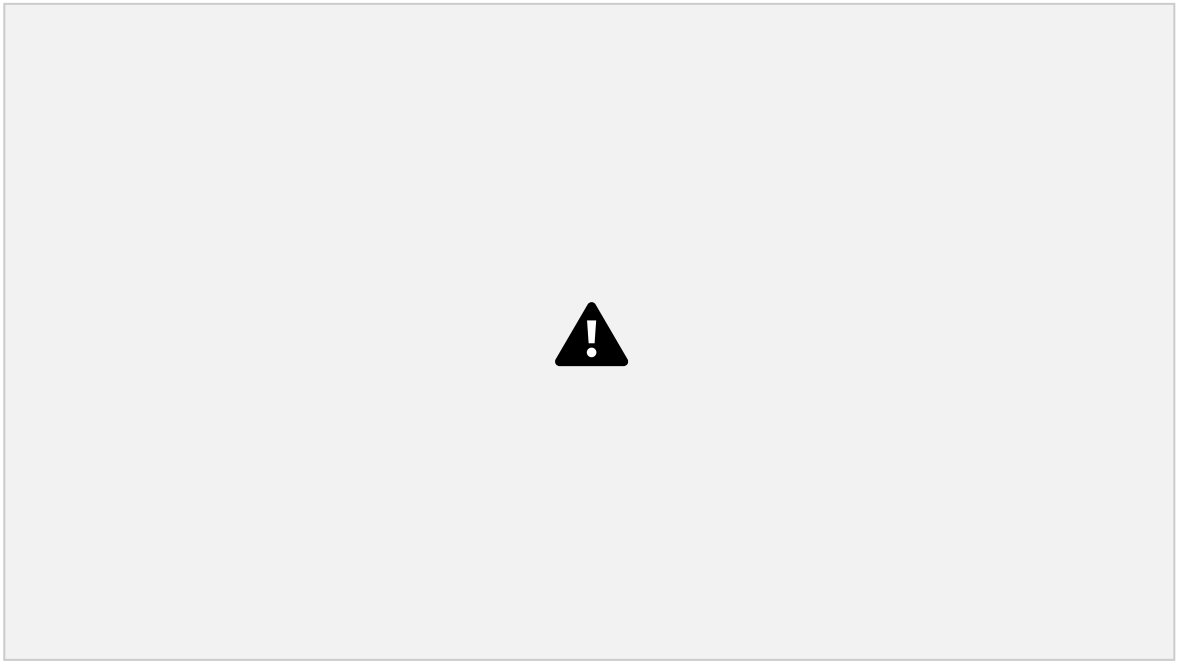
Halaman index menampilkan pesan "Belum ada berkas yang diunggah." dari blok @empty.

d. Klik + Unggah Berkas, isi judul, pilih file gambar atau PDF, lalu klik Simpan → berkas muncul pada tabel disertai pesan Berkas berhasil diunggah.



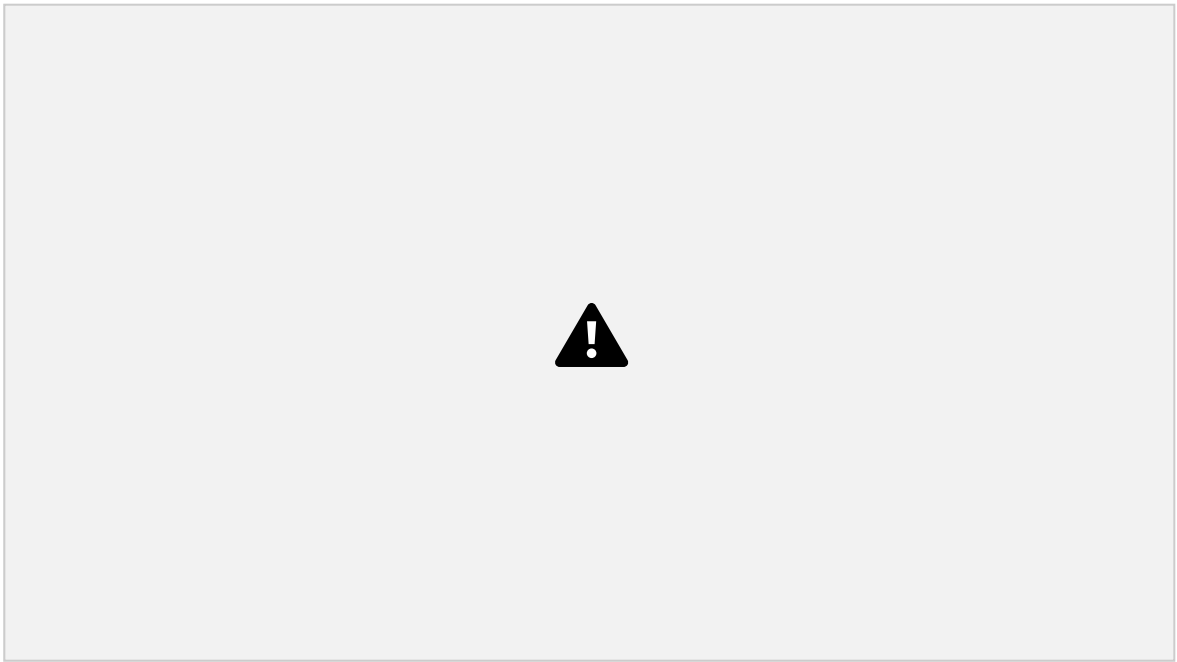
Gambar 7.16 Form Unggah Berkas (Create)

Form unggah berkas berisi judul, keterangan, dan input file dengan enctype multipart/form data.



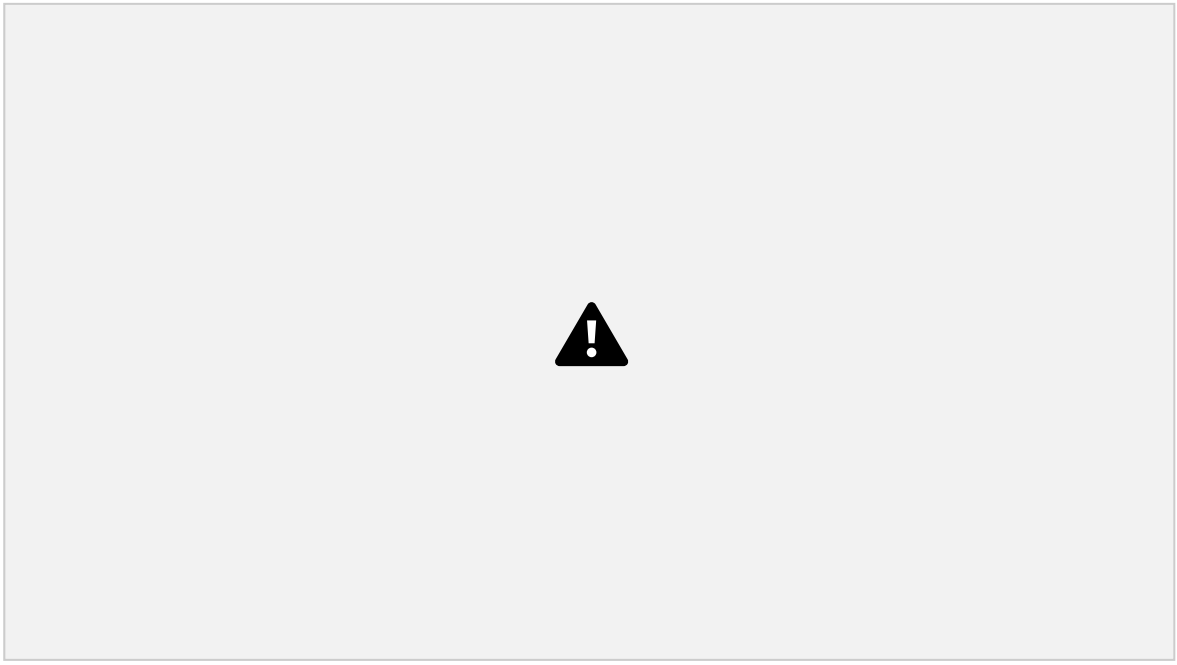
Gambar 7.17 Pesan berhasil setelah berkas diunggah

Setelah klik Simpan, berkas tersimpan dan halaman index menampilkan pesan "Berkas berhasil diunggah."



Gambar 7.18 Daftar Berkas halaman 1 dengan pagination

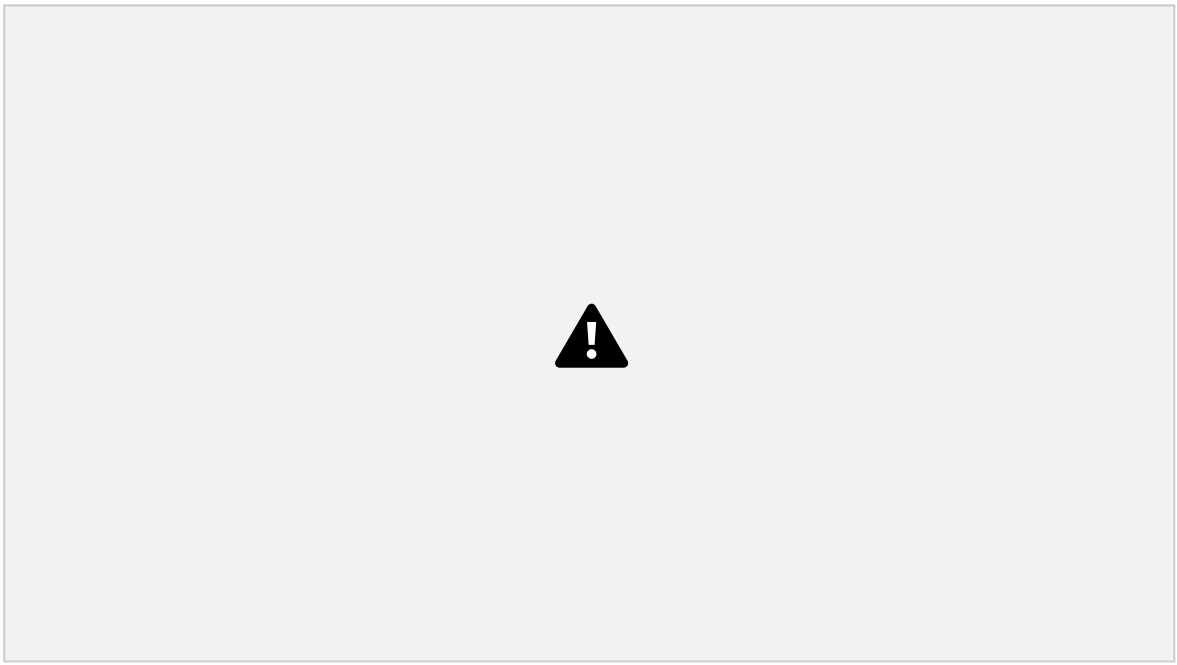
Daftar berkas milik pengguna diurutkan dari yang terbaru, 5 data per halaman, dengan nomor urut dan tombol pagination.



Gambar 7.19 Daftar Berkas halaman 2

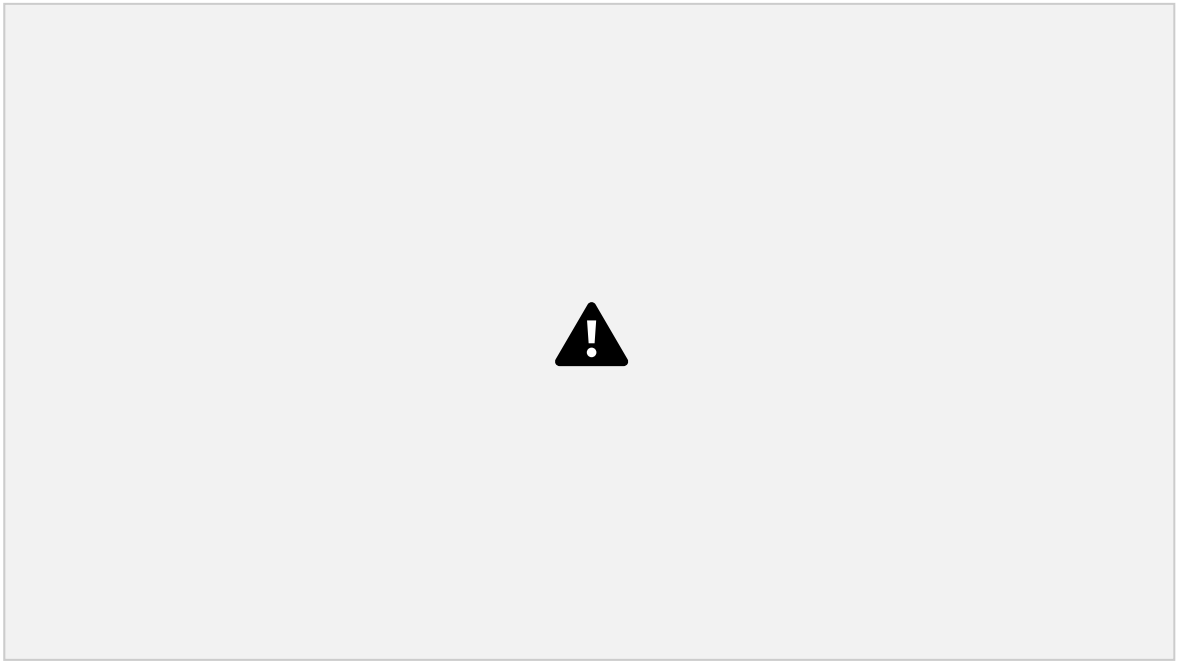
Halaman kedua pagination; nomor urut berlanjut karena memakai firstItem() + $loop->index.

e. Coba unggah file dengan format yang tidak diizinkan (misalnya .zip) atau berukuran lebih dari 2 MB → pesan galat validasi muncul di bawah input file.



Gambar 7.20 Pesan galat saat mengunggah file .zip

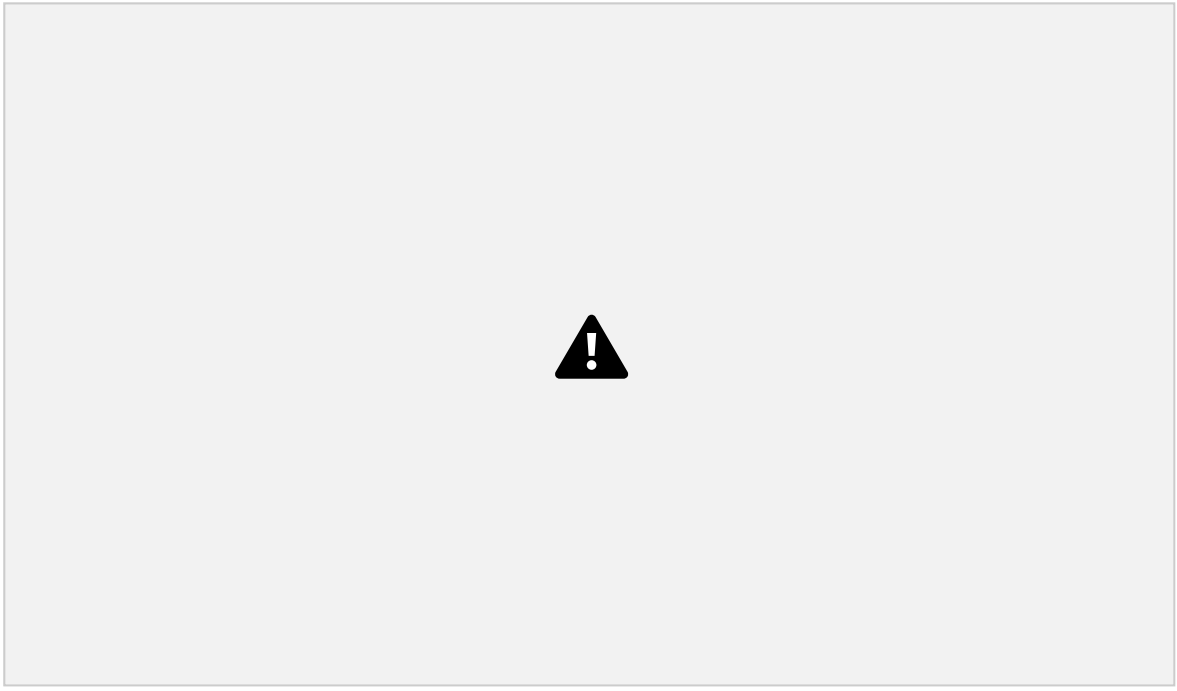
Validasi mimes menolak file .zip dan menampilkan pesan galat di bawah input file.



Gambar 7.21 Pesan galat saat mengunggah file lebih dari 2 MB

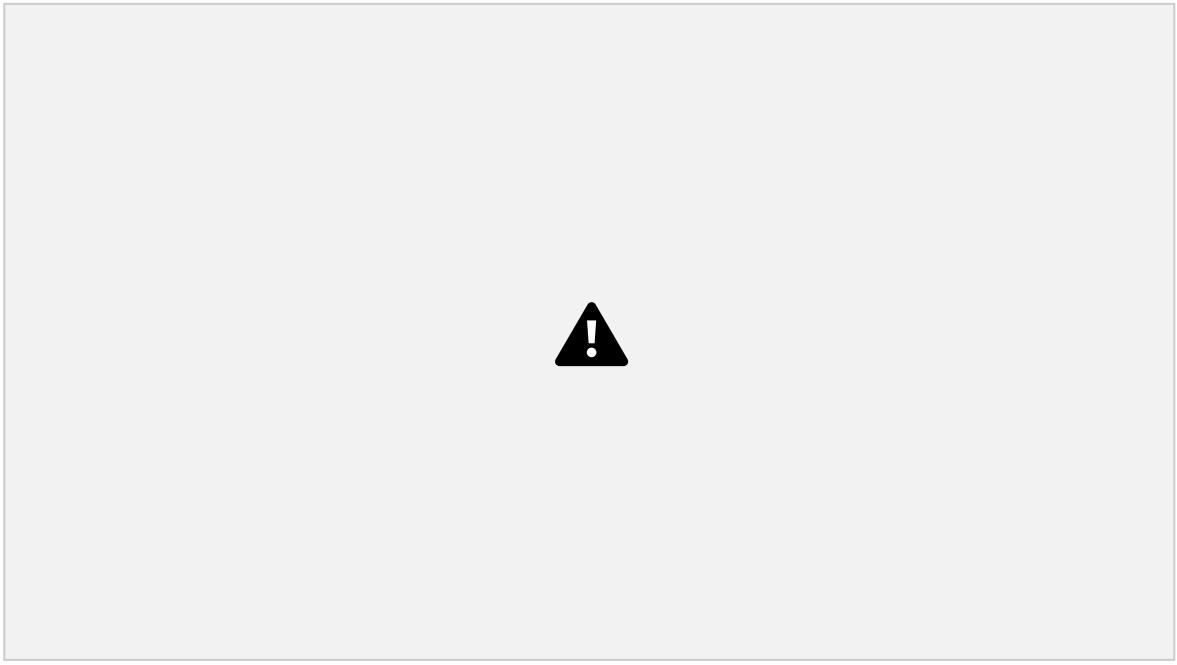
Validasi max:2048 menolak file berukuran sekitar 5,6 MB.

f. Klik Lihat untuk melihat detail dan pratinjau, klik Edit untuk mengganti judul atau file, lalu klik Hapus untuk menghapus berkas.



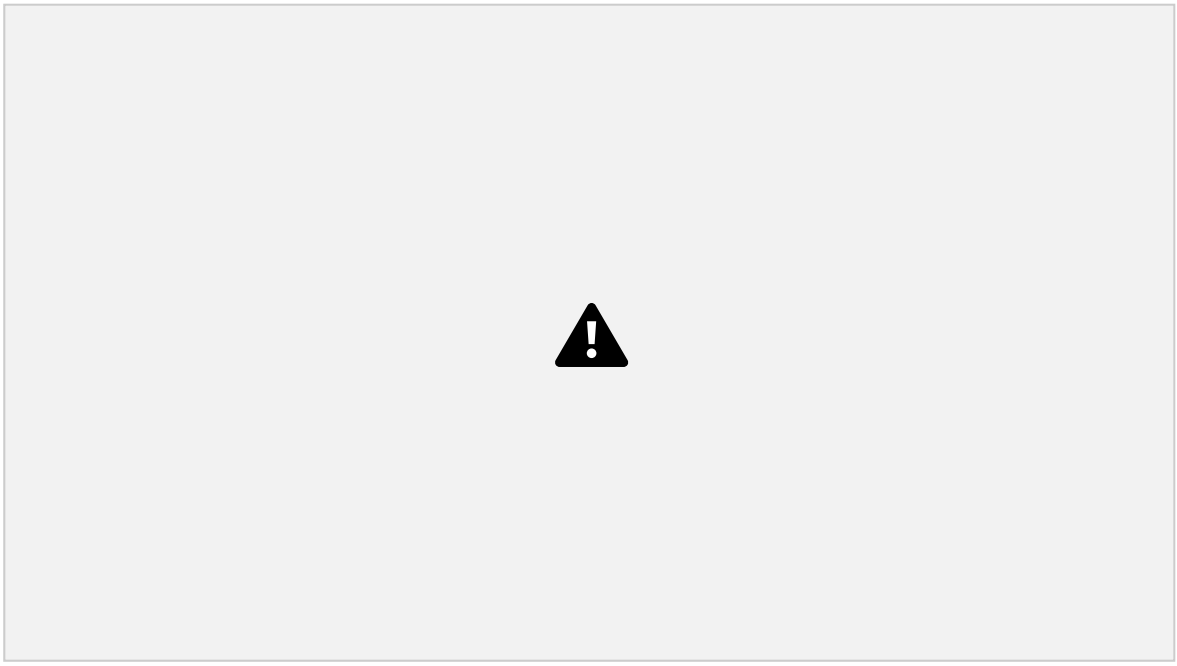
Gambar 7.22 Halaman Detail Berkas gambar (Show)

Detail berkas menampilkan judul, keterangan, nama file, pemilik, tanggal unggah, dan pratinjau gambar.



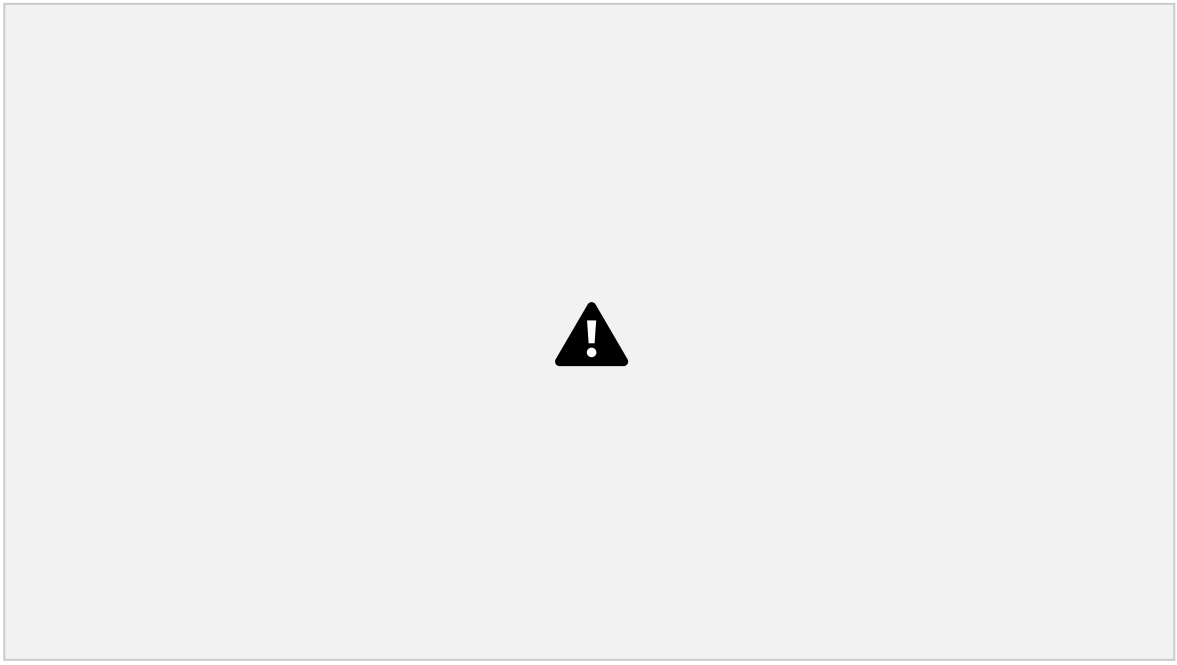
Gambar 7.23 Halaman Detail Berkas PDF

Untuk file non-gambar (PDF/Word), halaman detail menampilkan tombol Buka / Unduh Berkas.



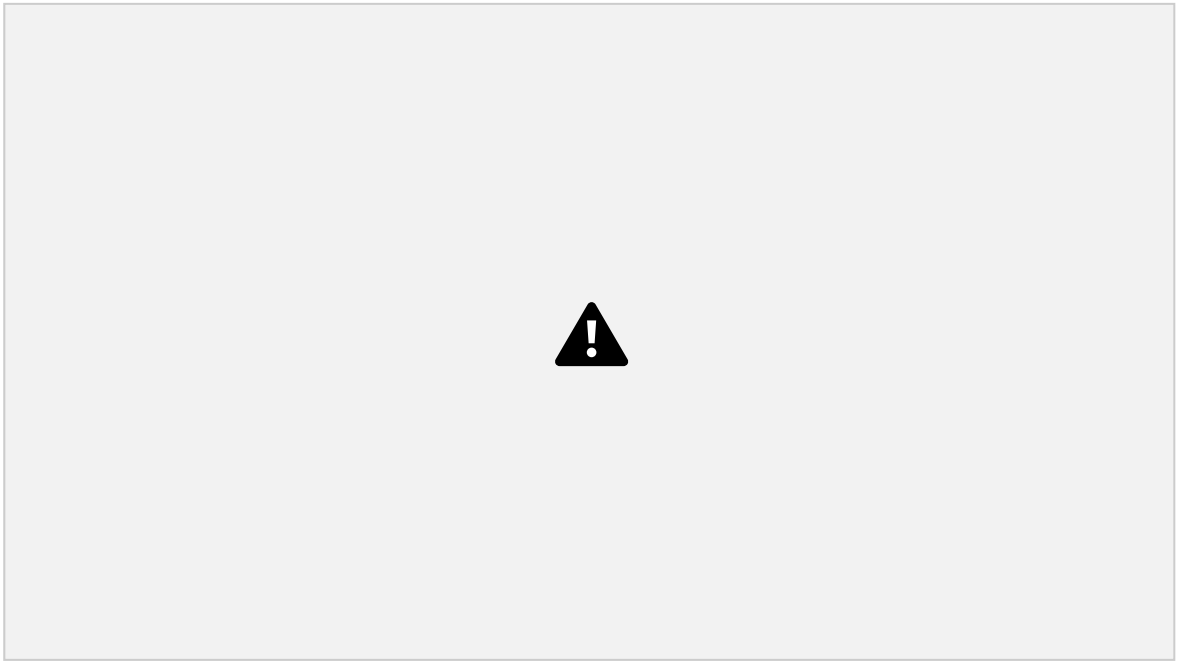
Gambar 7.24 File dibuka langsung melalui URL storage

Berkas dapat diakses melalui URL http://127.0.0.1:8123/storage/uploads/bhKdZv0Bnb2TOPUvPsCbA2PvTSxwUV712VshZojE. jpg karena php artisan storage:link sudah dijalankan.



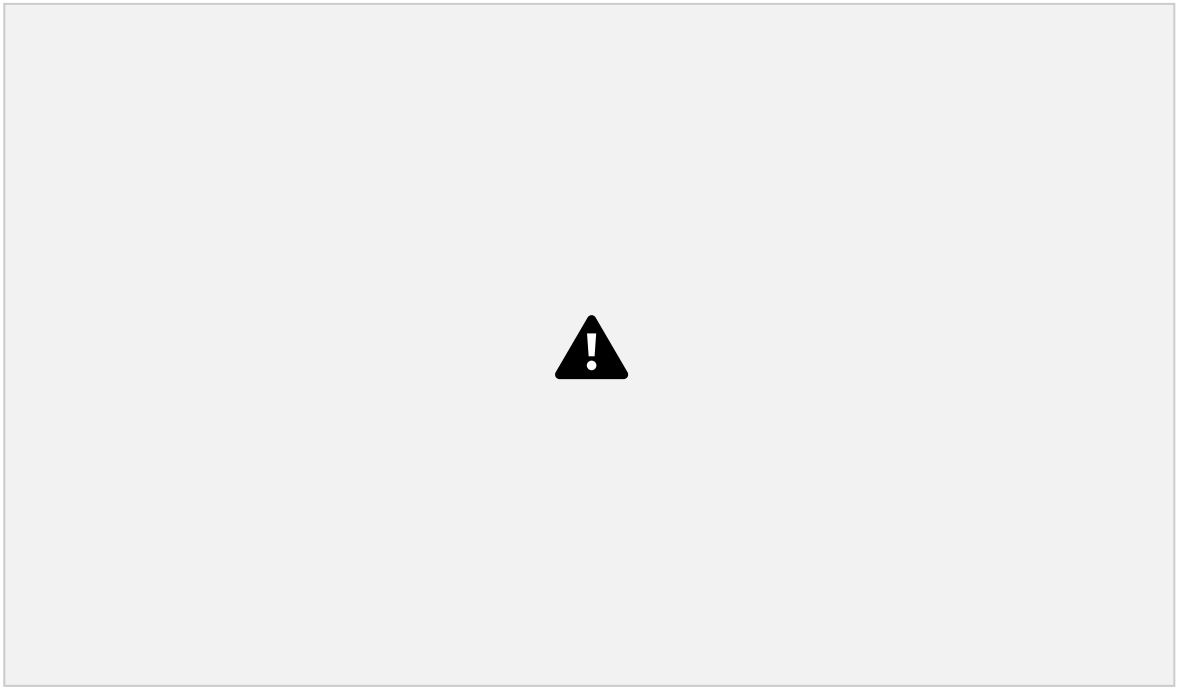
Gambar 7.25 Form Edit Berkas sebelum file diganti

Form edit menampilkan data lama dan tautan file saat ini; input file bersifat opsional.



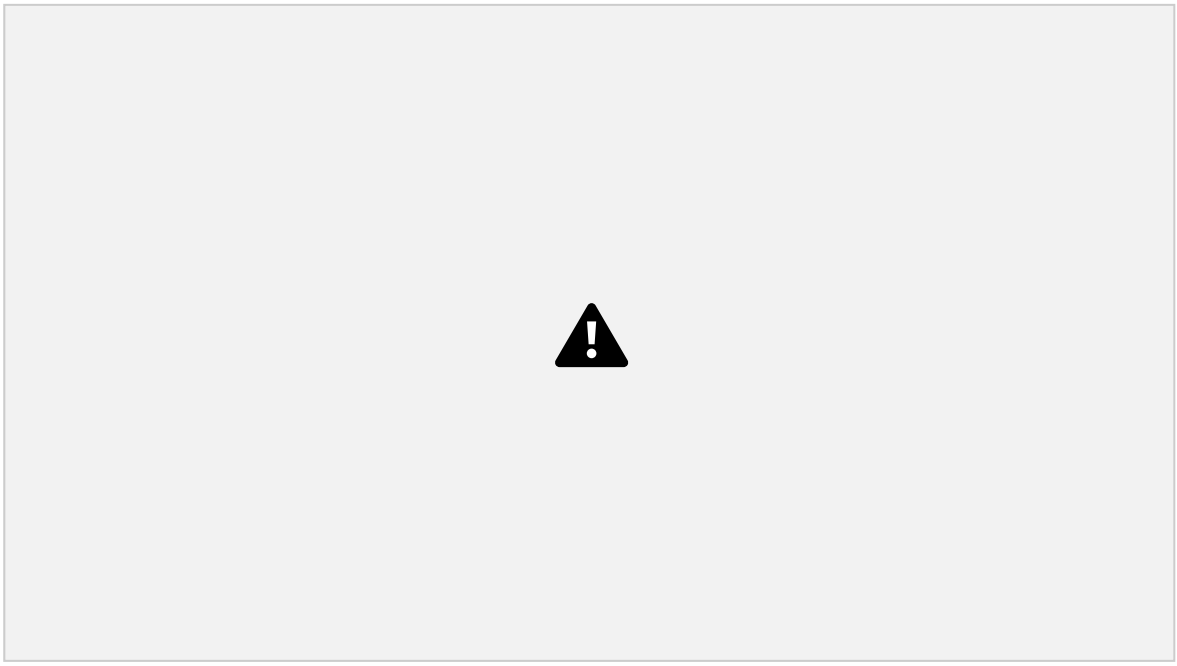
Gambar 7.26 Pesan berhasil setelah berkas diperbarui

Setelah klik Update, judul dan file berubah serta tampil pesan "Berkas berhasil diperbarui."



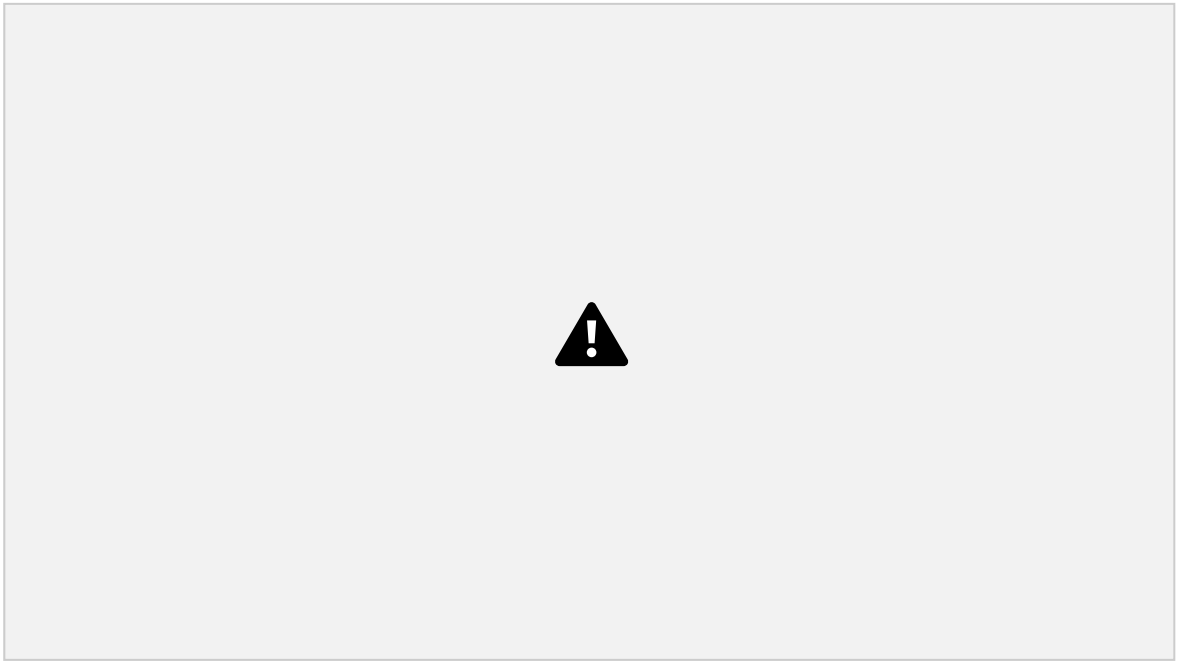
Gambar 7.27 Detail berkas setelah file diganti

Pratinjau menampilkan gambar pengganti; file lama sudah dihapus dari storage.



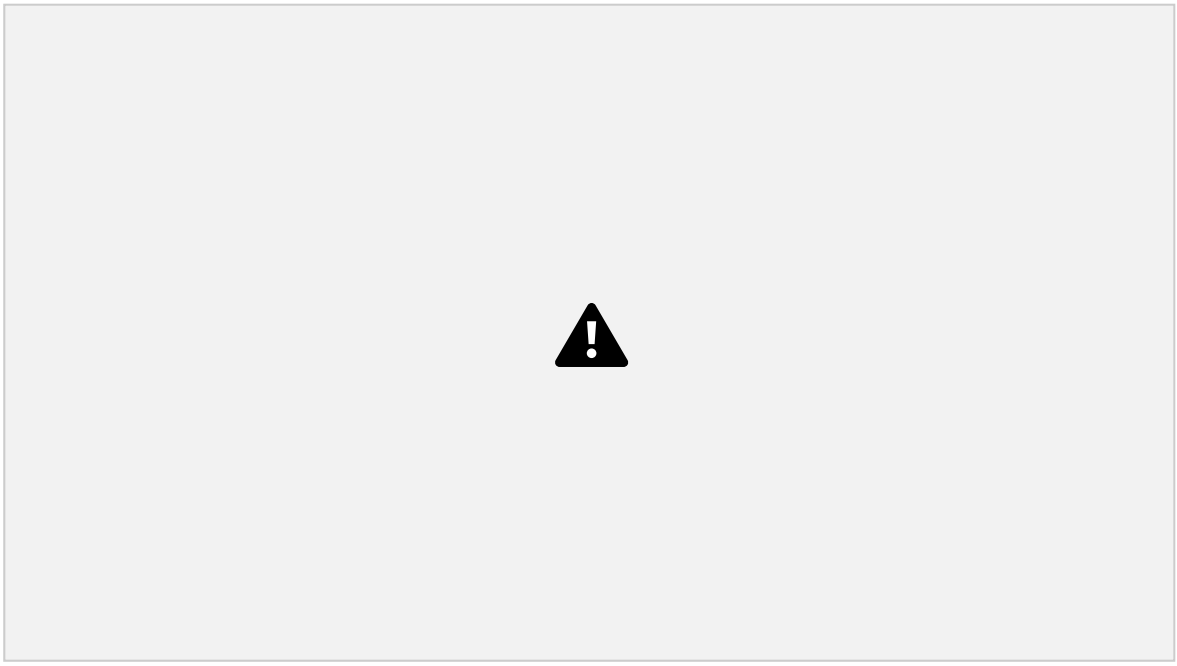
Gambar 7.28 Pesan berhasil setelah berkas dihapus

Setelah konfirmasi, data dan file fisik terhapus serta tampil pesan "Berkas berhasil dihapus."



Gambar 7.29 Galat 403 saat membuka berkas milik pengguna lain

abort_if($upload->user_id !== Auth::id(), 403) menolak akses pengguna lain ke berkas yang bukan miliknya.

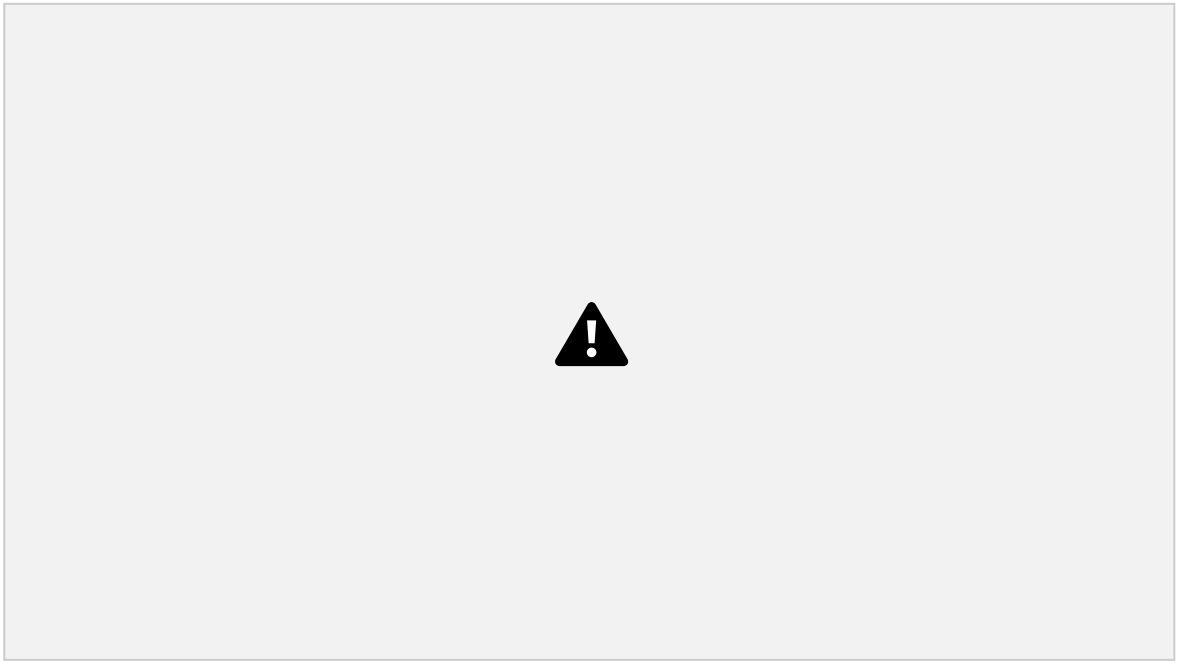


Gambar 7.30 Daftar berkas pengguna lain tetap kosong

Setiap pengguna hanya melihat berkas miliknya sendiri karena query memakai where('user_id', Auth::id()).

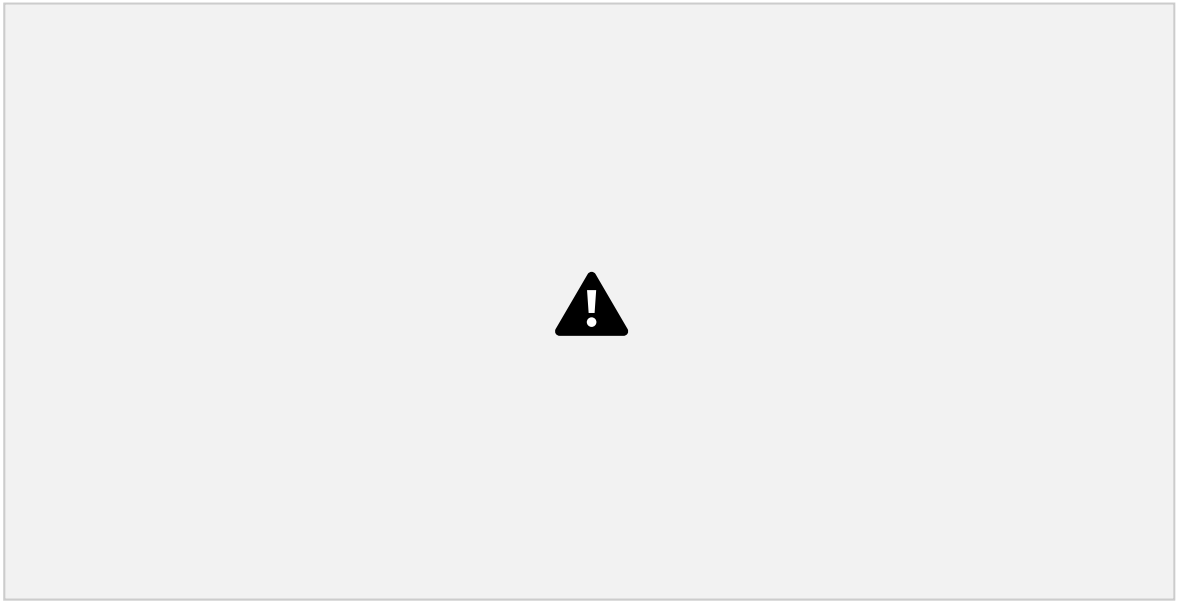
g. Buka folder storage/app/public/uploads dan tabel uploads pada phpMyAdmin → file fisik dan data path harus bertambah ketika berkas diunggah, berganti ketika file

diedit, dan hilang ketika berkas dihapus.



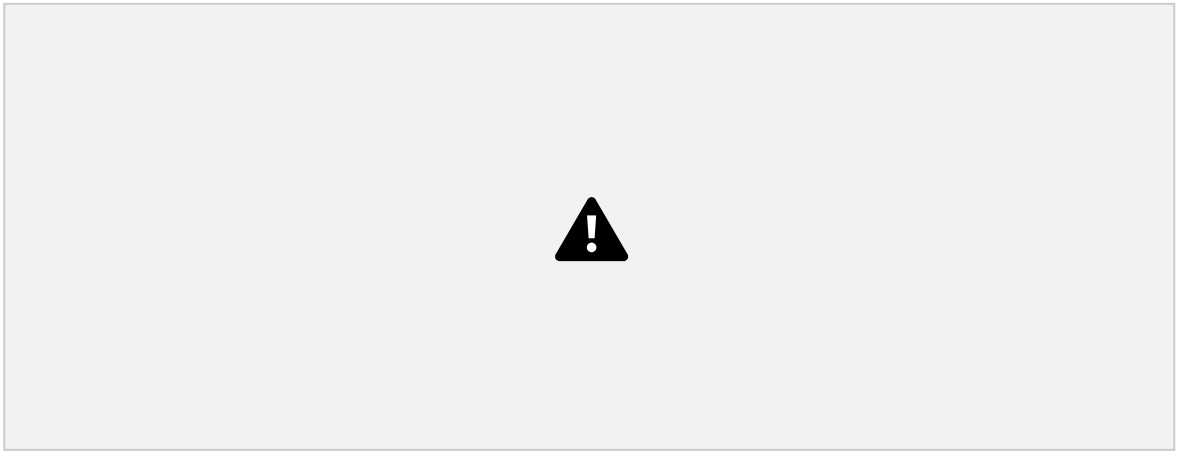
Gambar 7.31 Isi tabel uploads pada phpMyAdmin

Database hanya menyimpan path file (kolom filepath) dan nama asli file (kolom filename), bukan isi file.



Gambar 7.32 Isi folder storage/app/public/uploads

File fisik tersimpan di folder storage dengan nama acak hasil store(); jumlahnya sama dengan data pada tabel uploads.



Gambar 7.33 Hasil pengujian otomatis (Pest) fitur CRUD Berkas

Seluruh feature test untuk redirect login, upload, validasi, edit, hapus, dan pembatasan akses berhasil (passed).

7.4.5 CRUD dan Upload File Tanpa Breeze (Manual)

Pada subbab sebelumnya, tampilan dan navigasi sudah disediakan oleh Breeze. Pada subbab ini, kita akan membuat CRUD Kontak dari awal, dimulai dari pembuatan project Laravel baru, yang dilengkapi upload foto secara manual menggunakan layout Blade dan Bootstrap, sehingga seluruh bagian (route, controller, layout, dan view) ditulis sendiri. Setiap kontak memiliki nama, alamat, nomor HP, dan foto.

Tahap A: Menyiapkan Project Baru dan Layout Bootstrap

1. Buka terminal (Command Prompt, PowerShell, atau terminal Visual Studio Code) di folder tempat Anda menyimpan project, misalnya C:\xampp\htdocs atau folder www pada Laragon. Pastikan Apache dan MySQL pada XAMPP/Laragon sudah dijalankan (Start).
2. Buat project Laravel baru bernama crud-manual dengan perintah berikut:

```
composer create-project laravel/laravel crud-manual
```

Perintah ini mengunduh Laravel versi terbaru beserta seluruh dependency-nya ke dalam folder crud-manual. Jika menggunakan Laravel Installer, dapat juga menggunakan perintah laravel new crud-manual (pilih None ketika ditanya starter kit).

3. Masuk ke folder project lalu buka di Visual Studio Code:

```
cd crud-manual
code .
```

Seluruh perintah pada langkah berikutnya dijalankan di dalam folder crud-manual menggunakan terminal Visual Studio Code (menu Terminal → New Terminal).

4. Buka phpMyAdmin (http://localhost/phpmyadmin), klik New, lalu buat database baru bernama db_crud_manual.
5. Buka file .env, lalu ubah konfigurasi database menjadi seperti berikut dan simpan (Ctrl + S):

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_crud_manual
DB_USERNAME=root
DB_PASSWORD=
```

Ganti nilai DB_CONNECTION dari sqlite menjadi mysql, lalu hapus tanda # di depan baris DB_HOST sampai DB_PASSWORD jika masih berupa komentar.

6. Jalankan migration bawaan Laravel dengan perintah berikut:

```
php artisan migrate
```

Perintah ini membuat tabel bawaan seperti users, sessions, cache, dan jobs di database db_crud_manual. Tabel sessions dibutuhkan karena Laravel menyimpan session (termasuk pesan berhasil) di database.

7. Buat folder resources/views/layouts, lalu buat file resources/views/layouts/app.blade.php dan isi dengan kode berikut:

```
<!DOCTYPE html>
<html lang="id">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial
        scale=1.0">
 <title>@yield('title', 'CRUD Kontak')</title>
 <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/b
        o otstrap.min.css" rel="stylesheet">
 <link rel="stylesheet"
         href="https://cdnjs.cloudflare.com/ajax/libs/font
        awesome/6.4.0/css/all.min.css">
</head>
<body>
 {{-- Navbar (menu bagian atas website) --}}
 <nav class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">
<div class="container">
 <a class="navbar-brand"
        href="{{ route('kontaks.index') }}">CRUD Kontak</a>
```

```
<ul class="navbar-nav ms-auto">
<li class="nav-item">
<a class="nav-link"
        href="{{ route('kontaks.index') }}">
<i class="fas fa-address-book me-1"></i>Kontak  </a>
</li>
</ul>
</div>
</nav>

<div class="container">
{{-- Pesan berhasil yang dikirim dari controller dengan
        with('success', ...) --}}
@if (session('success'))
<div class="alert alert-success alert-dismissible fade  show"
        role="alert">
{{ session('success') }}
        <button type="button" class="btn-close" data-bs
         dismiss="alert" aria-label="Close"></button>
</div>
@endif

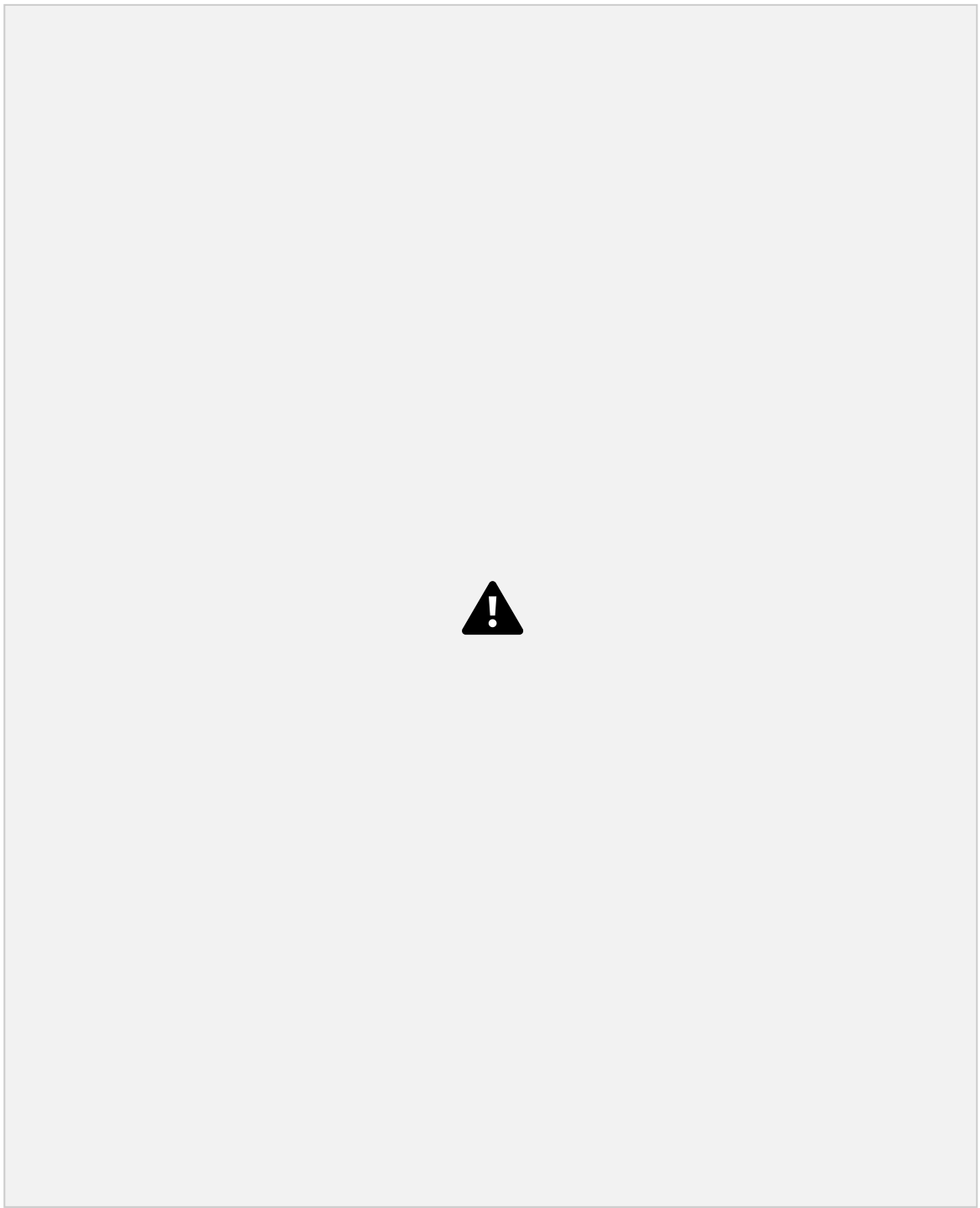
{{-- Ringkasan jika ada galat validasi --}}
@if ($errors->any())
<div class="alert alert-danger">
Data belum valid, periksa kembali isian form.
</div>
@endif

@yield('content')
</div>

<script
        src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/boo
        t strap.bundle.min.js"></script>
</body>
</html>
```

- Link CSS dan JavaScript Bootstrap serta Font Awesome melalui CDN → tampilan menjadi rapi tanpa perlu Node.js dan npm run dev.
- Navbar dengan menu Kontak → mengarah ke route kontaks.index yang akan dibuat pada Tahap B.
- @if (session('success')) → menampilkan pesan berhasil setelah data ditambah, diubah, atau dihapus, sehingga tidak perlu ditulis ulang pada setiap view.
- @if ($errors->any()) → menampilkan ringkasan jika validasi gagal; pesan galat per input ditampilkan di bawah masing-masing input.

- @yield('title') dan @yield('content') → tempat judul dan isi halaman dari view lain (index, create, edit, show) ditampilkan.



Gambar 7.34 Layout Bootstrap layouts/app.blade.php

Tahap B: Membuat CRUD Kontak dengan Upload Foto

1. Buat model Kontak beserta migration-nya dengan perintah berikut:

```
php artisan make:model Kontak -m
```

2. Buka file database/migrations/xxxx_xx_xx_xxxxxx_create_kontaks_table.php, lalu ubah isinya menjadi seperti berikut:

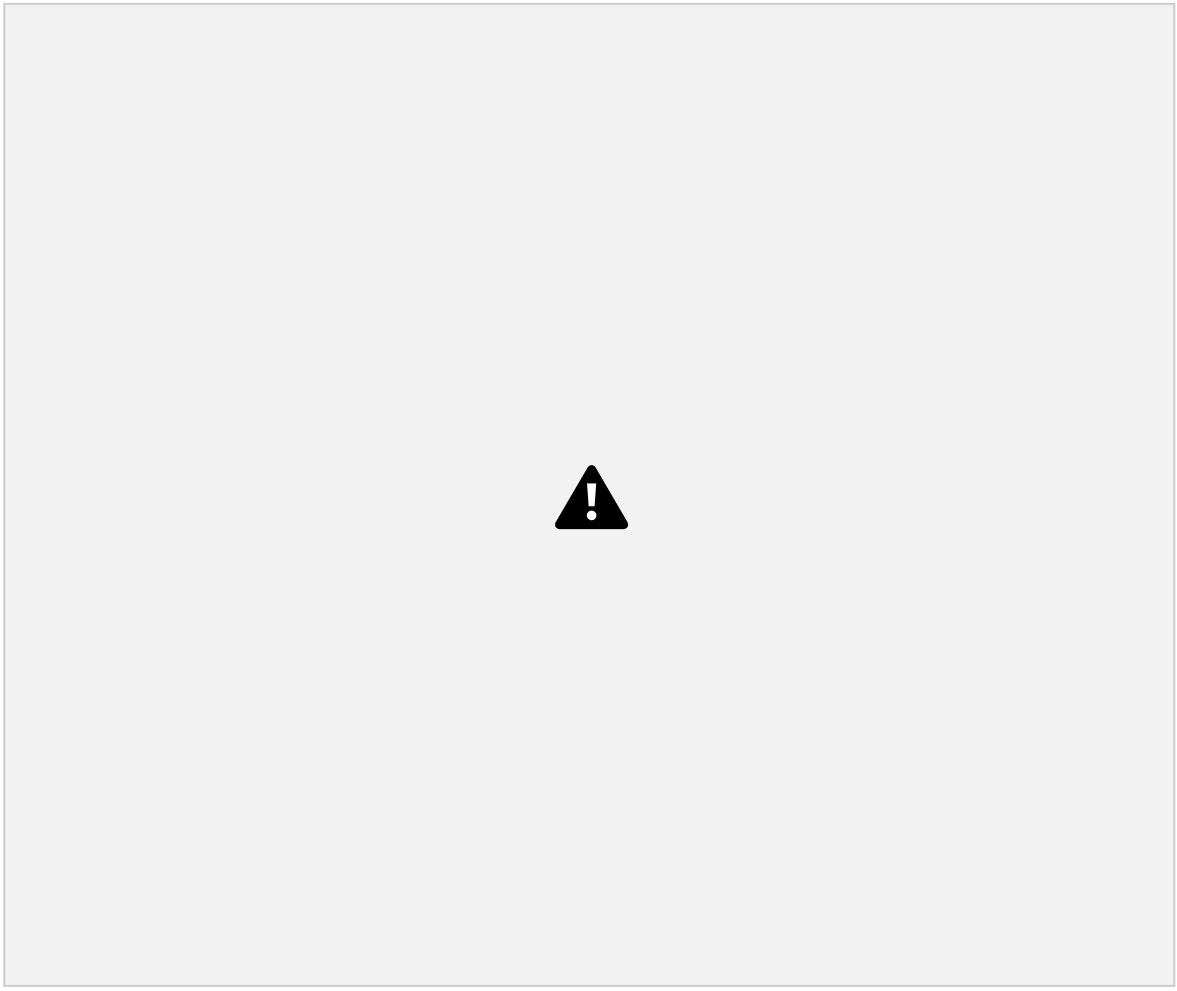
```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
 public function up(): void
 {
 Schema::create('kontaks', function (Blueprint $table) {
$table->id(); // Primary key  auto increment
  $table->string('nama'); // VARCHAR(255)  $table->text('alamat');
  // TEXT  $table->string('no_hp', 15); // Disimpan  sebagai teks
                agar angka 0 di depan tidak hilang
 $table->string('foto')->nullable(); // Path foto,  boleh kosong
 $table->timestamps(); // created_at dan  updated_at
 });
 }

 public function down(): void
 {
 Schema::dropIfExists('kontaks');
 }
};
```

- string('no_hp', 15) → nomor HP disimpan sebagai teks (bukan angka) agar angka 0 di depan, misalnya 0812..., tidak hilang dan dapat menampung tanda +62.
- string('foto')->nullable() → menyimpan path foto; bernilai null jika kontak tidak memiliki foto.



Gambar 7.35 Migration tabel kontaks

3. Jalankan migration dengan perintah php artisan migrate, lalu pastikan tabel kontaks sudah muncul di phpMyAdmin.
4. Buka file app/Models/Kontak.php, lalu ganti isinya dengan kode berikut:

```
<?php

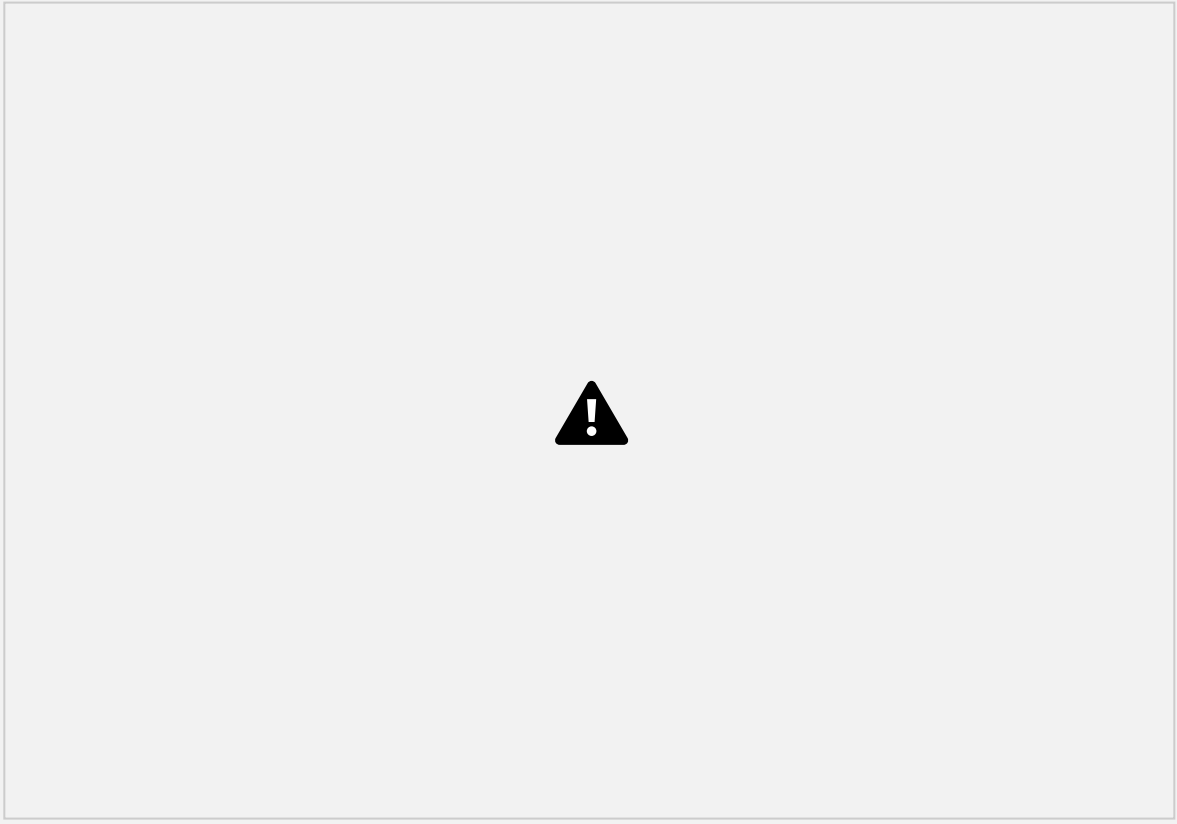
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Kontak extends Model
{
 protected $table = 'kontaks'; // Nama tabel yang dihubungkan

 // Kolom yang boleh diisi melalui Kontak::create() dan update()
protected $fillable = [
 'nama',
 'alamat',
```

```
 'no_hp',
 'foto',
 ];
}
```



Gambar 7.36 Model Kontak

5. Jalankan perintah php artisan storage:link (cukup sekali untuk setiap project) agar foto di folder storage dapat ditampilkan di browser.



Gambar 7.37 Hasil php artisan storage:link

Symbolic link public/storage berhasil dibuat dan mengarah ke folder storage/app/public.

6. Buat resource controller dengan perintah php artisan make:controller KontakController --resource, lalu buka file app/Http/Controllers/KontakController.php dan ganti seluruh isinya dengan kode berikut:

```
<?php
```

```
namespace App\Http\Controllers;
use App\Models\Kontak;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;

class KontakController extends Controller
{
 public function index()
 {
 $kontaks = Kontak::latest()->paginate(5); // 5 data per halaman
return view('kontaks.index', compact('kontaks'));  }

 public function create()
 {
 return view('kontaks.create');
 }

 public function store(Request $request)
 {
 $validated = $request->validate([
 'nama' => 'required|string|max:255',
 'alamat' => 'required|string',
 'no_hp' => 'required|string|max:15',
 'foto' => 'nullable|image|mimes:jpg,jpeg,png|max:2048',  // Foto
         opsional, maks. 2 MB
 ]);

 if ($request->hasFile('foto')) {
 // Simpan ke storage/app/public/foto_kontak, lalu simpan  path-nya
      $validated['foto'] = $request->file('foto')-
            >store('foto_kontak', 'public');
 }

 Kontak::create($validated);

    return redirect()->route('kontaks.index')->with('success',
                'Kontak berhasil ditambahkan.');
 }

 public function show(Kontak $kontak)
 {
 return view('kontaks.show', compact('kontak'));  }

 public function edit(Kontak $kontak)
 {
 return view('kontaks.edit', compact('kontak'));  }
```

```
 public function update(Request $request, Kontak $kontak)
{
 $validated = $request->validate([
 'nama' => 'required|string|max:255',
 'alamat' => 'required|string',
 'no_hp' => 'required|string|max:15',
 'foto' => 'nullable|image|mimes:jpg,jpeg,png|max:2048',  ]);

 if ($request->hasFile('foto')) {
 if ($kontak->foto) {
 Storage::disk('public')->delete($kontak->foto); //  Hapus foto
         lama
 }
      $validated['foto'] = $request->file('foto')-
           >store('foto_kontak', 'public');
 }

 $kontak->update($validated);

    return redirect()->route('kontaks.index')->with('success',
                'Kontak berhasil diperbarui.');
 }

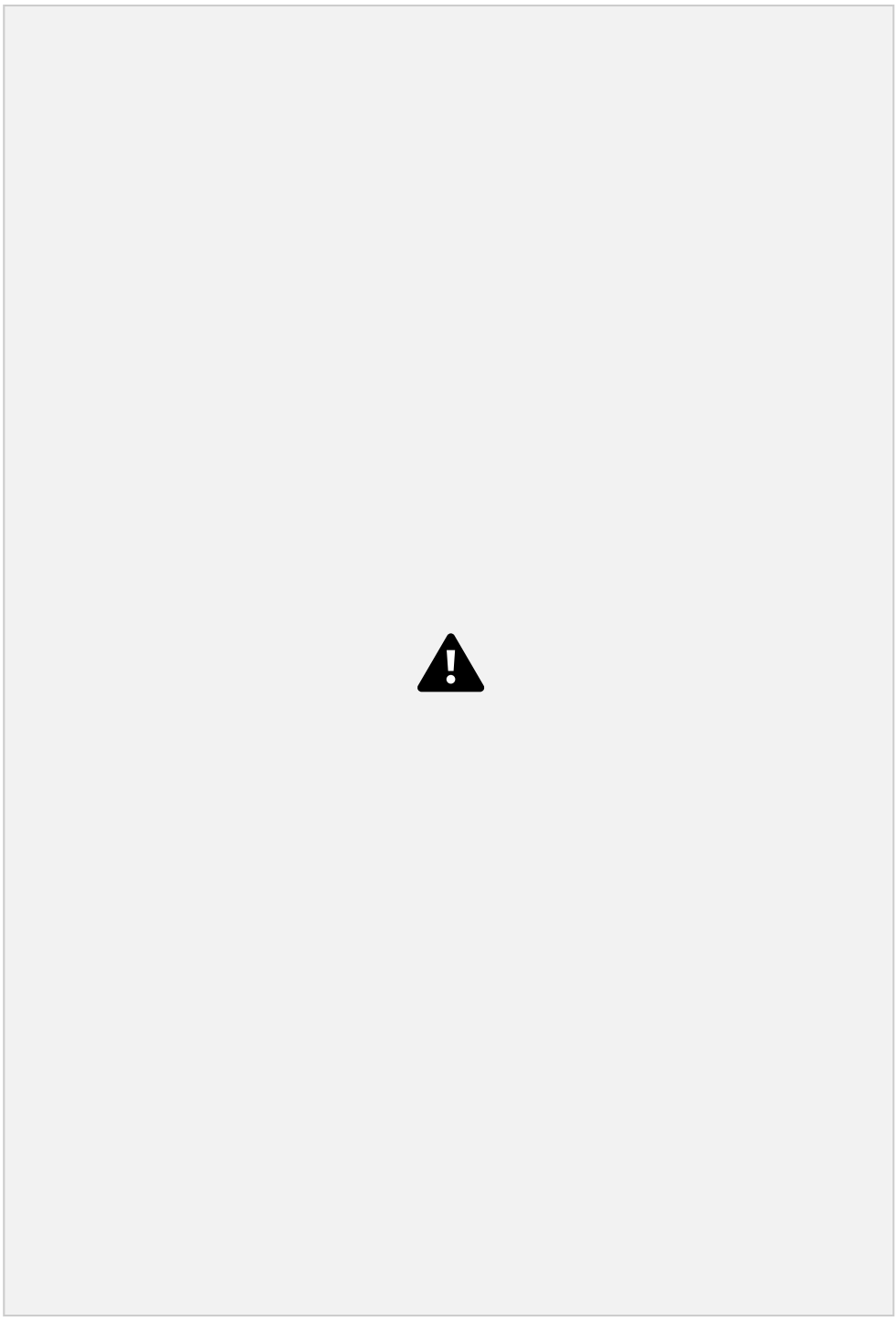
 public function destroy(Kontak $kontak)
 {
 if ($kontak->foto) {
 Storage::disk('public')->delete($kontak->foto); // Hapus  file foto
 }
 $kontak->delete(); // Hapus data dari database

 return redirect()->route('kontaks.index')->with('success',
         'Kontak berhasil dihapus.');
 }
}
```

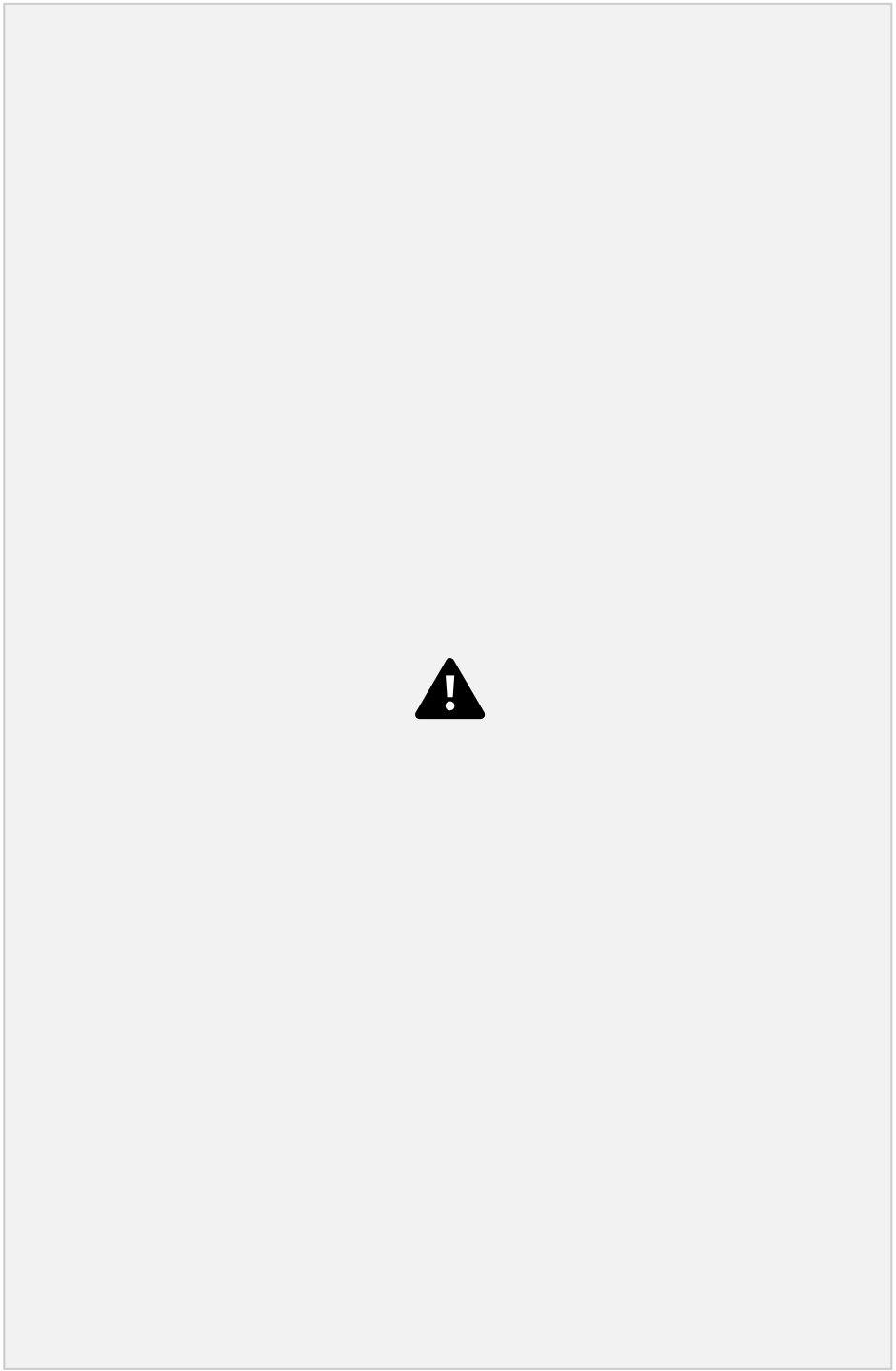
- $validated = $request->validate([...]) → hasil validasi disimpan ke variabel $validated, sehingga hanya data yang lolos validasi yang disimpan ke database (lebih aman daripada $request->all()).
- image|mimes:jpg,jpeg,png|max:2048 → file wajib berupa gambar berformat jpg, jpeg, atau png dengan ukuran maksimal 2 MB.
- $request->hasFile('foto') → foto hanya disimpan jika pengguna benar-benar memilih file, karena foto bersifat opsional (nullable).
- store('foto_kontak', 'public') → menyimpan foto ke folder storage/app/public/foto_kontak dan mengembalikan path-nya, lalu path tersebut dimasukkan ke $validated['foto'].
- if ($kontak->foto) { Storage::disk('public')->delete(...) } → menghapus foto lama

sebelum diganti, serta menghapus foto ketika kontak dihapus.

- Kontak $kontak → route model binding; Laravel otomatis mengambil data kontak berdasarkan id pada URL.



Gambar 7.38 KontakController bagian 1 (index, create, store)



Gambar 7.39 KontakController bagian 2 (show, edit, update, destroy)

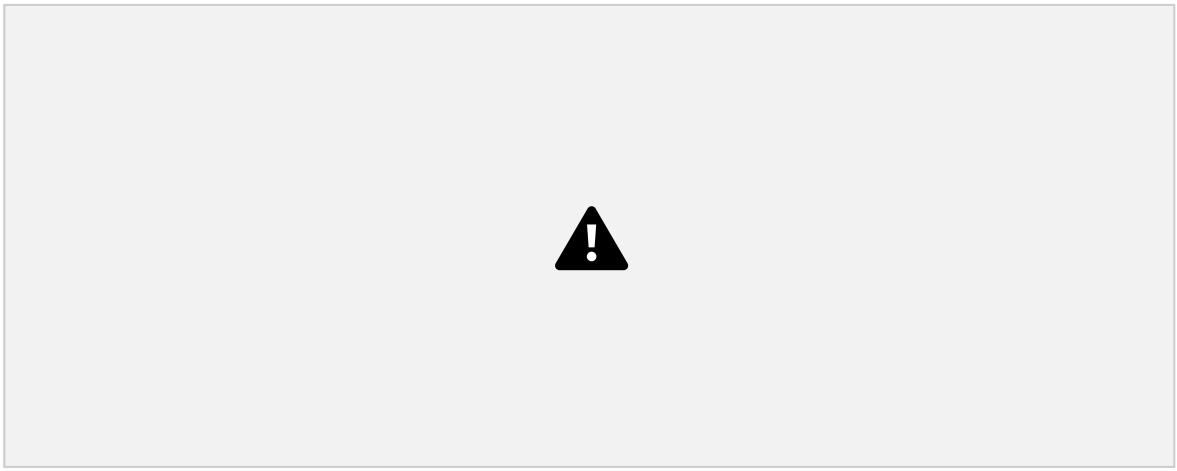
7. Buka file routes/web.php, lalu ganti seluruh isinya dengan kode berikut: `<?php`

```
use Illuminate\Support\Facades\Route;
```

```
use App\Http\Controllers\KontakController;
Route::get('/', function () {
 return redirect()->route('kontaks.index'); // Halaman awal langsung
         ke daftar kontak
});

// CRUD Kontak dengan upload foto
Route::resource('kontaks', KontakController::class);
```

Route::resource('kontaks', KontakController::class) membuat tujuh route CRUD sekaligus (lihat subbab 7.4.3). Route / diarahkan langsung ke halaman daftar kontak. Pada tahap ini halaman kontak belum dilindungi login. Perlindungan login ditambahkan pada Tahap C.

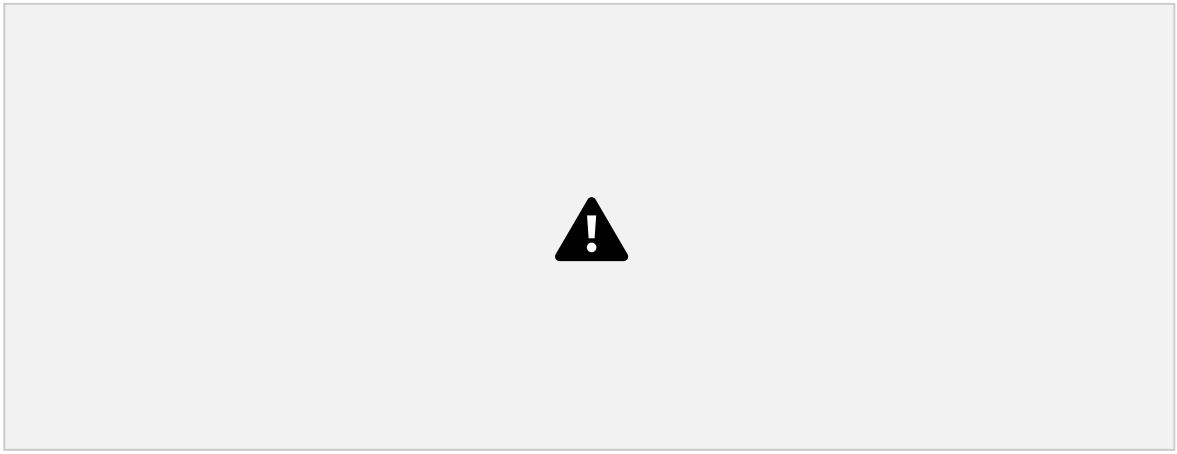


Gambar 7.40 Route resource kontaks pada routes/web.php

8. Periksa route yang sudah terdaftar dengan menjalankan perintah berikut:

```
php artisan route:list --name=kontaks
```

Terminal akan menampilkan tujuh route kontaks (index, create, store, show, edit, update, dan destroy) beserta method HTTP dan nama route-nya. Jika daftar tersebut muncul, berarti route resource sudah terdaftar dengan benar.



Gambar 7.41 Hasil php artisan route:list --name=kontaks

Tujuh route resource kontaks (index, store, create, show, update, destroy, edit) sudah terdaftar.

9. Agar tombol pagination menggunakan gaya Bootstrap (bukan Tailwind), buka file app/Providers/AppServiceProvider.php, lalu tambahkan kode berikut:

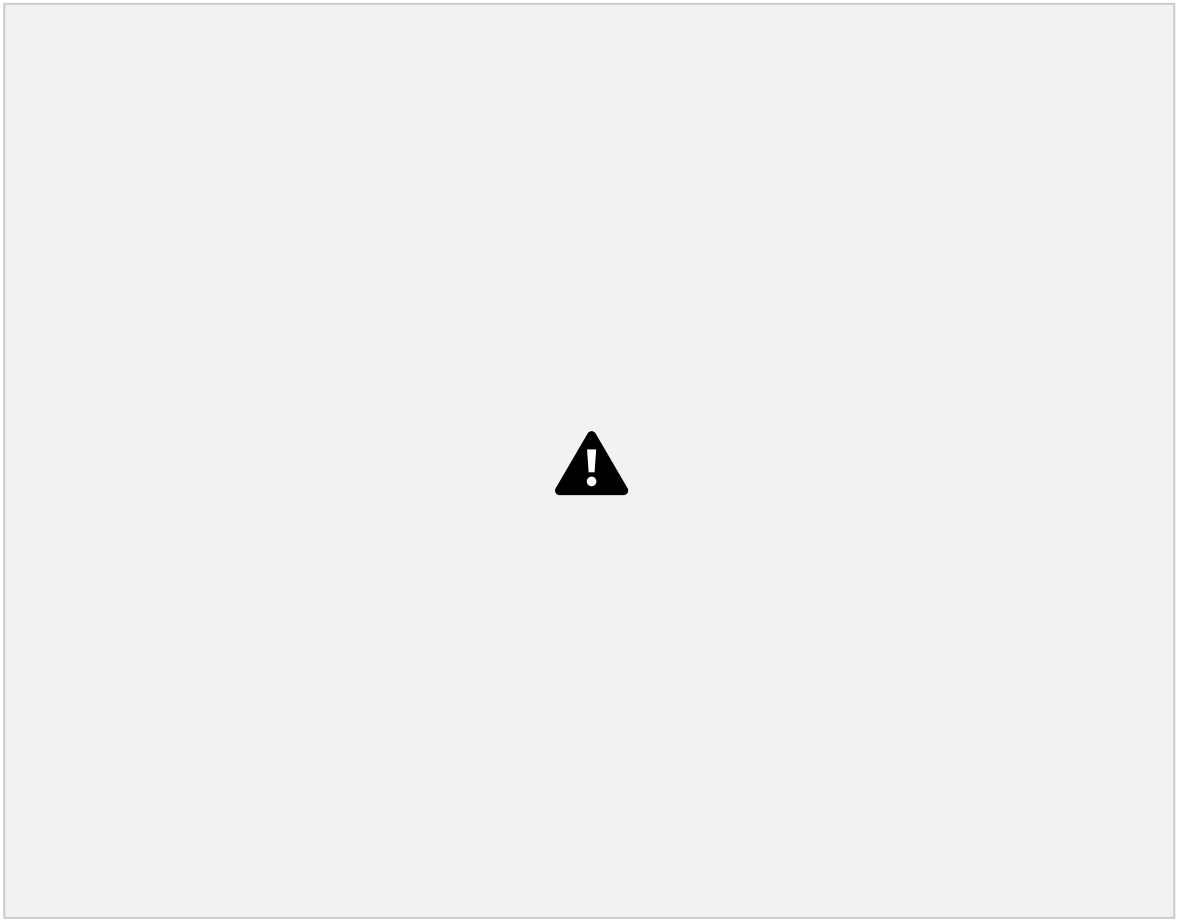
```
<?php

namespace App\Providers;

use Illuminate\Pagination\Paginator;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
 public function register(): void
 {
 //
 }

 public function boot(): void
 {
 Paginator::useBootstrapFive(); // Pagination menggunakan
         tampilan Bootstrap 5
 }
}
```



Gambar 7.42 AppServiceProvider dengan Paginator::useBootstrapFive()

10. Buat folder resources/views/kontaks, lalu buat file resources/views/kontaks/index.blade.php:

```
@extends('layouts.app')

@section('title', 'Daftar Kontak')

@section('content')
<div class="d-flex justify-content-between align-items-center
mb-3">  <h2>Daftar Kontak</h2>
 <a href="{{ route('kontaks.create') }}" class="btn btn-primary">
<i class="fas fa-plus me-1"></i>Tambah Kontak
 </a>
</div>

<table class="table table-bordered align-middle">
 <thead class="table-light">
 <tr>
 <th>No</th>
 <th>Foto</th>
```

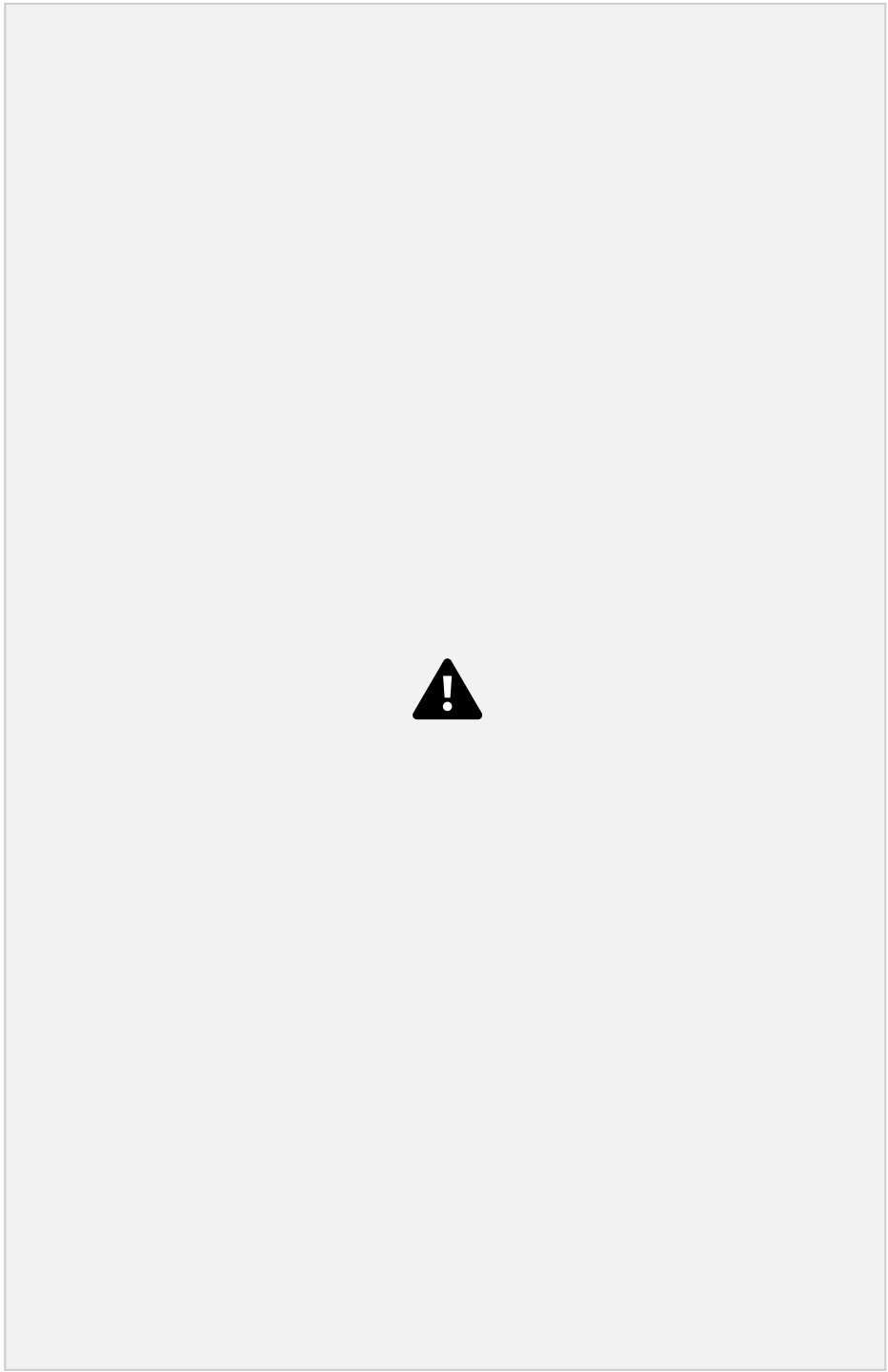
```
 <th>Nama</th>
 <th>Alamat</th>
 <th>No HP</th>
 <th>Aksi</th>
 </tr>
 </thead>
 <tbody>
 @forelse ($kontaks as $kontak)
 <tr>
 <td>{{ $kontaks->firstItem() + $loop->index }}</td>  <td>
 @if ($kontak->foto)
   <img src="{{ asset('storage/' . $kontak- >foto) }}" alt="{{
                        $kontak->nama }}"
 width="60" height="60" class="rounded  object-fit-cover">
 @else
 <span class="text-muted">Tidak ada</span>  @endif
 </td>
 <td>{{ $kontak->nama }}</td>
 <td>{{ $kontak->alamat }}</td>
 <td>{{ $kontak->no_hp }}</td>
 <td>
     <a href="{{ route('kontaks.show', $kontak) }}"  class="btn
                     btn-info btn-sm">Lihat</a>
     <a href="{{ route('kontaks.edit', $kontak) }}"  class="btn
                    btn-warning btn-sm">Edit</a>
    <form action="{{ route('kontaks.destroy',  $kontak) }}"
                method="POST" class="d-inline">
 @csrf
                        @method('DELETE')
                         <button type="submit" class="btn btn-danger
         btn-sm"
          onclick="return confirm('Apakah Anda  yakin ingin
                     menghapus?')">Hapus</button>
 </form>
 </td>
 </tr>
 @empty
 <tr>
 <td colspan="6" class="text-center">Belum ada data  kontak.</td>
 </tr>
 @endforelse
 </tbody>
</table>

{{ $kontaks->links() }}
@endsection
```

• @extends('layouts.app') dan @section('content') → menggunakan layout Bootstrap

yang dibuat pada Tahap A langkah 7 dan mengisi bagian content.

- @if ($kontak->foto) ... @else ... @endif → menampilkan foto jika ada, atau teks Tidak ada jika kontak belum memiliki foto.
- asset('storage/' . $kontak->foto) → membuat URL foto melalui symbolic link public/storage.
- {{ $kontaks->links() }} → menampilkan tombol pagination bergaya Bootstrap karena Paginator::useBootstrapFive() sudah diaktifkan.



Gambar 7.43 View index.blade.php (daftar kontak)

11. Buat file resources/views/kontaks/create.blade.php:

```
@extends('layouts.app')

@section('title', 'Tambah Kontak')
@section('content')
```

```
<div class="row justify-content-center">
 <div class="col-md-8">
 <div class="card">
 <div class="card-header">Tambah Kontak Baru</div>  <div
class="card-body">
 {{-- enctype wajib ada agar file foto ikut terkirim - -}}
  <form action="{{ route('kontaks.store') }}"  method="POST"
               enctype="multipart/form-data">
 @csrf

 <div class="mb-3">
 <label for="nama" class="form
         label">Nama</label>
 <input type="text" id="nama" name="nama"  value="{{ old('nama')
         }}"
 class="form-control @error('nama') is invalid @enderror" required>
  @error('nama') <div class="invalid feedback">{{ $message
                    }}</div> @enderror
 </div>

 <div class="mb-3">
  <label for="alamat" class="form label">Alamat</label>
    <textarea id="alamat" name="alamat" rows="3"  class="form-control
   @error('alamat')  is-invalid @enderror" required>{{ old('alamat')
    }}</textarea>  @error('alamat') <div class="invalid feedback">{{
                      $message }}</div> @enderror
 </div>

 <div class="mb-3">
 <label for="no_hp" class="form-label">No  HP</label>
 <input type="text" id="no_hp" name="no_hp"  value="{{
         old('no_hp') }}"
 class="form-control @error('no_hp') is invalid @enderror" required>
  @error('no_hp') <div class="invalid feedback">{{ $message
                    }}</div> @enderror
 </div>

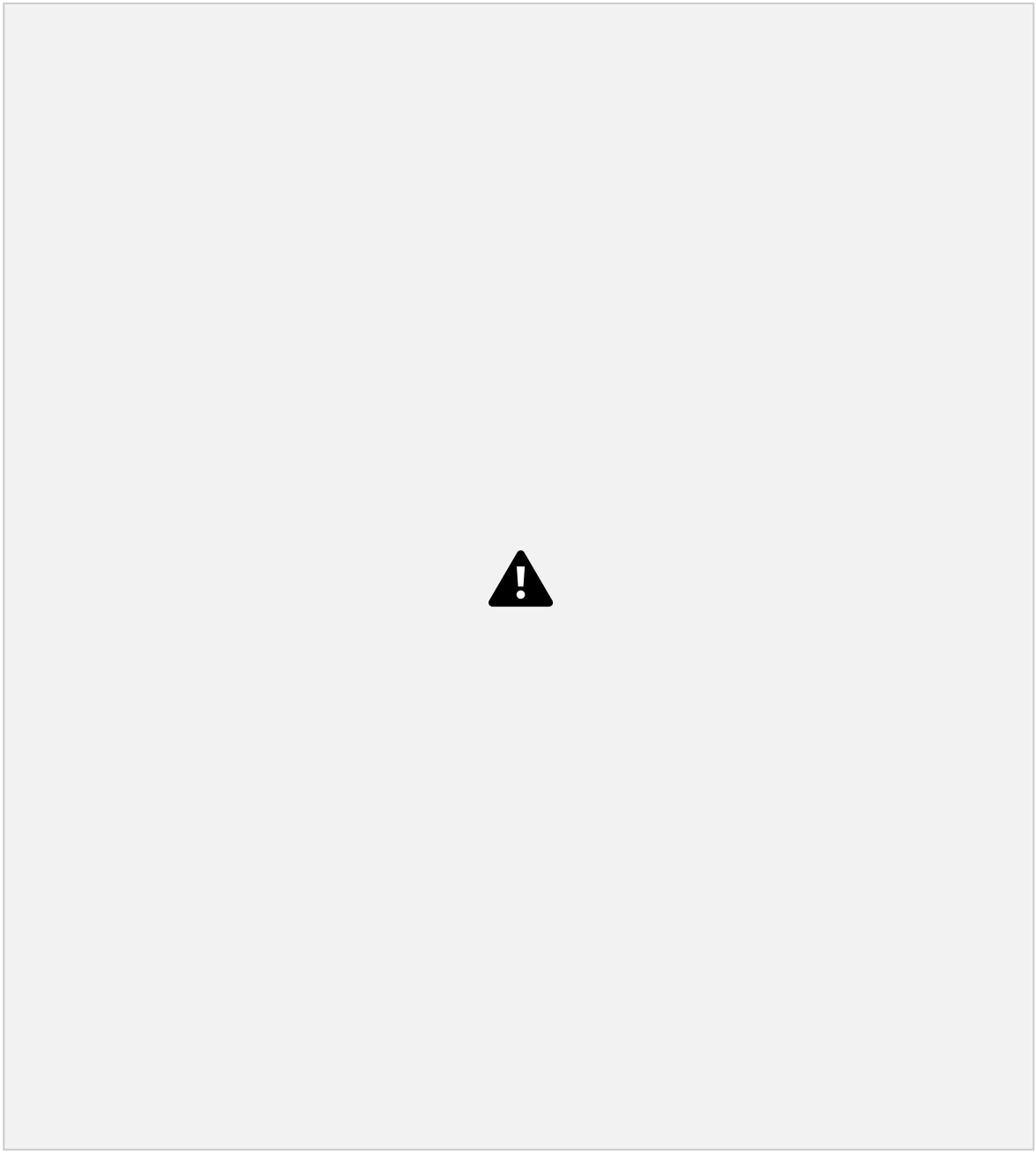
 <div class="mb-3">
 <label for="foto" class="form-label">Foto (jpg,  jpeg, png - maks. 2
         MB)</label>
 <input type="file" id="foto" name="foto"  accept="image/*"
 class="form-control @error('foto') is invalid @enderror">
  @error('foto') <div class="invalid feedback">{{ $message
                    }}</div> @enderror
 </div>

           <button type="submit" class="btn btn
```

```
                primary">Simpan</button>
 <a href="{{ route('kontaks.index') }}" class="btn
         btn-secondary">Batal</a>
 </form>
 </div>
 </div>
 </div>
</div>
@endsection
```

• enctype="multipart/form-data" → wajib ada agar foto ikut terkirim ke controller. • accept="image/*" → jendela pemilihan file hanya menampilkan file gambar. Atribut ini hanya membantu pengguna; pemeriksaan sebenarnya tetap dilakukan oleh validasi di controller.

• @error('foto') is-invalid @enderror dan → memberi garis merah pada input dan menampilkan pesan galat di bawahnya jika validasi gagal.



Gambar 7.44 View create.blade.php (form tambah kontak)

12. Buat file resources/views/kontaks/edit.blade.php:

```
@extends('layouts.app')

@section('title', 'Edit Kontak')

@section('content')
<div class="row justify-content-center">
 <div class="col-md-8">
 <div class="card">
 <div class="card-header">Edit Kontak</div>
 <div class="card-body">
```

```
 <form action="{{ route('kontaks.update', $kontak) }}" method="POST"
           enctype="multipart/form-data">
 @csrf
                   @method('PUT')

 <div class="mb-3">
 <label for="nama" class="form-label">Nama</label> <input type="text" id="nama"
                        name="nama" value="{{ old('nama',
           $kontak->nama) }}"
 class="form-control @error('nama') is-invalid @enderror"  required>
 @error('nama') <div class="invalid-feedback">{{ $message }}</div>  @enderror
 </div>

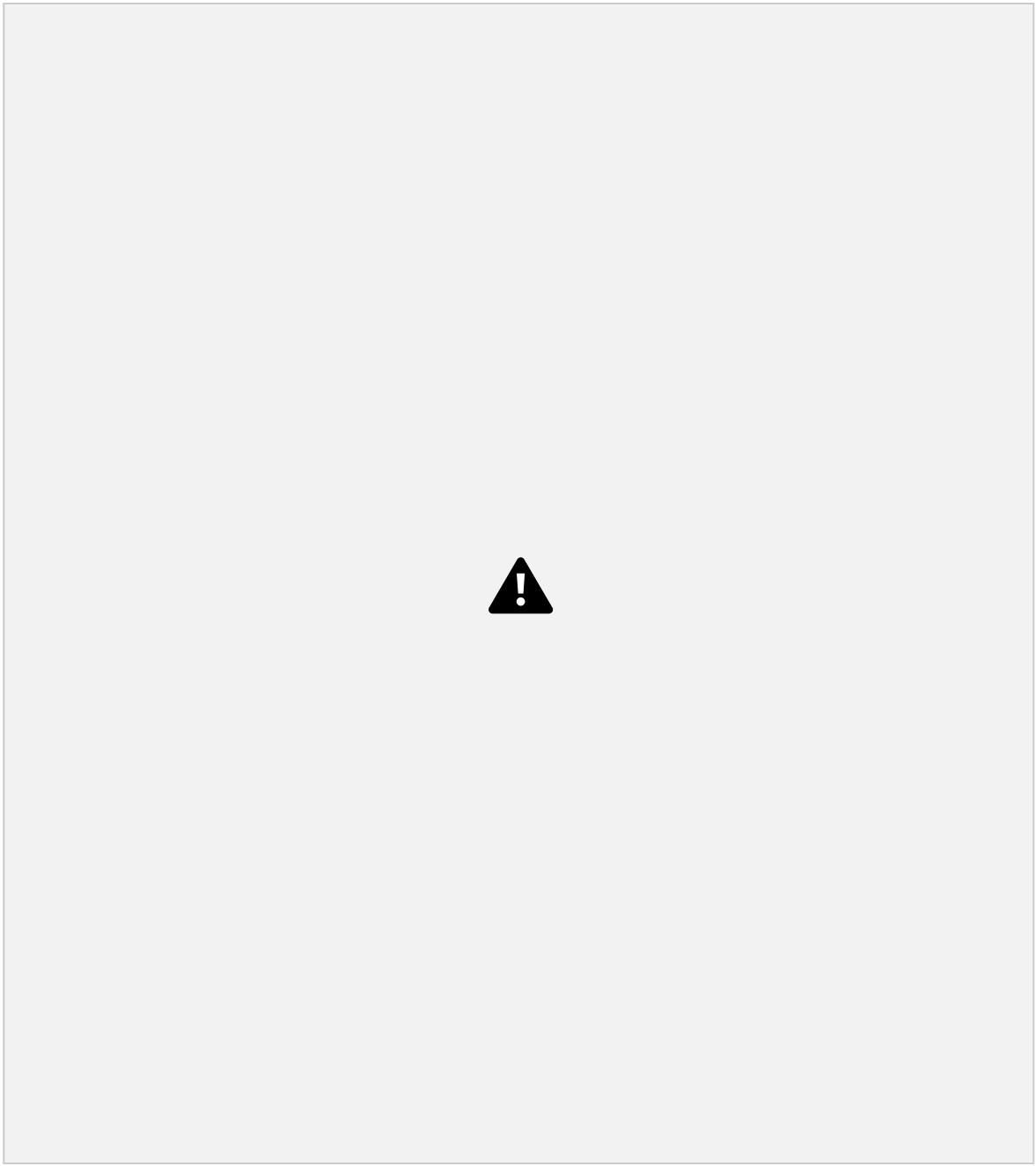
 <div class="mb-3">
  <label for="alamat" class="form-label">Alamat</label> <textarea id="alamat"
                           name="alamat" rows="3"
    class="form-control @error('alamat') is-invalid @enderror"  required>{{ old('alamat',
                                $kontak->alamat) }}</textarea>
 @error('alamat') <div class="invalid-feedback">{{ $message }}</div>  @enderror
 </div>

 <div class="mb-3">
 <label for="no_hp" class="form-label">No HP</label> <input type="text" id="no_hp"
                        name="no_hp" value="{{ old('no_hp',
           $kontak->no_hp) }}"
 class="form-control @error('no_hp') is-invalid @enderror"  required>
 @error('no_hp') <div class="invalid-feedback">{{ $message }}</div>  @enderror
 </div>

 <div class="mb-3">
 <label class="form-label d-block">Foto Saat Ini</label>  @if ($kontak->foto)
 <img src="{{ asset('storage/' . $kontak->foto) }}"  alt="{{ $kontak->nama }}"
 width="120" class="rounded mb-2">
 @else
 <p class="text-muted">Belum ada foto.</p>
 @endif
                        <input type="file" id="foto" name="foto" accept="image/*"
 class="form-control @error('foto') is-invalid @enderror">  <small
class="text-muted">Kosongkan jika tidak ingin mengganti  foto.</small>
 @error('foto') <div class="invalid-feedback">{{ $message }}</div>  @enderror
 </div>

    <button type="submit" class="btn btn-primary">Update</button> <a href="{{
                  route('kontaks.index') }}" class="btn btn
           secondary">Batal</a>
 </form>
 </div>
 </div>
 </div>
</div>
@endsection
```

- @method('PUT') → form diarahkan ke method update() pada KontakController.
- old('nama', $kontak->nama) → menampilkan data lama dari database sebagai isi awal input.
- Bagian Foto Saat Ini → menampilkan foto yang sedang digunakan, sehingga pengguna dapat memutuskan apakah foto perlu diganti.



Gambar 7.45 View edit.blade.php (form ubah kontak)

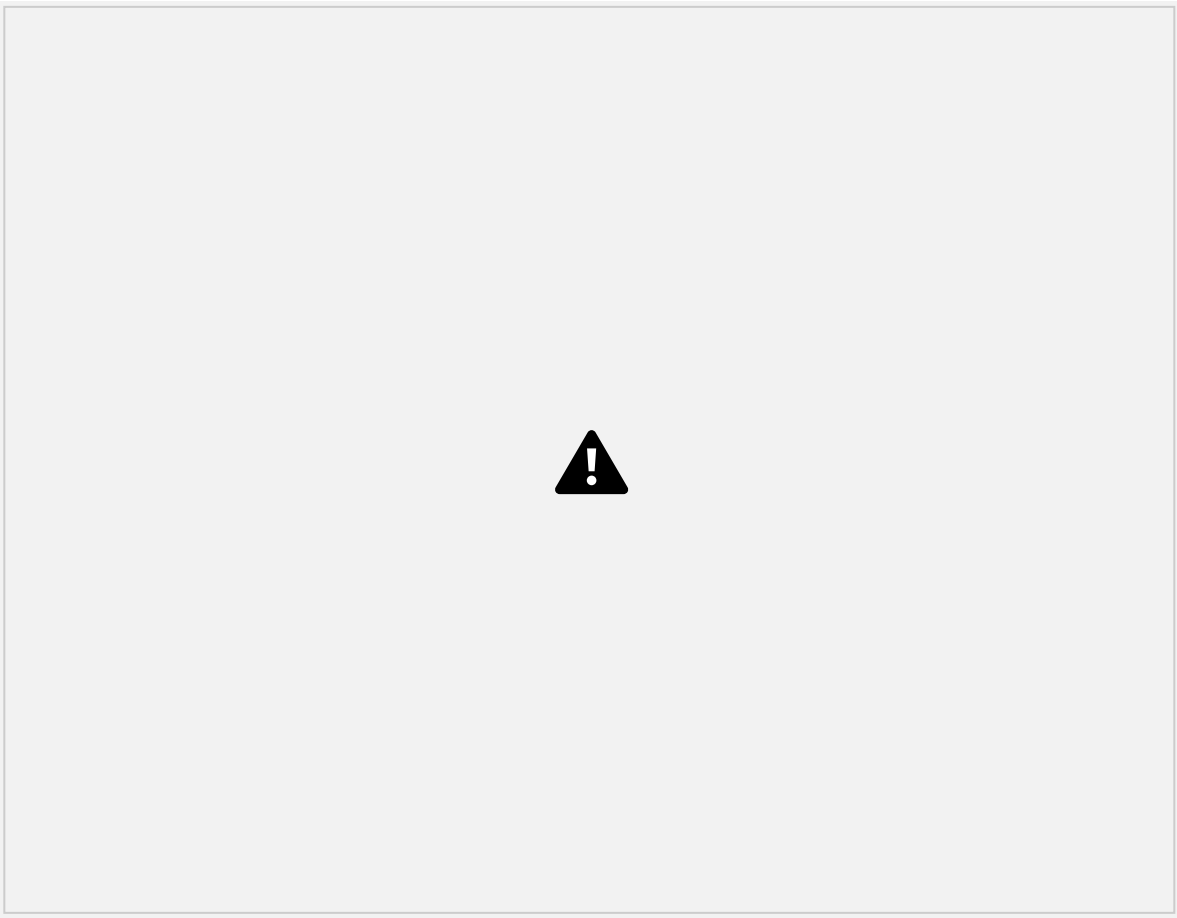
13. Buat file resources/views/kontaks/show.blade.php:

```
@extends('layouts.app')

@section('title', 'Detail Kontak')
@section('content')
<div class="row justify-content-center">
 <div class="col-md-8">
 <div class="card">
```

```
 <div class="card-header">Detail Kontak</div>  <div
class="card-body text-center">
 @if ($kontak->foto)
 <img src="{{ asset('storage/' . $kontak->foto) }}"  alt="{{
         $kontak->nama }}"
 width="180" class="rounded mb-3">  @endif
 <h5 class="card-title">{{ $kontak->nama }}</h5>  <p
class="card-text"><strong>Alamat:</strong>  {{ $kontak->alamat
}}</p>
 <p class="card-text"><strong>No HP:</strong>  {{
         $kontak->no_hp }}</p>
          <p class="card-text"><small class="text
       muted">Ditambahkan {{ $kontak->created_at-
         >diffForHumans() }}</small></p>

   <a href="{{ route('kontaks.edit', $kontak) }}"  class="btn
                    btn-warning">Edit</a>
 <form action="{{ route('kontaks.destroy', $kontak) }}"  method="POST"
         class="d-inline">
 @csrf
                   @method('DELETE')
                   <button type="submit" class="btn btn-danger"
 onclick="return confirm('Apakah Anda yakin  ingin
         menghapus?')">Hapus</button>
 </form>
 <a href="{{ route('kontaks.index') }}" class="btn btn
         secondary">Kembali</a>
 </div>
 </div>
 </div>
</div>
@endsection
```



Gambar 7.46 View show.blade.php (detail kontak)

14. Jalankan aplikasi dengan perintah php artisan serve, lalu uji fitur yang telah dibuat dengan langkah berikut:

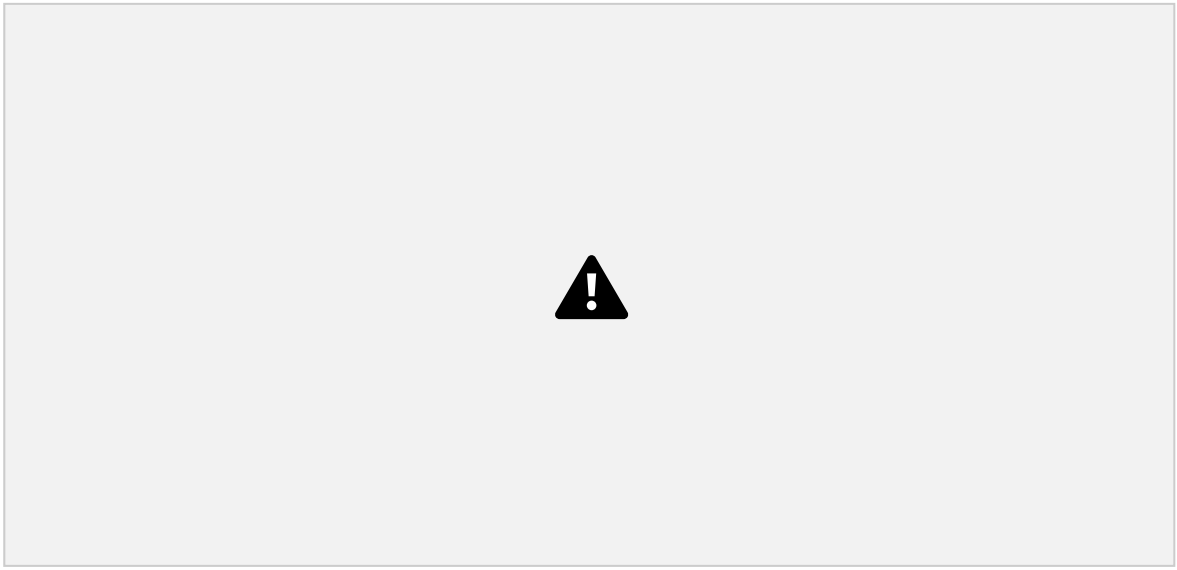
    a. Buka http://127.0.0.1:8000 → Anda otomatis diarahkan ke halaman daftar kontak (/kontaks) yang masih kosong.



Gambar 7.47 Daftar Kontak masih kosong

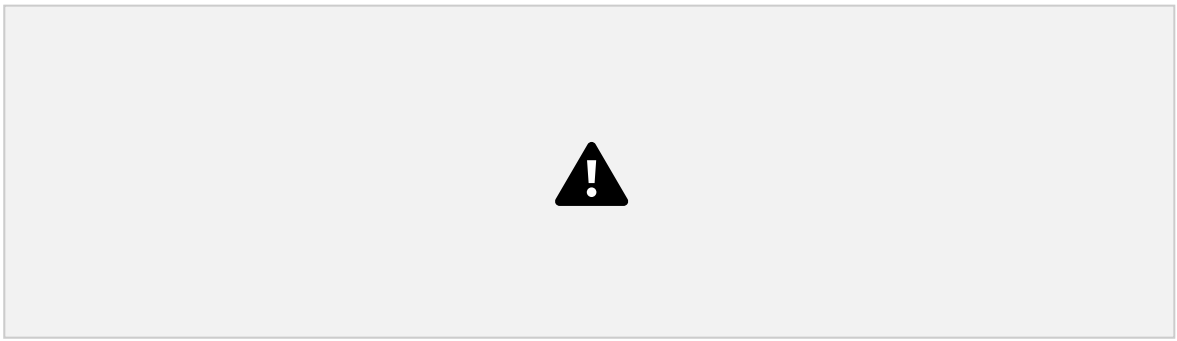
Membuka http://127.0.0.1:8123 langsung diarahkan ke /kontaks; tabel menampilkan pesan "Belum ada data kontak." dari blok @empty.

b. Klik tombol Tambah Kontak. Isi seluruh data dan pilih foto, lalu klik Simpan → kontak beserta foto muncul pada tabel.



Gambar 7.48 Form Tambah Kontak (Create)

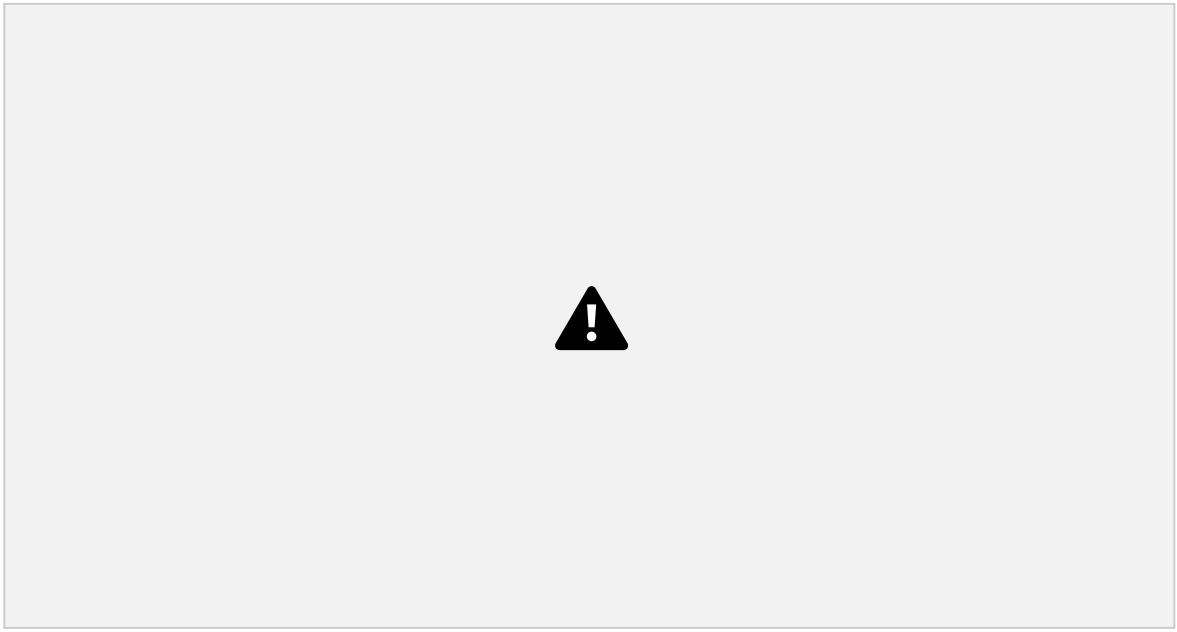
Form tambah kontak berisi nama, alamat, nomor HP, dan input foto dengan enctype multipart/form-data.



Gambar 7.49 Pesan berhasil setelah kontak ditambahkan

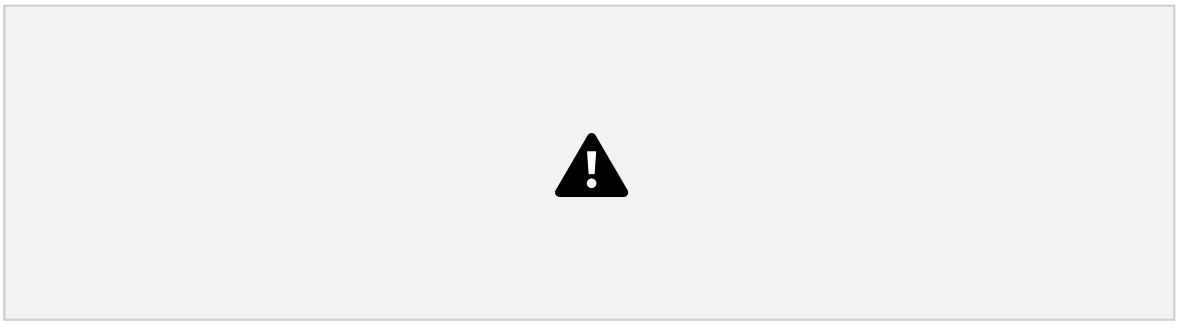
Setelah klik Simpan, kontak beserta foto tersimpan dan halaman index menampilkan pesan "Kontak berhasil ditambahkan."

c. Tambahkan juga kontak tanpa foto → kolom foto berisi teks Tidak ada.



Gambar 7.50 Daftar Kontak halaman 1 dengan kontak tanpa foto

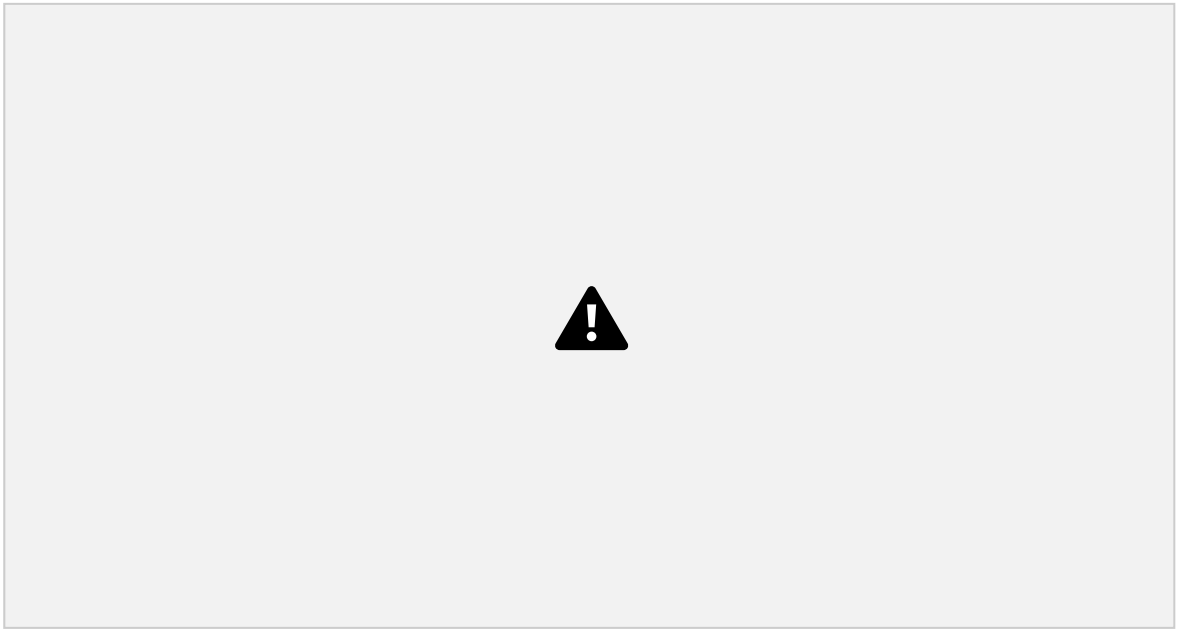
Kontak tanpa foto (Rina Marlina) menampilkan teks Tidak ada pada kolom foto. Data diurutkan dari yang terbaru, 5 data per halaman, dengan pagination bergaya Bootstrap 5.



Gambar 7.51 Daftar Kontak halaman 2

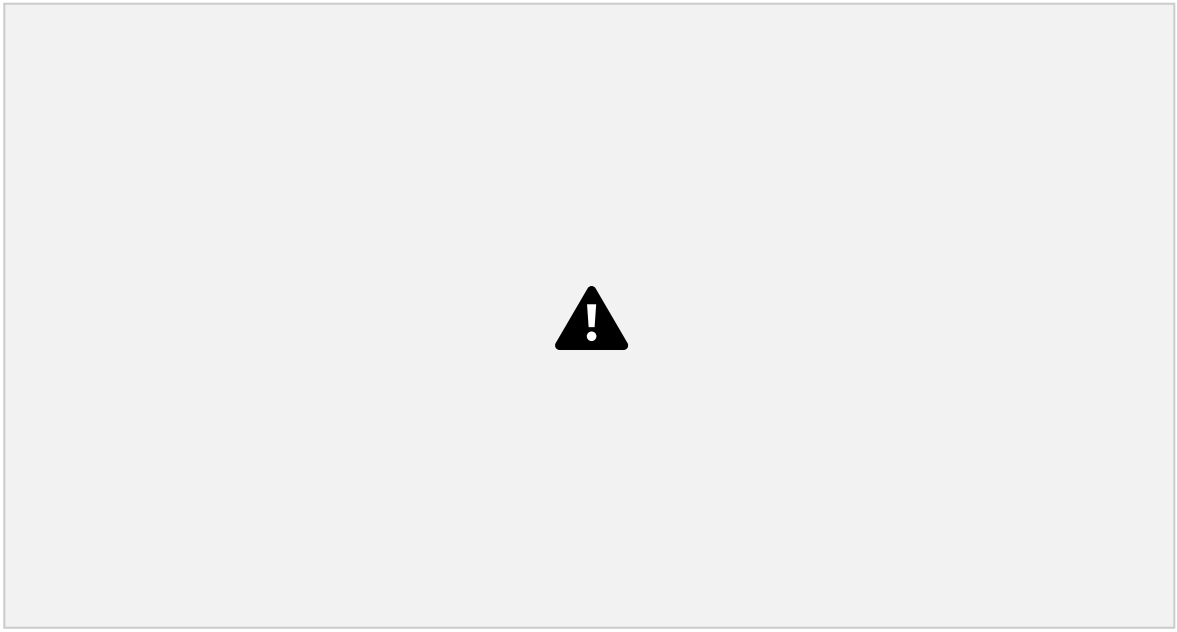
Halaman kedua pagination; nomor urut berlanjut karena memakai firstItem() + $loop->index.

d. Coba unggah file PDF atau gambar berukuran lebih dari 2 MB pada input foto → pesan galat validasi muncul di bawah input.



Gambar 7.52 Pesan galat saat mengunggah file PDF

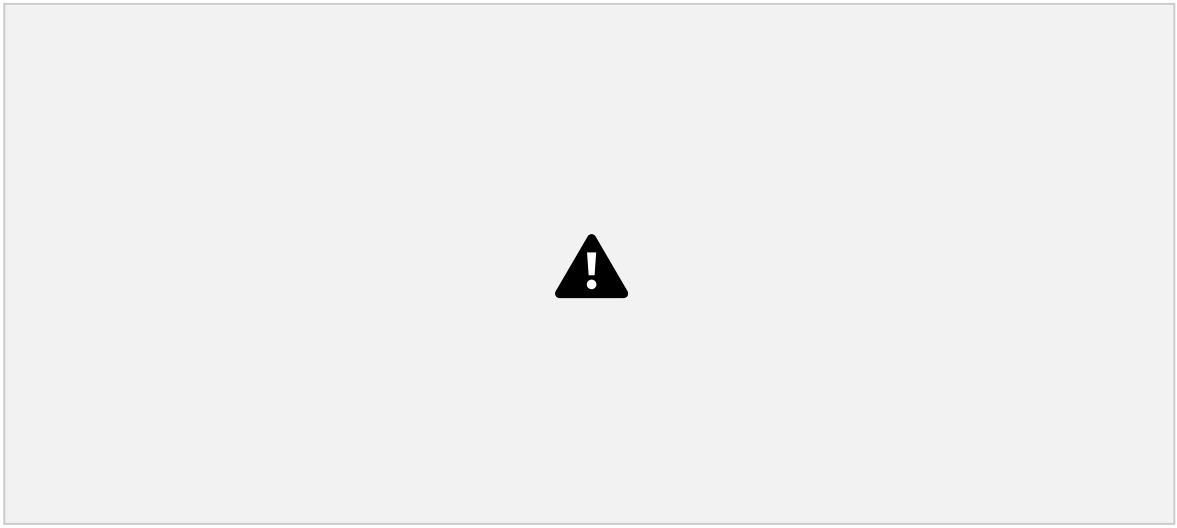
Validasi image dan mimes menolak file PDF; input foto diberi garis merah (is-invalid) dan pesan galat tampil di bawahnya.



Gambar 7.53 Pesan galat saat mengunggah foto lebih dari 2 MB

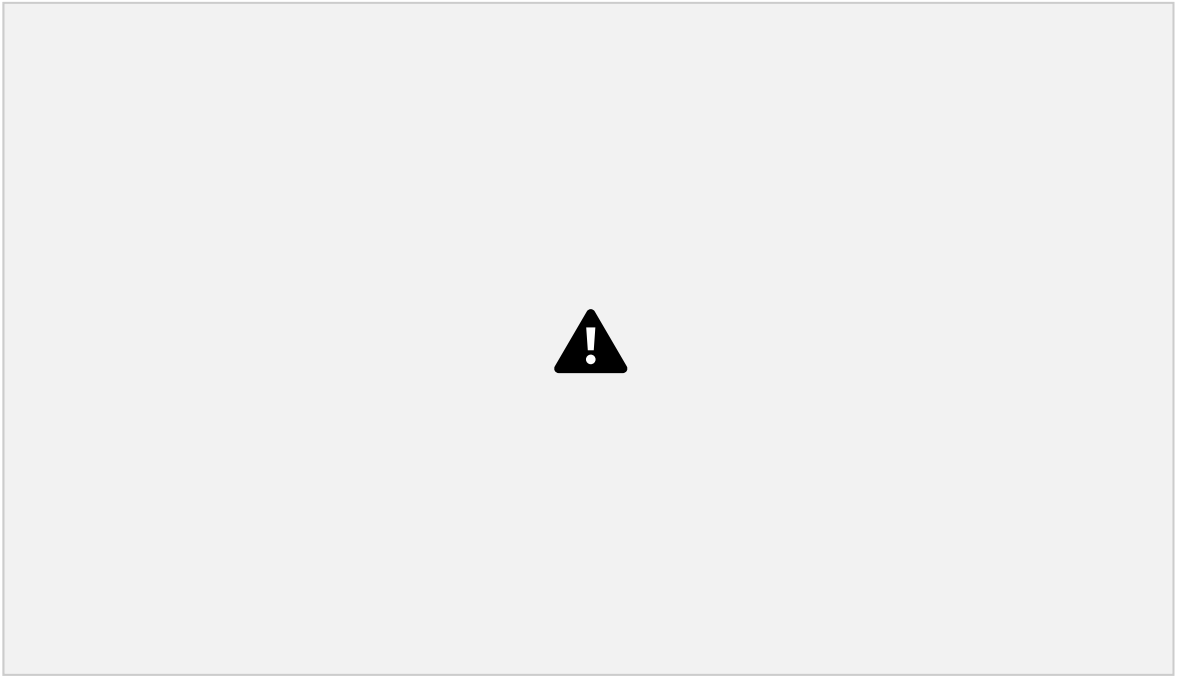
Validasi max:2048 menolak gambar berukuran sekitar 3 MB; ringkasan galat dari layout ikut tampil di bagian atas.

e. Klik Edit, ganti foto, lalu klik Update → foto lama di folder storage/app/public/foto_kontak terhapus dan diganti foto baru.



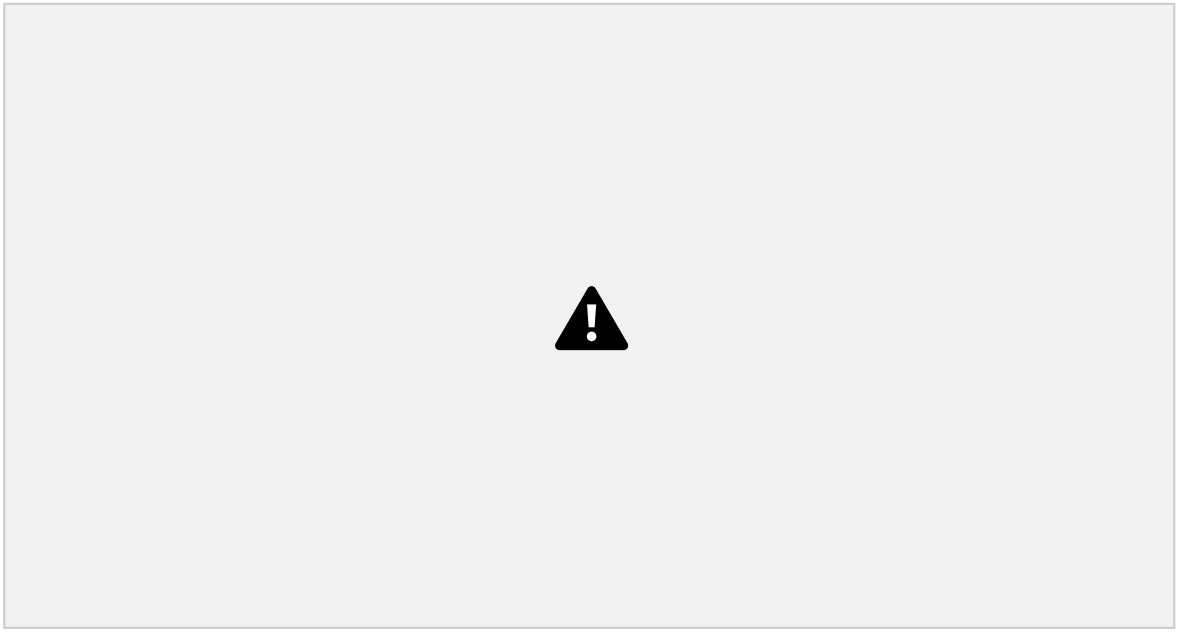
Gambar 7.54 Halaman Detail Kontak (Show) sebelum foto diganti

Halaman detail menampilkan foto, nama, alamat, nomor HP, dan waktu data ditambahkan.



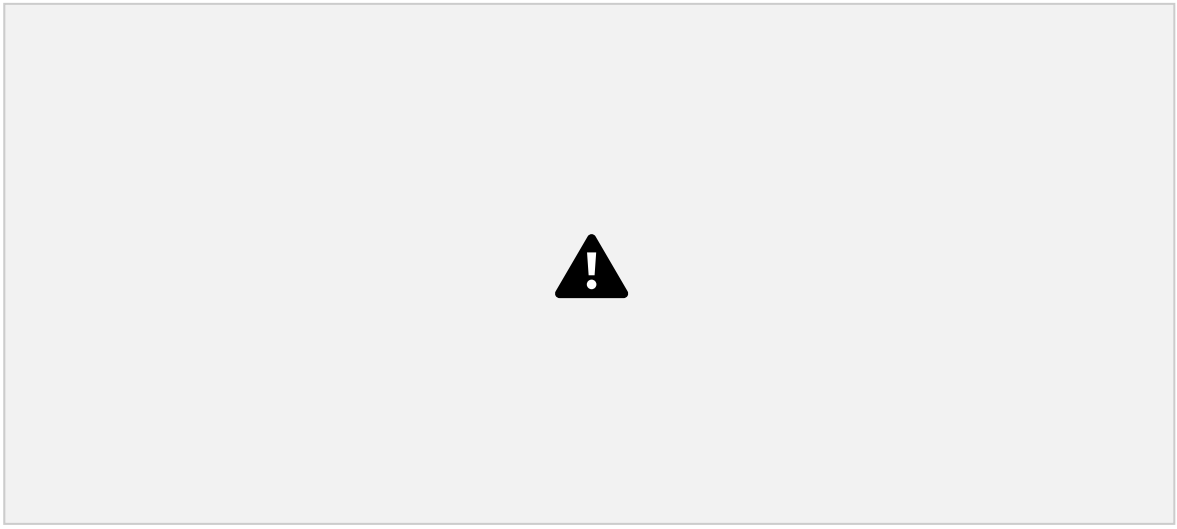
Gambar 7.55 Form Edit Kontak sebelum foto diganti

Form edit menampilkan data lama dan foto saat ini; nomor HP diubah dan foto pengganti dipilih.



Gambar 7.56 Pesan berhasil setelah kontak diperbarui

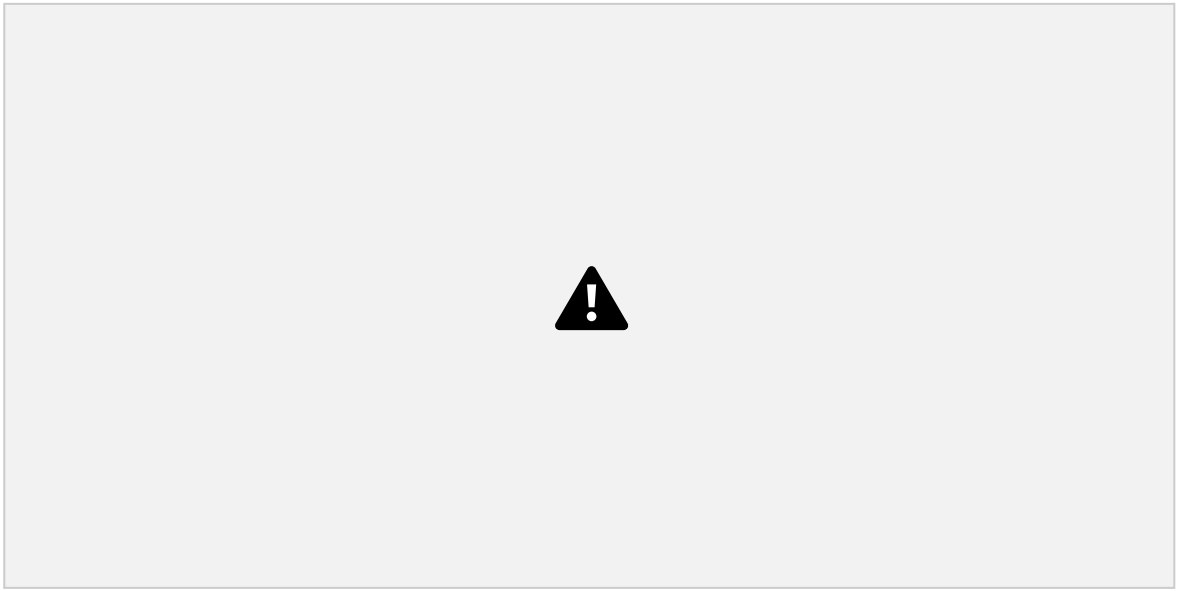
Setelah klik Update, data berubah serta tampil pesan "Kontak berhasil diperbarui."



Gambar 7.57 Detail kontak setelah foto diganti

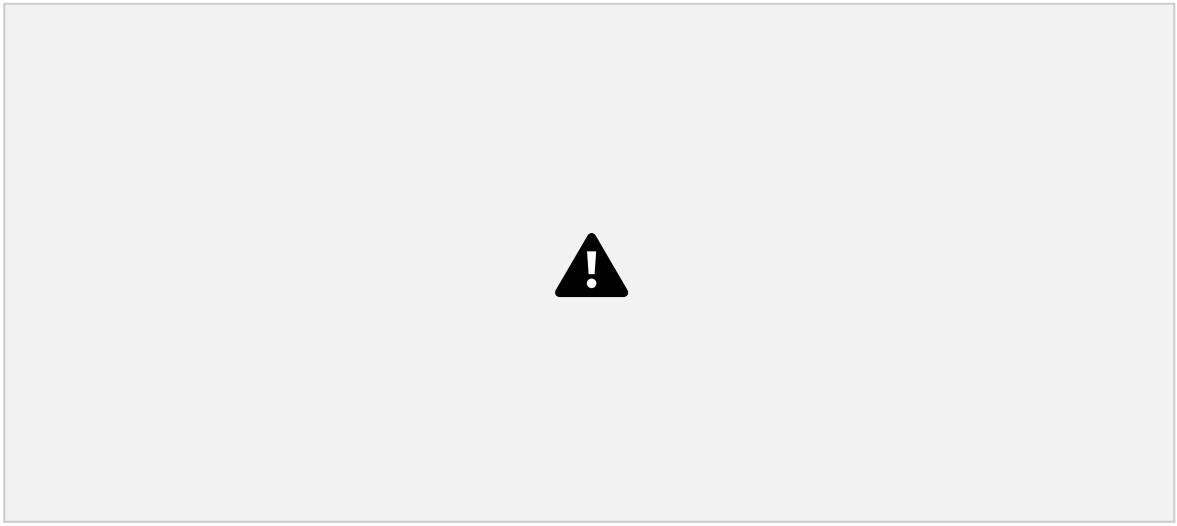
Detail kontak menampilkan foto pengganti; foto lama sudah dihapus dari storage/app/public/foto_kontak.

f. Klik Hapus → data kontak di tabel kontaks dan file foto di storage ikut terhapus.



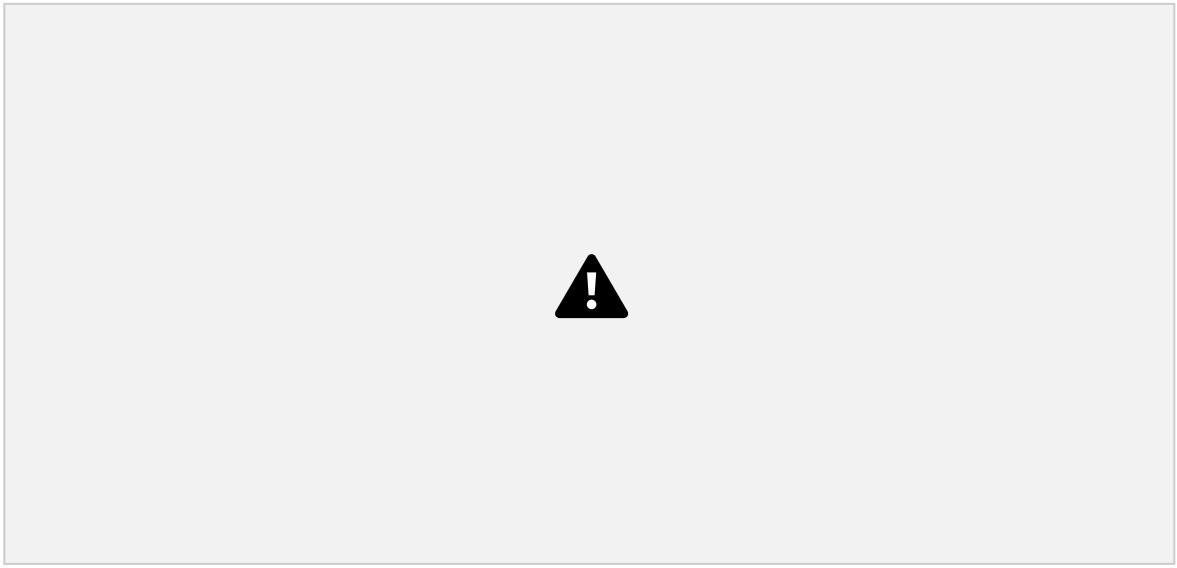
Gambar 7.58 Pesan berhasil setelah kontak dihapus

Setelah konfirmasi, kontak Andi Pratama beserta file fotonya terhapus dan tampil pesan "Kontak berhasil dihapus."



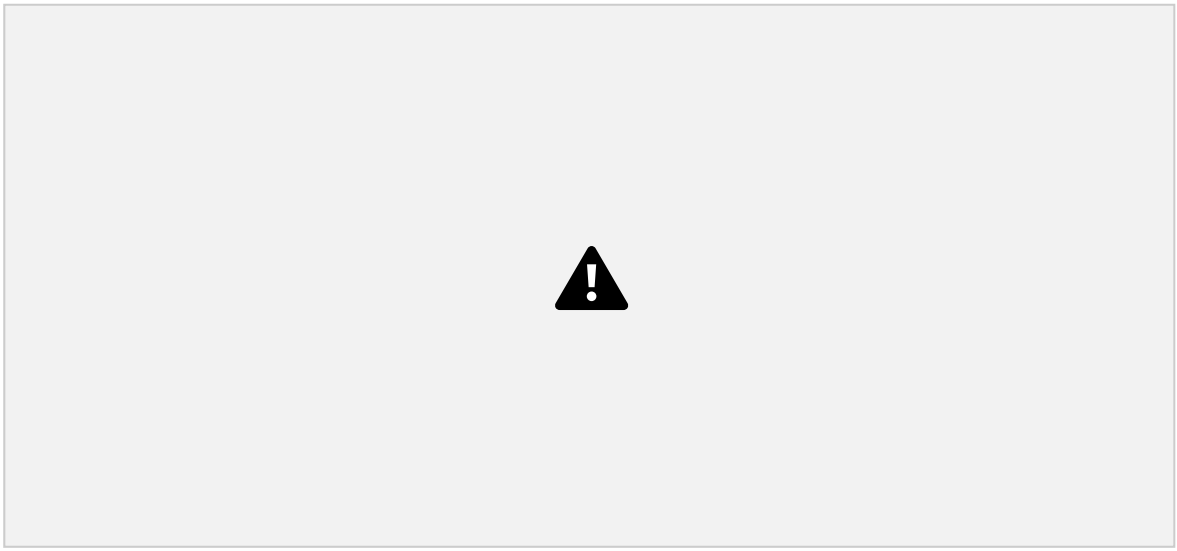
Gambar 7.59 Isi tabel kontaks pada phpMyAdmin

Database hanya menyimpan path foto (kolom foto); kontak tanpa foto bernilai NULL.



Gambar 7.60 Isi folder storage/app/public/foto_kontak

Folder foto_kontak berisi 4 file dengan nama acak hasil store(), sama dengan jumlah kontak yang memiliki foto. Foto lama Budi Santoso dan foto Andi Pratama sudah terhapus.



Gambar 7.61 Hasil pengujian otomatis (Pest) fitur CRUD Kontak

Seluruh feature test untuk redirect, tambah, validasi foto, edit, dan hapus kontak berhasil (passed).

Tahap C: Menambahkan Authentication (Register, Login, dan Logout) secara Manual

Pada Tahap B, halaman kontak masih dapat dibuka oleh siapa saja. Pada tahap ini kita menambahkan fitur register, login, dan logout tanpa Breeze, yaitu dengan menulis sendiri controller, route, dan view-nya, lalu melindungi seluruh route kontak menggunakan middleware auth. Tabel users dan model User sudah tersedia dari project Laravel bawaan (Tahap A

langkah 6), sehingga tidak perlu membuat migration baru.

1. Buat controller untuk authentication dengan perintah berikut:

```
php artisan make:controller AuthController
```

2. Buka file app/Http/Controllers/AuthController.php, lalu ganti seluruh isinya dengan kode berikut:

```
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

class AuthController extends Controller
{
 /**
 * Menampilkan form register.
 */
 public function showRegister(): View
 {
 return view('auth.register');
 }

 /**
 * Memvalidasi data, membuat akun baru, lalu langsung login.
*/
 public function register(Request $request): RedirectResponse
{
 $validated = $request->validate([
 'name' => 'required|string|max:255',
 'email' =>
        'required|string|email|max:255|unique:users,email',
 'password' => 'required|string|min:8|confirmed', // Harus  sama
        dengan password_confirmation
 ]);

 $user = User::create($validated); // Password otomatis di-hash  oleh
        cast 'hashed' pada model User

 Auth::login($user);
       $request->session()->regenerate(); // Mencegah session fixation
```

```
 return redirect()->route('kontaks.index')->with('success',
         'Registrasi berhasil, selamat datang '.$user->name.'.');
 }
 /**
 * Menampilkan form login.
 */
 public function showLogin(): View
 {
 return view('auth.login');
 }

 /**
 * Memeriksa email dan password, lalu membuat session login.
*/
 public function login(Request $request): RedirectResponse
{
 $credentials = $request->validate([
 'email' => 'required|string|email',
 'password' => 'required|string',
 ]);

 if (Auth::attempt($credentials, $request->boolean('remember')))  {
 $request->session()->regenerate();

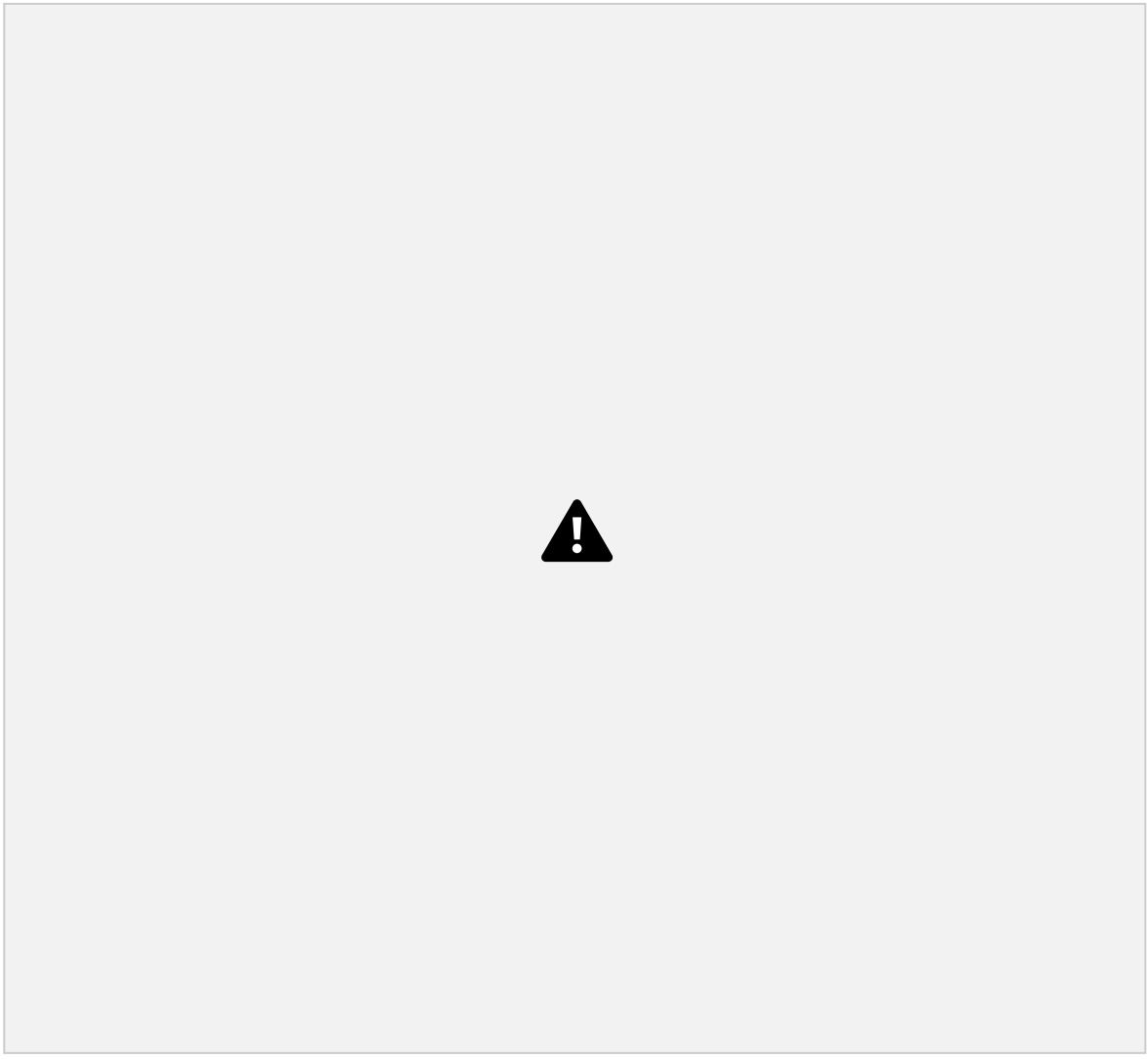
      return redirect()->intended(route('kontaks.index'))-
              >with('success', 'Login berhasil.');
 }

 return back()
 ->withErrors(['email' => 'Email atau password salah.'])
->onlyInput('email');
 }

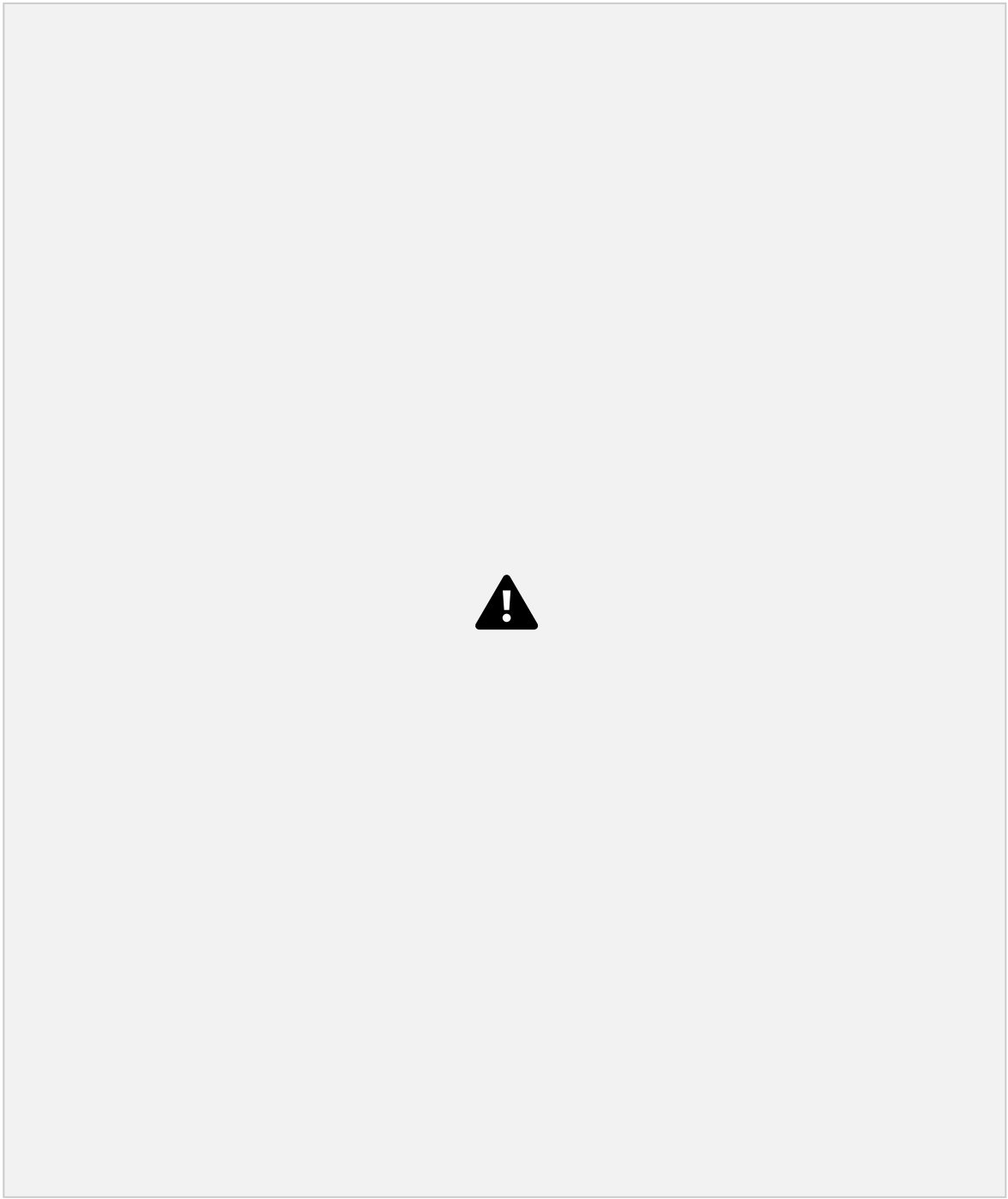
 /**
 * Mengakhiri session login.
 */
 public function logout(Request $request): RedirectResponse
{
 Auth::logout();
 $request->session()->invalidate(); // Hapus seluruh data  session
       $request->session()->regenerateToken(); // Buat token CSRF baru

 return redirect()->route('login')->with('success', 'Anda
         berhasil logout.');
 }
}
```

- unique:users,email → email yang sama tidak dapat didaftarkan dua kali, sedangkan aturan confirmed mewajibkan input password_confirmation bernilai sama dengan password.
- User::create($validated) → password otomatis di-hash (bcrypt) karena model User bawaan Laravel memiliki cast 'password' => 'hashed', sehingga password tidak pernah disimpan sebagai teks biasa.
- Auth::login($user) → langsung memasukkan pengguna yang baru mendaftar ke dalam session login.
- Auth::attempt($credentials, $request->boolean('remember')) → mencocokkan email dan password dengan data di tabel users; bernilai true jika cocok. Opsi remember (Ingat saya) membuat login bertahan lebih lama.
- $request->session()->regenerate() → membuat ID session baru setelah login untuk mencegah serangan session fixation.
- redirect()->intended(...) → mengembalikan pengguna ke halaman yang tadi ingin dibuka sebelum diarahkan ke halaman login.
- back()->withErrors([...])->onlyInput('email') → kembali ke form login dengan pesan galat dan hanya mengisi ulang email (password tidak dikembalikan).
- Auth::logout(), session()->invalidate(), dan session()->regenerateToken() → mengakhiri login, menghapus seluruh data session, dan membuat token CSRF baru.



Gambar 7.62 AuthController bagian 1 (showRegister, register)



Gambar 7.63 AuthController bagian 2 (showLogin, login, logout)

3. Buka file routes/web.php, lalu ganti seluruh isinya dengan kode

   berikut: `<?php`

```
use App\Http\Controllers\AuthController;
use App\Http\Controllers\KontakController;
use Illuminate\Support\Facades\Route;
```

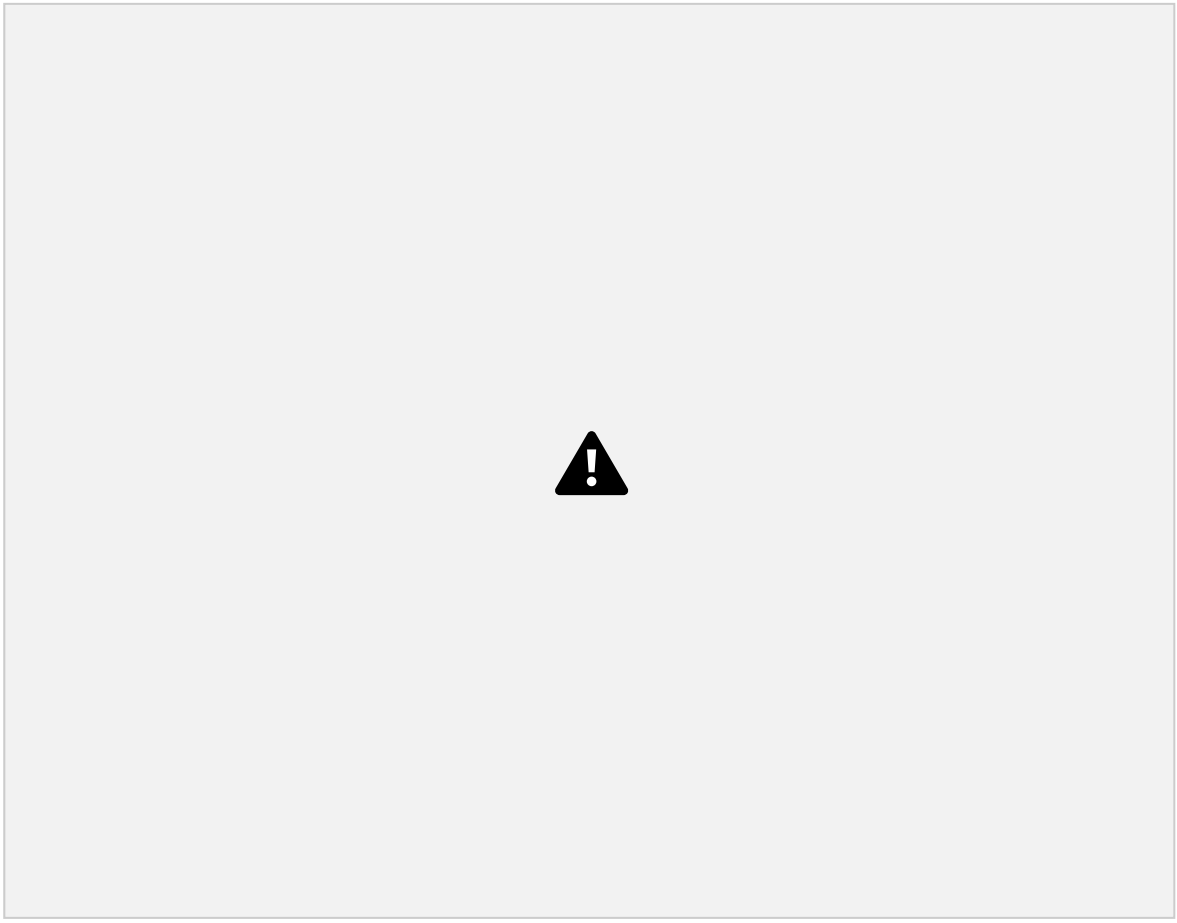
```
Route::get('/', function () {
 return redirect()->route('kontaks.index'); // Halaman awal langsung
         ke daftar kontak
});

// Halaman register dan login hanya untuk pengguna yang belum
login Route::middleware('guest')->group(function () {
 Route::get('/register', [AuthController::class, 'showRegister'])-
         >name('register');
 Route::post('/register', [AuthController::class, 'register']);
Route::get('/login', [AuthController::class, 'showLogin'])-
>name('login');
 Route::post('/login', [AuthController::class, 'login']);
});

// Halaman yang hanya dapat diakses setelah login
Route::middleware('auth')->group(function () {
 Route::post('/logout', [AuthController::class, 'logout'])-
         >name('logout');

 // CRUD Kontak dengan upload foto
 Route::resource('kontaks', KontakController::class);
});
```

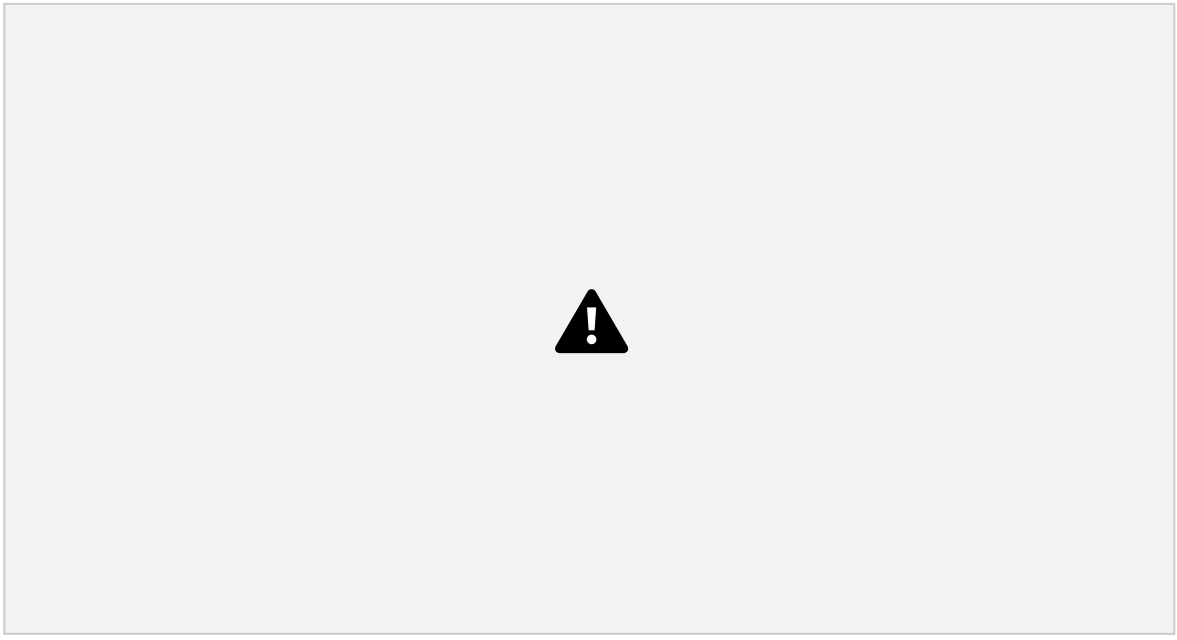
- Route::middleware('guest')->group(...) → halaman register dan login hanya dapat dibuka oleh pengguna yang belum login; pengguna yang sudah login otomatis dialihkan ke halaman awal.
- ->name('login') → nama route login wajib ada, karena middleware auth secara otomatis mengarahkan tamu ke route bernama login.
- Route::middleware('auth')->group(...) → seluruh route kontaks dan logout hanya dapat diakses setelah login.
- Route::post('/logout') → logout menggunakan method POST (bukan GET) agar terlindungi token CSRF.



Gambar 7.64 Route register, login, logout, dan middleware auth pada routes/web.php

4. Periksa route yang sudah terdaftar dengan perintah berikut:

```
php artisan route:list --except-vendor
```



Gambar 7.65 Hasil php artisan route:list --except-vendor

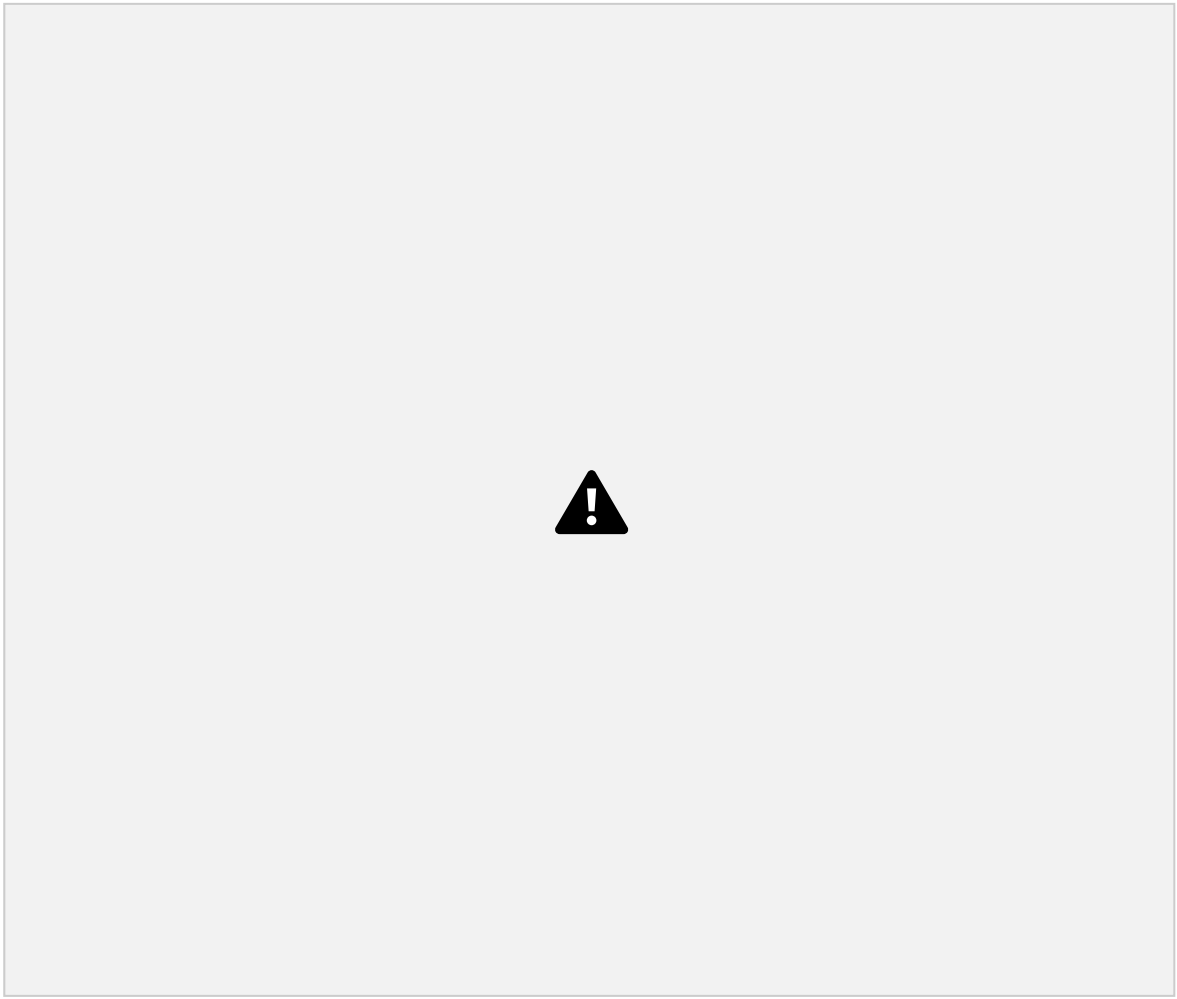
Selain tujuh route kontaks, kini terdaftar route login, register (GET dan POST), serta logout.

5. Buka file resources/views/layouts/app.blade.php (Tahap A langkah 7), lalu ganti bagian navbar menjadi seperti berikut:

```
{{-- Navbar (menu bagian atas website) --}}
 <nav class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">
<div class="container">
 <a class="navbar-brand"
        href="{{ route('kontaks.index') }}">CRUD Kontak</a>
 <ul class="navbar-nav ms-auto align-items-center">  @auth
 {{-- Menu untuk pengguna yang sudah login --}}  <li
class="nav-item">
 <a class="nav-link"
        href="{{ route('kontaks.index') }}">
 <i class="fas fa-address-book me 1"></i>Kontak
 </a>
 </li>
                    <li class="nav-item">
  <span class="nav-link text-white"><i class="fas  fa-user me-1"></i>{{
                      Auth::user()->name }}</span>
 </li>
                    <li class="nav-item">
 <form action="{{ route('logout') }}"  method="POST">
 @csrf
                              <button type="submit" class="btn btn-light
        btn-sm ms-2">
 <i class="fas fa-sign-out-alt me 1"></i>Logout
 </button>
```

```
 </form>
 </li>
 @else
 {{-- Menu untuk tamu (belum login) --}}  <li
class="nav-item">
 <a class="nav-link"
         href="{{ route('login') }}"><i class="fas fa-sign-in-alt me
         1"></i>Login</a>
 </li>
                    <li class="nav-item">
 <a class="nav-link"
         href="{{ route('register') }}"><i class="fas fa-user-plus me
         1"></i>Register</a>
 </li>
 @endauth
 </ul>
 </div>
 </nav>
```

• @auth ... @else ... @endauth → menampilkan menu Kontak, nama pengguna, dan tombol Logout bagi pengguna yang sudah login, serta menu Login dan Register bagi tamu. • Auth::user()->name → menampilkan nama pengguna yang sedang login. • Tombol Logout dibuat di dalam form method POST yang berisi @csrf, sesuai route logout.



Gambar 7.66 Navbar dengan menu login dan logout pada layouts/app.blade.php

6. Buat folder resources/views/auth, lalu buat file

resources/views/auth/register.blade.php: `@extends('layouts.app')`

```
@section('title', 'Register')

@section('content')
<div class="row justify-content-center">
 <div class="col-md-6">
 <div class="card">
 <div class="card-header">Register Akun</div>  <div
class="card-body">
 <form action="{{ route('register') }}" method="POST">  @csrf

 <div class="mb-3">
 <label for="name" class="form
         label">Nama</label>
 <input type="text" id="name" name="name"  value="{{ old('name')
```

```
        }}"
  class="form-control @error('name') is invalid @enderror" required
                                autofocus>
  @error('name') <div class="invalid feedback">{{ $message
                    }}</div> @enderror
 </div>

 <div class="mb-3">
  <label for="email" class="form label">Email</label>
 <input type="email" id="email" name="email"  value="{{
        old('email') }}"
 class="form-control @error('email') is invalid @enderror" required>
  @error('email') <div class="invalid feedback">{{ $message
                    }}</div> @enderror
 </div>

 <div class="mb-3">
     <label for="password" class="form label">Password
               (minimal 8 karakter)</label>
 <input type="password" id="password"  name="password"
 class="form-control @error('password')  is-invalid @enderror"
        required>
  @error('password') <div class="invalid feedback">{{ $message
                     }}</div> @enderror
 </div>

 <div class="mb-3">
 <label for="password_confirmation" class="form label">Konfirmasi
        Password</label>
 <input type="password"
        id="password_confirmation"
name="password_confirmation"  class="form-control" required>
</div>

          <button type="submit" class="btn btn
               primary">Register</button>
 <a href="{{ route('login') }}" class="ms-2">Sudah  punya akun?
        Login</a>
 </form>
 </div>
 </div>
 </div>
</div>
@endsection
```